PARADIGMAS DE LINGUAGENS COMPUTACIONAIS

Miguel Enrique Parra Munoz

EMENTA

Conceitos de Linguagens de Programação. Métodos de Programação. Descrição formal de linguagens de programação envolvendo sintaxe e semântica. Princípios de construção de compiladores. Princípios de Construção de Linguagens. Programação Imperativa. Programação Lógica. Programação Funcional. Programação Orientada a Objetos. Programação Concorrente.

6º Semestre - Ciência da Computação

Horário	Segunda	Terça	Quarta	Quinta	Sexta
08h às 09h40	**MODELOS E ALGORITMOS DE OTIMIZAÇÃO** SÉRGIO QUEIROZ	**PARADIGMAS DE LINGUAGENS COMPUTACIONAIS** MIGUEL MUÑOZ	**SISTEMAS OPERACIONAIS** JEREMIAS GOMES	**PARADIGMAS DE LINGUAGENS COMPUTACIONAIS** MIGUEL MUÑOZ	**MODELOS E ALGORITMOS DE OTIMIZAÇÃO** SÉRGIO QUEIROZ
	Sala 106 (1º andar)	Sala 106 (1º andar)	Lab Windows (Térreo)	Sala 106 (1º andar)	Sala 106 (1º andar)
10h às 11h40	**SISTEMAS OPERACIONAIS** JEREMIAS GOMES	**OFICINA EM CIÊNCIA DA COMPUTAÇÃO** LEONARDO FREITAS	**COMPILADORES**	**OFICINA EM CIÊNCIA DA COMPUTAÇÃO** LEONARDO FREITAS	**COMPILADORES**
	Lab Windows (Térreo)	Sala 106 (1º andar)	Sala 106 (1º andar)	Sala 106 (1º andar)	Sala 106 (1º andar)
14h às 15h40	**OFICINA EM IMPRESSÕES 3D** LETÍCIA BARROS **Turma "B"**			**OFICINA EM IMPRESSÕES 3D** LETÍCIA BARROS **Turma "B"**	
	Idea Lab (Térreo)			Idea Lab (Térreo)	

PLANEJAMENTO

Cálculo da média: Média Intermediária (MInt) = 0,5*A + 0,5*Atividades

- **A1: Nota do primeiro bimestre – de 0 a 10**

Composição de A1:

50%: Prova 1 – escrita e individual (A1)

50% Atividades, laboratórios e tarefas

- **A2: Nota do segundo bimestre – de 0 a 10**

Composição de A1:

50%: Prova 1 – escrita e individual (A1)

50% Atividades, laboratórios e tarefas

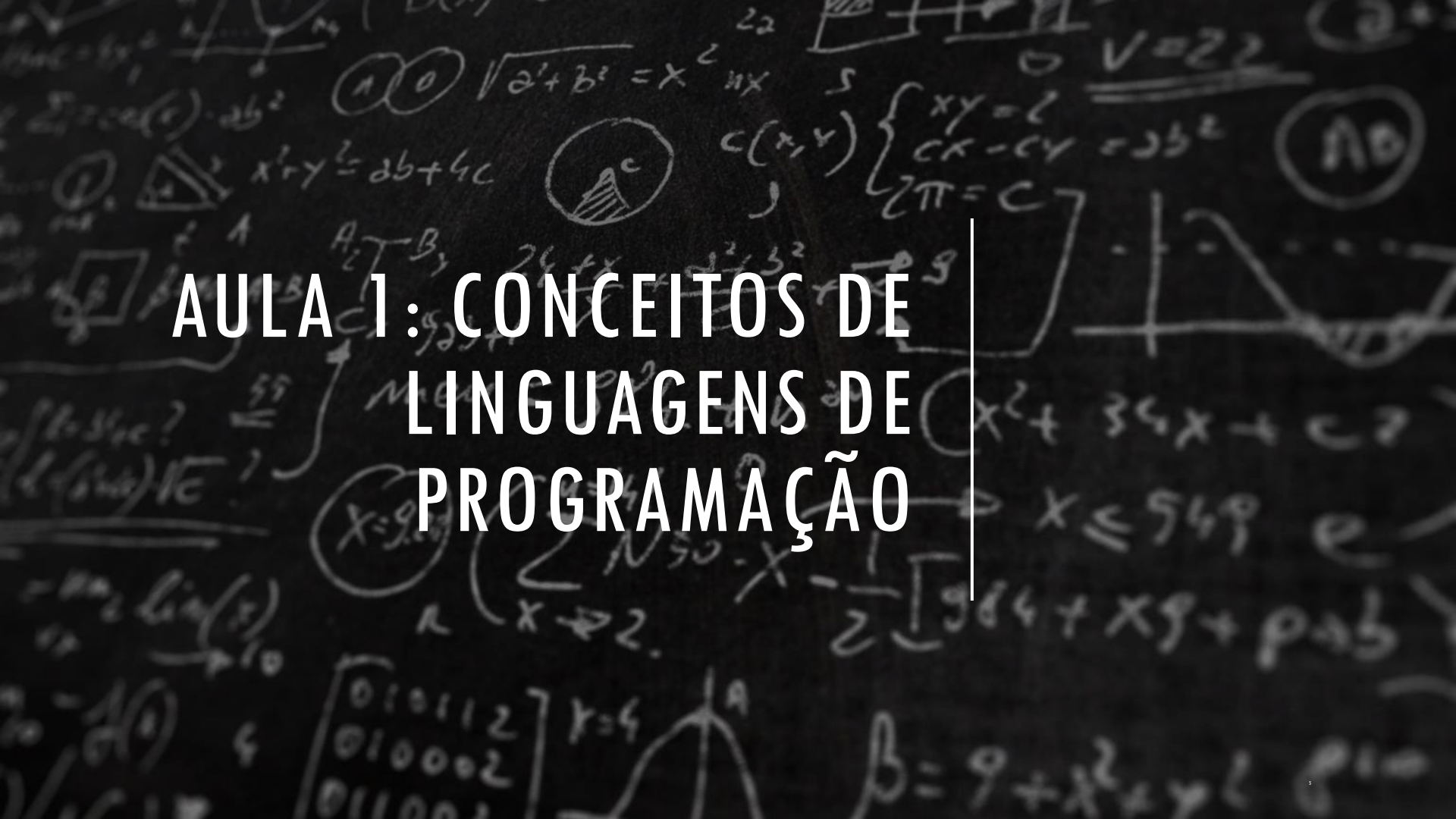
Cálculo da nota Final:

$NF = \frac{(A_1 + A_2)}{2}, 0 \leq NF \leq 10$

Datas das avaliações:

- A1: 01 de Outubro.
- A2: 26 de Novembro.
- A3: 08 de Dezembro.

AULA	DATA	CONTEÚDO
		Agosto
1	04/08/26	Apresentação da disciplina e metodologia de avaliação
2	06/08/26	Conceitos de linguagem de programação
3	11/08/26	Conceitos de linguagem de programação
4	13/08/26	Atividade valendo nota
5	18/08/26	Apresentação da plataforma de programação, introdução as sentencias em C, Criação de digramas de fluxo
6	20/08/26	Atividade valendo nota
7	25/08/26	Métodos de programação
8	27/08/26	Atividade valendo nota
		Setembro
9	01/09/26	Descrição formal de linguagens de programação envolvendo sintaxe e semântica
10	03/09/26	Atividade valendo nota
11	08/09/26	Princípios de construção de compiladores
12	10/09/26	Atividade valendo nota
13	15/09/26	Princípios de Construção de Linguagens
14	17/09/26	Atividade valendo nota
15	22/09/26	Programação Imperativa
16	24/09/26	Atividade valendo nota
17	29/09/26	Atividade valendo nota
		Outubro
18	01/10/26	Aplicação da Atividade Avaliativa 1
19	06/10/26	Devolução da Atividade Avaliativa 1
20	08/10/26	Programação Lógica.
21	13/10/26	Atividade valendo nota
22	15/10/26	Programação Lógica.
23	20/10/26	Atividade valendo nota
24	22/10/26	Programação Funcional



AULA 1: CONCEITOS DE LINGUAGENS DE PROGRAMAÇÃO

O QUE É UM COMPUTADOR?

Um computador é um dispositivo capaz de realizar computações e tomar decisões lógicas milhões (e até bilhões) de vezes mais rapidamente que o homem. Muitos computadores realizam um bilhão de adições por segundo!

Uma pessoa operando uma calculadora de mesa pode gastar a vida toda para fazer cálculos e ainda assim não concluir a mesma quantidade de cálculos que um poderoso computador pessoal (PC) pode realizar em um segundo.



INTRODUÇÃO

Programação está se tornando (se já não for) um pré-requisito para qualquer pessoa do século 21. Cada dia que passa mais e mais de nossas atividades cotidianas e trabalhos são facilitados com a ajuda de softwares ou até inteiramente automatizadas por eles.

Ao contrário do que muita gente pensa, **programação não é um bicho de sete cabeças**! Se fosse fazer uma comparação, diria que é como aprender um idioma diferente, mas talvez seja até mais simples. Para os computadores não existe subjetividade…ou é ou não é… ou é zero ou é um!

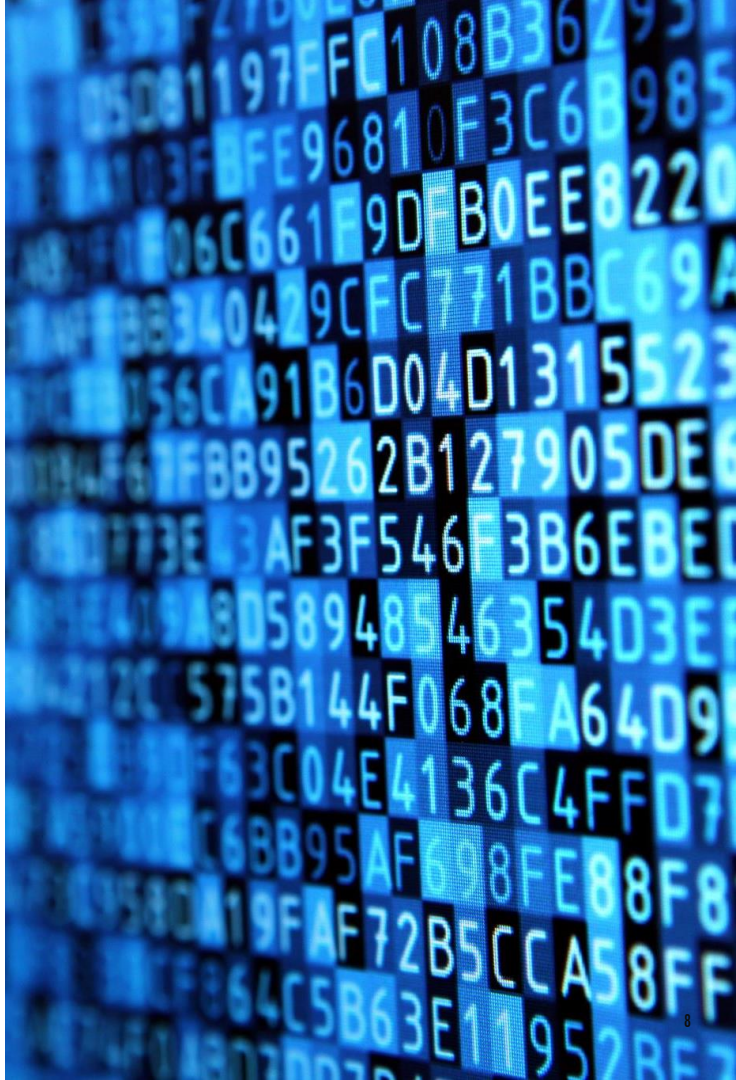
Como tudo na vida (e Newton já sabia disso…), o **conhecimento precisa ser construído de forma incremental** para que com o tempo possamos compreender assuntos mais complexos. Assim, pretendo **apresentar a você os conceitos básicos de programação** sem os formalismos do meio acadêmico. Isto é um ponto de partida e o objetivo é que você entenda as ideias! Falarei sobre todos os conceitos básicos? Não, pois tem muito conceito! Contudo, falarei sobre o que você precisa saber neste momento para iniciar sua jornada de aprendizado. Conte comigo e com a comunidade para tirar suas dúvidas por meio do nosso fórum ou dos demais canais.

O QUE É UM COMPUTADOR?

Questões para ponderar:

Como saber se a pessoa somou os números corretamente?

Como saber se o computador somou os números corretamente?



O QUE É UM COMPUTADOR?

Os computadores processam dados sob o controle de conjuntos de instruções chamados programas de computador.

Esses programas orientam o computador por meio de conjuntos ordenados de ações especificadas por pessoas chamadas programadores de computador.



O QUE É UM COMPUTADOR?

Um computador consiste de vários dispositivos referidos como hardware: unidades de processamento

- teclado
- mouse
- monitor
- memória (RAM / SSD) etc

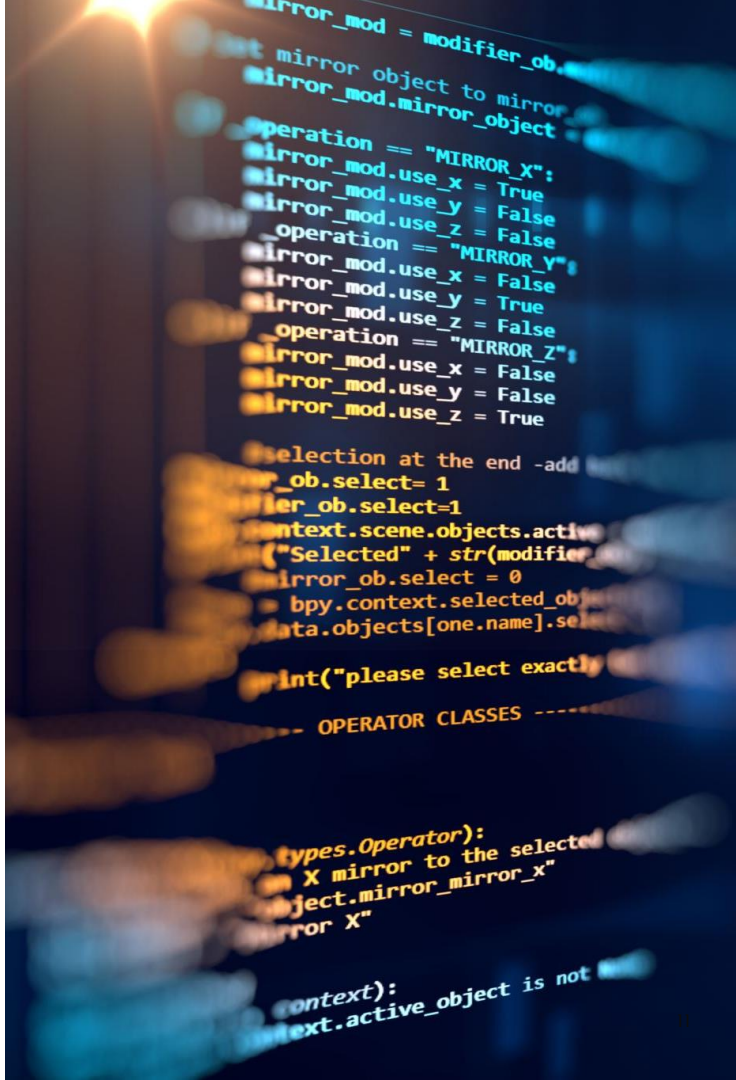
Os programas executados em um computador são referidos como software.



O QUE É LÓGICA DE PROGRAMAÇÃO?

A lógica de programação é a **base por trás da escrita de qualquer código**, fornecendo a estrutura necessária para a resolução de problemas e para a execução de tarefas em um algoritmo. Por meio da construção de **instruções, comandos e sequências lógicas**, como as funções, repetições e condições, o algoritmo consegue **transformar problemas complexos em pequenas tarefas gerenciáveis** e resolver esses problemas de acordo com sua programação.

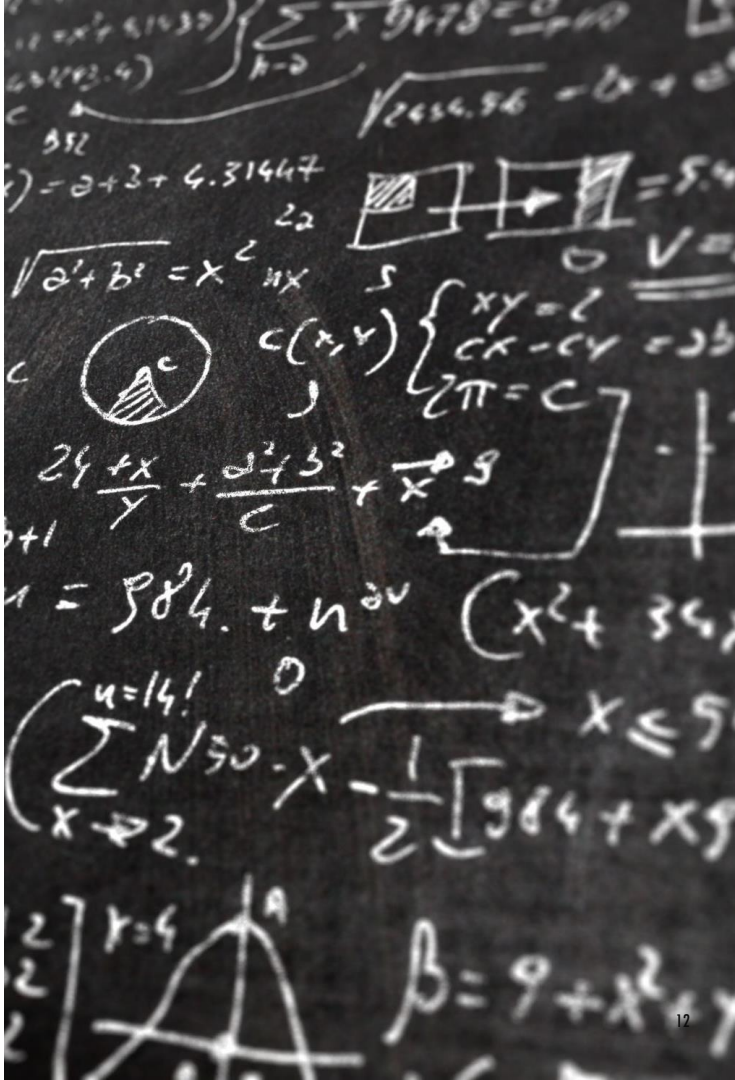
Assim, a programação envolve escrever um roteiro detalhado em uma linguagem que o computador entenda (código) para executar tarefas e ações para as quais foi programado.



O QUE SÃO ALGORITMOS?

A noção de algoritmo foi identificada formalmente pela primeira vez na tradução de trabalhos de um matemático persa do século XII. Fique tranquilo porque não vamos aprofundar na história do desenvolvimento humano! Queria apenas ressaltar que o conceito de algoritmo surgiu quando não existia sequer a ideia de computadores e, no entanto, é amplamente empregado na área de computação.

Para ser objetivo, um **ALGORITMO é uma sequência finita de ações que visam resolver um problema.** Esta **sequência de ações deve ser PRECISA** (dizer exatamente o que precisa ser feito), não pode ser ambígua (não pode haver dúvidas sobre o que cada ação significa), **CORRETA** (deve resolver o problema) e, de preferência, **EFICIENTE** (quanto menos ações ou recursos forem exigidos para resolver o problema, melhor!).



TIPOS DE DADOS

Já os tipos de dados referem-se às **informações inseridas no programa.** É tudo aquilo que adiciona-se no algoritmo, fornecendo uma base para a eficiência do programa. Os dados podem vir em diversos formatos, como os dados primitivos ou os dados estruturados.

Por exemplo, os **dados primitivos podem ser números em diferentes formatos**, **caracteres**, **strings**, **booleanos**, etc. Já os dados estruturados, podem vir em formatos de *arrays* (como uma lista de números), classes e outros tipos de estruturas.

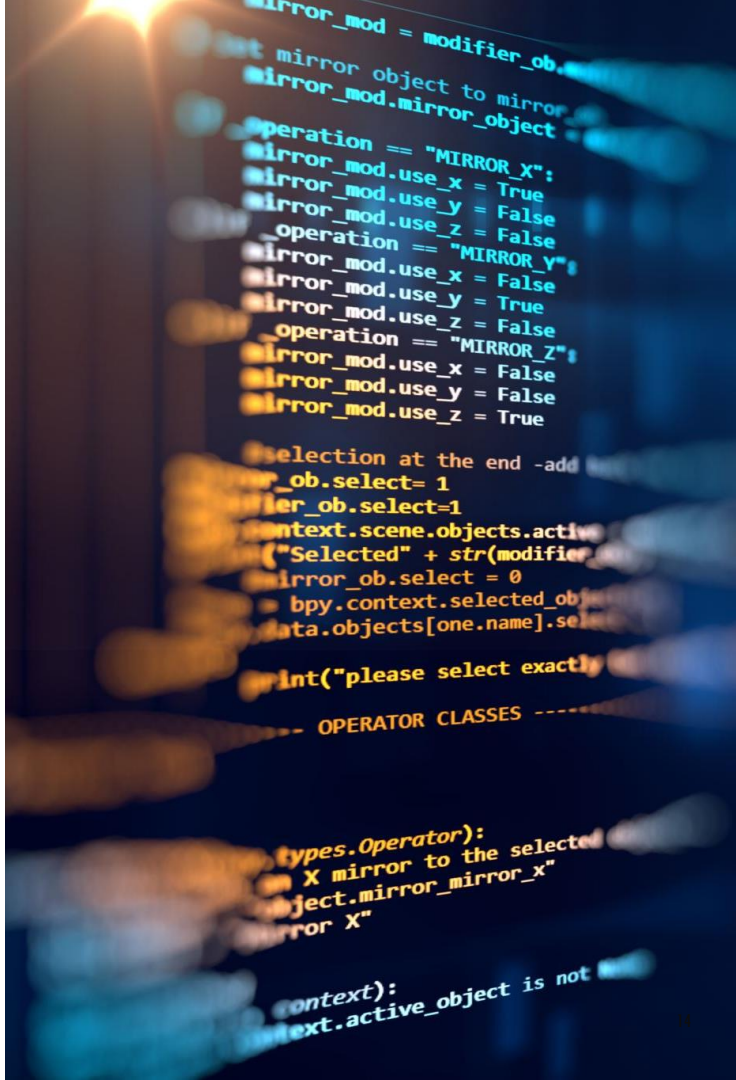
No geral, os **dados estruturados representam um conjunto de dados primitivos que são conectados**. Por exemplo, para representar um usuário do produto, pode ser útil juntar o nome, a idade e o e-mail dele em uma estrutura única. Assim, os tipos de dados são essenciais para o funcionamento adequado de um algoritmo.



VARIÁVEIS E CONSTANTES

As variáveis e constantes também são conceitos cruciais de programação básica, pois através destes elementos pode-se definir a **natureza da organização** e **manipulação dos dados de um algoritmo.**

Variáveis são **dados que podem mudar** ao longo da execução do programa, como por exemplo, as horas em um relógio, que vão mudar com o passar do dia. Já as constantes, **são valores imutáveis**, que uma vez inseridos no programa, permanecerão iguais, como por exemplo, a data de nascimento de um usuário. No processo de desenvolvimento, é importante entender quais dados serão constantes e quais não, já que isso influencia a escrita, a performance e a segurança do programa.



CONDIÇÕES E REPETIÇÕES

As condições permitem que o algoritmo tome decisões a partir de critérios estabelecidos. Ele executará diversos blocos de código considerando diferentes situações possíveis. Por exemplo, um programa de notas pode estabelecer que o critério para a aprovação é obter um resultado a partir de 70. Assim, qualquer valor abaixo desse critério, será reprovado.

Já as **repetições são blocos de código compostos por uma condição**. Eles podem ser executados várias vezes enquanto a condição estabelecida for verdadeira. Por exemplo, no programa de notas mencionado acima, uma repetição seria necessária para analisar individualmente as notas de todos os alunos. Isso evita que o programador tenha que escrever as mesmas instruções para cada aluno.



FUNÇÕES

As funções são essenciais para a organização, manutenção e modularização do código. Elas são blocos de código reutilizáveis que realizam uma tarefa específica. As funções podem ser chamadas em diversas partes do programa, aceitando valores de *input* e retornando outros valores de *output*.

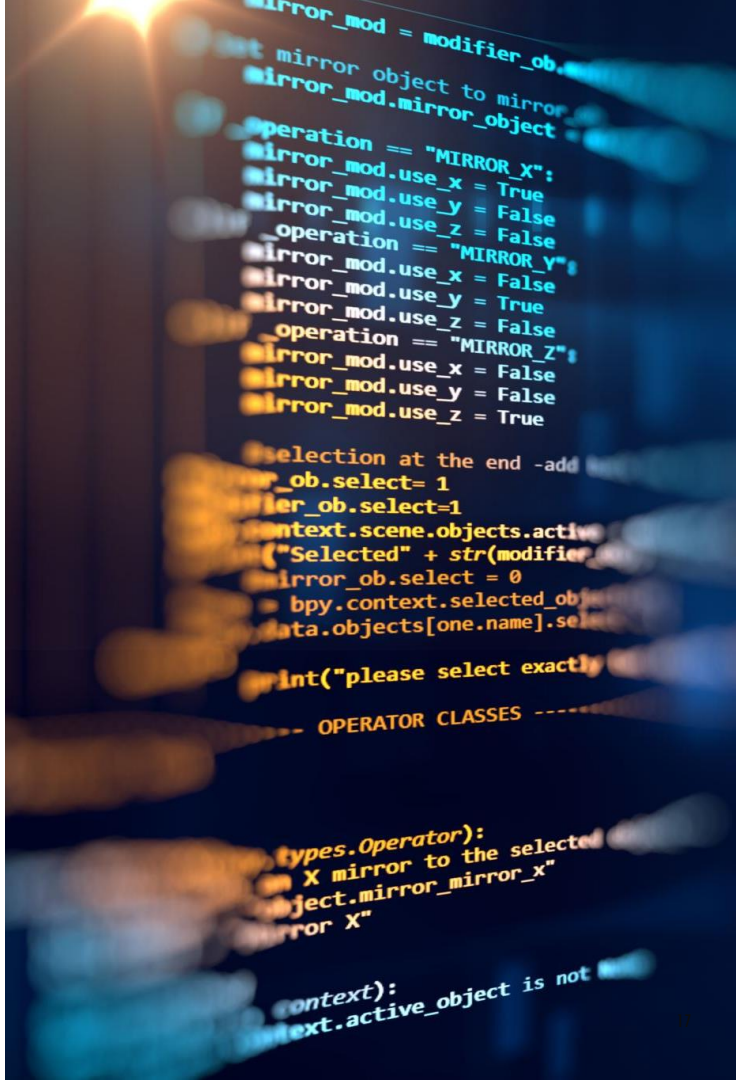
Por exemplo, uma função de soma numérica, aceitará dois números como *input* e retornará um número como *output*, sendo ele a soma dos dois *inputs*. Essa função pode ser reutilizada em qualquer ponto do programa.



PRINCIPAIS LINGUAGENS DE PROGRAMAÇÃO E APLICAÇÕES

A área de programação é vasta e possui diversas linguagens de programação, com múltiplas finalidades. Quando falamos no desenvolvimento de produtos digitais, as principais linguagens de programação são:

- Python;
- JavaScript, HTML e CSS;
- Java;
- Swift;
- Kotlin.



PYTHON

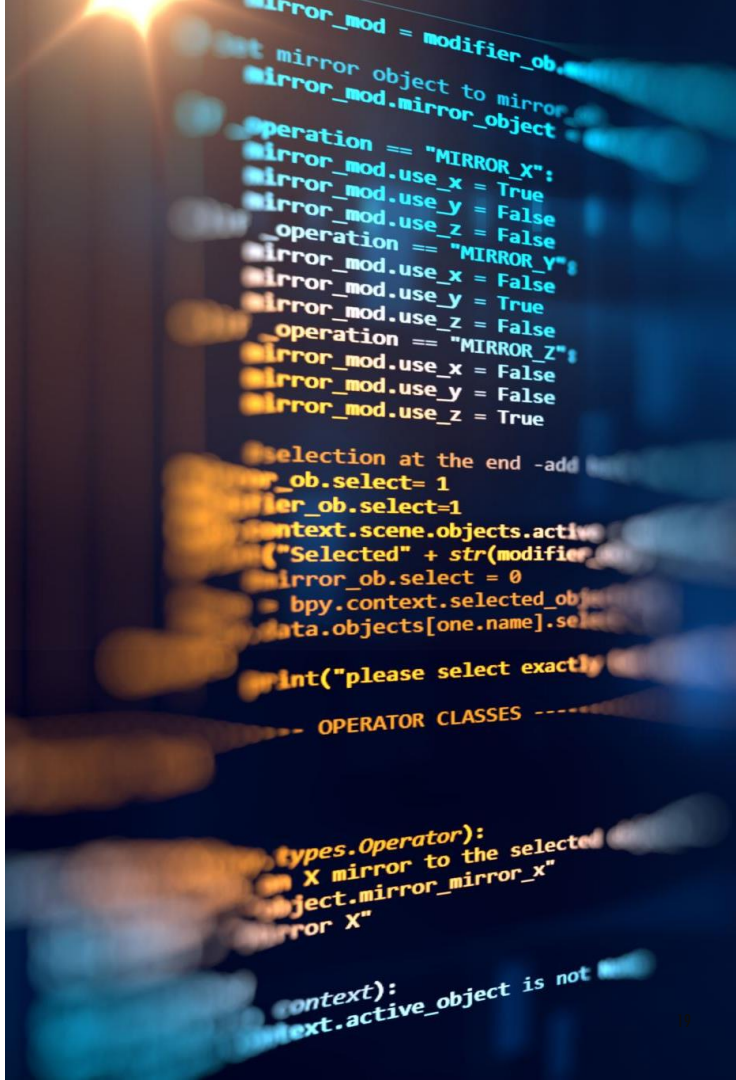
Python é uma linguagem de programação muito versátil e popular, criada não só para desenvolvimento web, mas também para outros objetivos. Ela é muito utilizada na análise de dados com Python, automação de tarefas, implementação de funcionalidades, *machine learning*, ciência de dados, entre outros. Essa linguagem é **fácil de aprender, ler e executar,** pois possui uma **sintaxe descomplicada**, sendo uma opção ideal para iniciantes que queiram aprender programação básica.

JAVASCRIPT, HTML E CSS

JavaScript, HTML e CSS são as **principais linguagens de desenvolvimento web** e geralmente são usadas em conjunto para **criar interfaces interativas e responsivas na web.**

HTML é uma linguagem usada para estruturar os elementos de uma página, como cabeçalhos, links, imagens, parágrafos, etc. Já CSS é usada para gerenciar os elementos visuais da página, como layout, cores, tipografia, etc. JavaScript, por outro lado, é o **verdadeiro motor que torna um site dinâmico, funcional e responsivo para o usuário**, criando interfaces interativas e fazendo integrações de APIs de maneira eficiente.

Embora JavaScript tenha como foco o desenvolvimento *front-end* do site (a parte que o usuário interage), nos últimos tempos, a linguagem tem sido usada também para *back-end* (o servidor com que o site comunica). Essas três linguagens são a base do desenvolvimento web. Por isso, adquirir esses conhecimentos pode ser muito útil para entender aspectos de programação web e comunicar-se com desenvolvedores sobre as decisões de produto.



JAVA

Java é uma linguagem muito conhecida no mundo da programação principalmente por sua **escalabilidade e flexibilidade.** Ela é robusta e de alto desempenho, permitindo desenvolver vários tipos de aplicações, como aplicativos mobile, websites, jogos e ser executada em variadas plataformas como Windows, Android, Linux e macOS. Sendo assim, é uma linguagem muito vasta e versátil e aprendê-la proporciona um conhecimento muito útil que apoia a tomada de decisões no desenvolvimento de produtos.



SWIFT

Swift é uma linguagem de programação focada no **desenvolvimento de aplicativos mobile** em sistemas operacionais como macOS e iOS. Criada pela Apple, ela possui uma sintaxe descomplicada e eficiente, mas ao mesmo tempo é robusta e otimizada para o alto desempenho, oferecendo modernidade, segurança e confiabilidade. Entender o funcionamento e a base do Swift pode ser útil para colaboração com desenvolvedores de aplicativos mobile.



KOTLIN

Kotlin é a versão do Swift voltada para o sistema Android. Ela é integrada e interoperável com a linguagem Java, o que facilita muito a migração gradual de projetos em Java para Kotlin. Kotlin é a linguagem de preferência do Google para o desenvolvimento de aplicativos Android, principalmente por ser **moderna** e **oferecer uma sintaxe clara**, além de ser **altamente segura** e **confiável.**

Assim como Swift, é útil para a colaboração de profissionais de Produto em projetos de sistemas operacionais iOS, o Kotlin é o que vai te ajudar a entender o funcionamento técnico de aplicativos Android.



COMO APRENDER PROGRAMAÇÃO?

Aprender programação básica é o primeiro passo para **ultrapassar a barreira da comunicação** entre PMs e desenvolvedores. Você não precisa dominar completamente uma linguagem ou saber todos os termos técnicos, mas entender a base em que se fundamenta a programação de software é indispensável para melhorar os processos de desenvolvimento de um produto e trazer mais efetividade na tomada de decisões.

Algumas dicas para ajudar nessa jornada de aprendizagem são:

- **Comece pelas linguagens de programação mais fáceis**, como Python, JavaScript, HTML e CSS. Esses conhecimentos vão te ajudar a entender o universo da programação e fornecer uma base para que, no futuro, você possa aprender outras linguagens;
- **Utilize recursos de aprendizagem** como vídeos, tutoriais, livros, cursos livres e bootcamps de programação para aprimorar seus conhecimentos e habilidades. Um recurso interessante são as IAs, como o ChatGPT, que tem auxiliado significativamente o trabalho de desenvolvedores;
- **Engaje-se na comunidade de desenvolvedores.** A comunidade de programadores é muito consolidada e oferecem recursos, suporte e um ambiente de discussões para que os iniciantes na área possam ter trocas significativas com outros profissionais. Uma ferramenta referência na área é o StackOverFlow, uma plataforma de perguntas e respostas sobre o universo da programação que permite a interação entre diversos profissionais, desde iniciantes até sêniores;
- **Coloque seus conhecimentos em prática.** Crie projetos reais, mesmo que sejam pessoais, para testar seus conhecimentos e adquirir experiência prática.

ESTRUTURAS DE DADOS

Se tem algo que não podemos deixar de falar quando falamos sobre algoritmos são as **ESTRUTURA DE DADOS.** As estruturas de dados são **mecanismos para armazenar etapas e resultados intermediários ao longo da execução do algoritmo,** ou até mesmo a solução final do problema.



LINGUAGEM DE PROGRAMAÇÃO

Como vimos até agora, os **algoritmos e estrutura de dados existem independentemente da linguagem de programação** e este é o conceito que abordaremos agora. O que é linguagem de programação? Senta que lá vem um pouco de história…mas espero que seja interessante para você!

Linguagem de programação é a forma pela qual você se comunicará com o seu computador. O computador é extremamente veloz, consegue guardar muita informação…mas tem um problema…ele só faz o que você ordena ele fazer!

No início da computação, as linguagens eram muito rudimentares, ou em termos mais técnicos, de baixo nível. Isso significa que o programador precisava transcrever seus algoritmos em uma sequência de 0 e 1, no que denominamos código de máquina. Tipo isso:

0000101110101110111000011101100001101110101000110101101101101001010101101010101.....e por aí vaí milhares de zeros e uns.

LINGUAGEM DE PROGRAMAÇÃO

Eram tempos difíceis para a comunicação homem-máquina…ainda bem que a tecnologia permitiu o incremento do poder de processamento dos computadores e a sua capacidade de memória. Com o tempo vieram os cartões perfurados, depois começaram a surgir linguagens que permitiam comandos em palavras de 3 letras, tipo: ADD (soma), MOV (move um número de uma posição de memória para outra), STO (grava isso na memória), etc. E de forma gradativa, a computação foi evoluindo para chegar nas linguagens de alto nível, que são as linguagens que temos contato hoje!

Intitulamos as **linguagens de alto nível** aquelas linguagens que nos **permitem abstrair do que realmente acontece dentro do computador**…ou seja não utilizamos comandos muito primitivos, como aqueles zeros e uns ou comandos com três letras…Em analogia, é como pedir para você escrever o seu nome numa folha…eu não precisaria te dizer letra a letra e como fazer cada letra para escrevê-lo. Eu simplesmente peço, escreva seu nome e você o fará de uma forma que eu não preciso saber como.

Tem muitas linguagens diferentes, elas podem ter propósito geral, tais como Pascal, Java, C, C++, Pyhton etc, com as quais você pode fazer qualquer tipo de programa, ou pode ser desenvolvida para um fim específico, por exemplo, renderizar as páginas que você acessa na internet (HTML).

ORGANIZAÇÃO DO COMPUTADOR

Praticamente, todos os computadores podem ser considerados como divididos em seis unidades lógicas:

Entrada

Saída

Armazenamento Principal

Armazenamento Secundário

Unidade de Aritmética e Lógica

Unidade Central de Processamento



UNIDADE DE ENTRADA

Esta é a seção “receptora” do computador.

Ela obtém informações (dados e programas de computador) de dispositivos de entrada e coloca essas informações à disposição das outras unidades para o processamento.

A maioria das informações é inserida em computadores por meio de dispositivos de entrada, como teclados, touch screens e mouse.

UNIDADE DE ENTRADA

Outras formas de entrada:

falar com seu computador, digitalizar imagens e códigos de barra;

leitura de dispositivos de memória secundária (HD, DVD, Blu-ray Disc, USB Flash drives);

receber um vídeo de uma webcam ou receber informações de uma rede, como a Internet;

obter dados de posicionamento a partir de um dispositivo de GPS;

coletar informações de movimento e orientação a partir de um acelerômetro em um smartphone;

etc



UNIDADE DE SAÍDA

Esta é a seção de “envio” do computador.

A maioria das informações enviadas para a saída de computadores é exibida em telas, impressas em papel ou utilizadas para controlar outros dispositivos.

Os computadores também podem gerar saída de suas informações para redes, como a Internet, entre outros.

UNIDADE DE ARMAZENAMENTO PRINCIPAL

Esta é a seção de armazenamento de relativamente baixa capacidade e rápido acesso do computador.

Armazena os programas de computador enquanto estão sendo executados. Retém informações que foram inseridas pela unidade de entrada, para se tornarem imediatamente disponíveis para o processamento quando for necessário.

As informações na unidade de memória são voláteis, são perdidas quando o computador é desligado. A unidade de memória costuma ser chamada de memória ou memória principal.



UNIDADE DE ARMAZENAMENTO SECUNDÁRIA

Esta é a seção de armazenamento de alta capacidade e longo prazo do computador (acesso lento).

As informações no armazenamento secundário são persistentes; preservadas quando o computador é desligado. Em geral exigem muito mais tempo para serem acessadas do que as informações na memória principal, mas o custo por unidade de armazenamento secundário é muito menor.

Exemplo: unidades de disco, CDs, DVDs, HDs, etc.



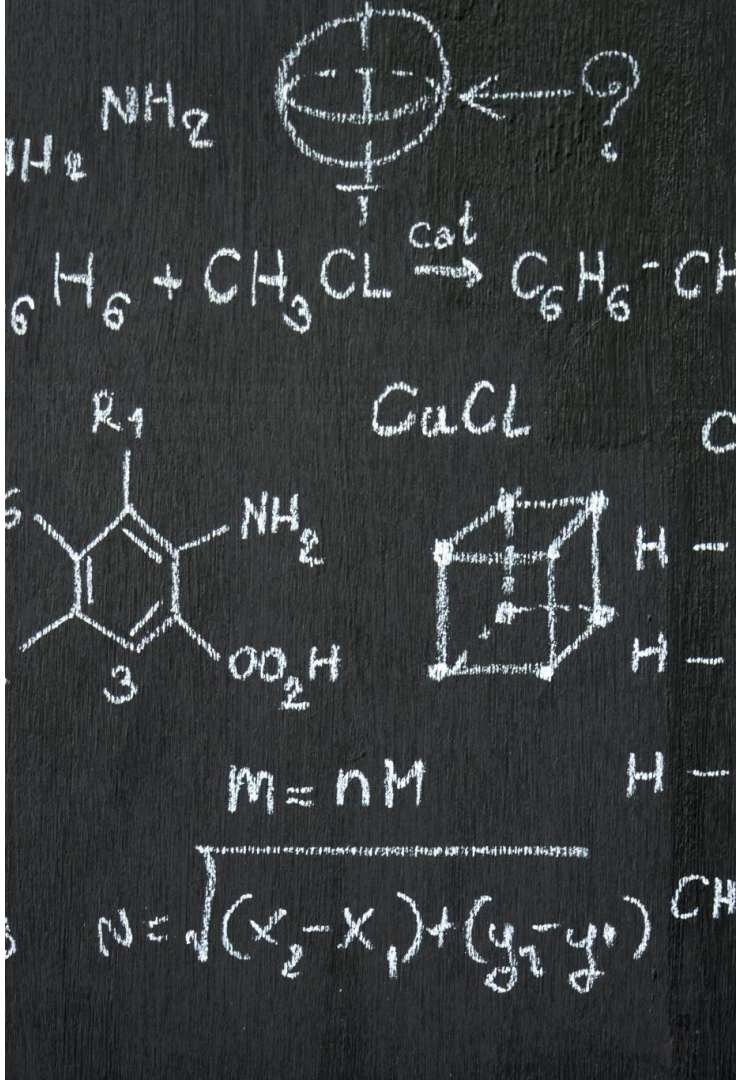
UNIDADE DE ARITMÉTICA E LÓGICA

(ALU – Arithmetic and Logic Unit) Esta é a seção de "produção" do computador.

Ela é responsável pela realização de cálculos, como adição, subtração, multiplicação e divisão.

Contém os mecanismos de decisão que permitem ao computador, por exemplo, comparar dois itens da unidade de memória para determinar se são iguais ou não.

Nos sistemas atuais, a ALU é usualmente implementada como uma parte da CPU.



UNIDADE CENTRAL DE PROCESSAMENTO

(CPU – Central Processing Unit) Esta é a seção “administrativa” do computador.

Ela coordena e supervisiona a operação das outras seções.

A CPU diz à unidade de entrada quando as informações devem se lidas e transferidas para a unidade de memória, informa à ALU quando as informações da unidade de memória devem ser utilizadas em cálculos e instrui a unidade de saída sobre quando enviar as informações da unidade de memória para certos dispositivos de saída.

UNIDADE CENTRAL DE PROCESSAMENTO

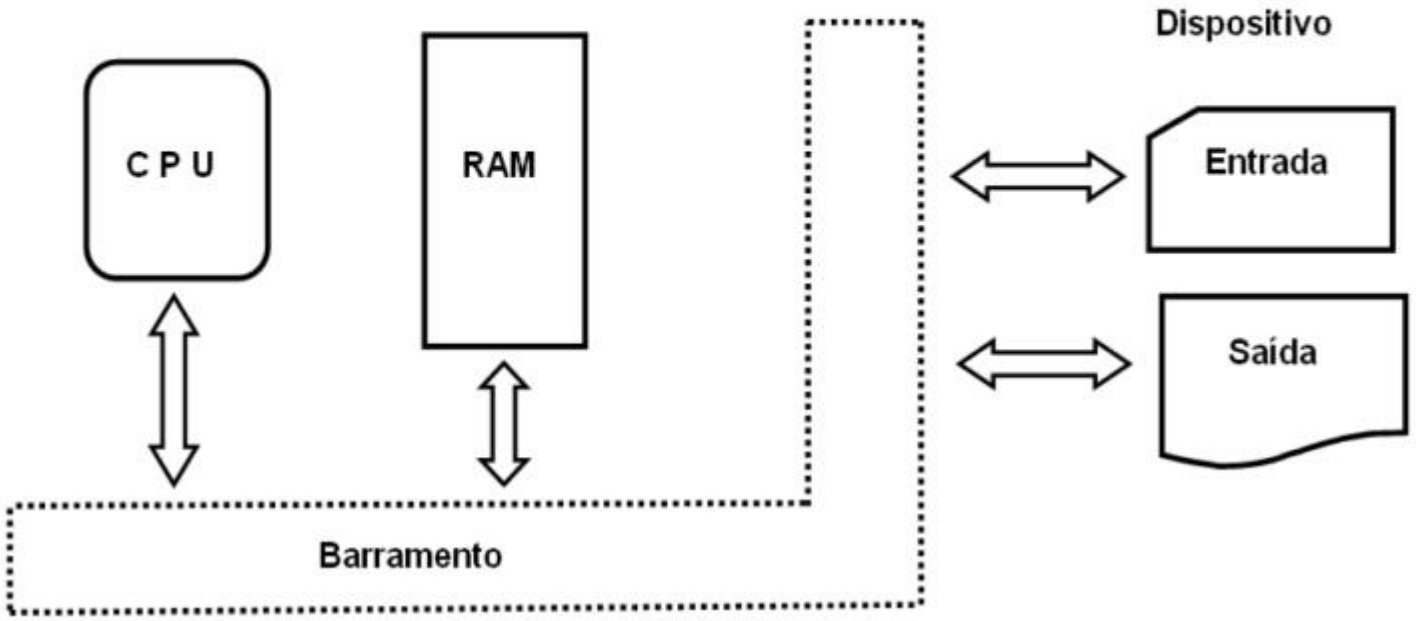
(CPU – Central Processing Unit) Muitos computadores de hoje têm múltiplas CPUs e, portanto, podem realizar muitas operações simultaneamente - esses computadores são chamados de multiprocessados.

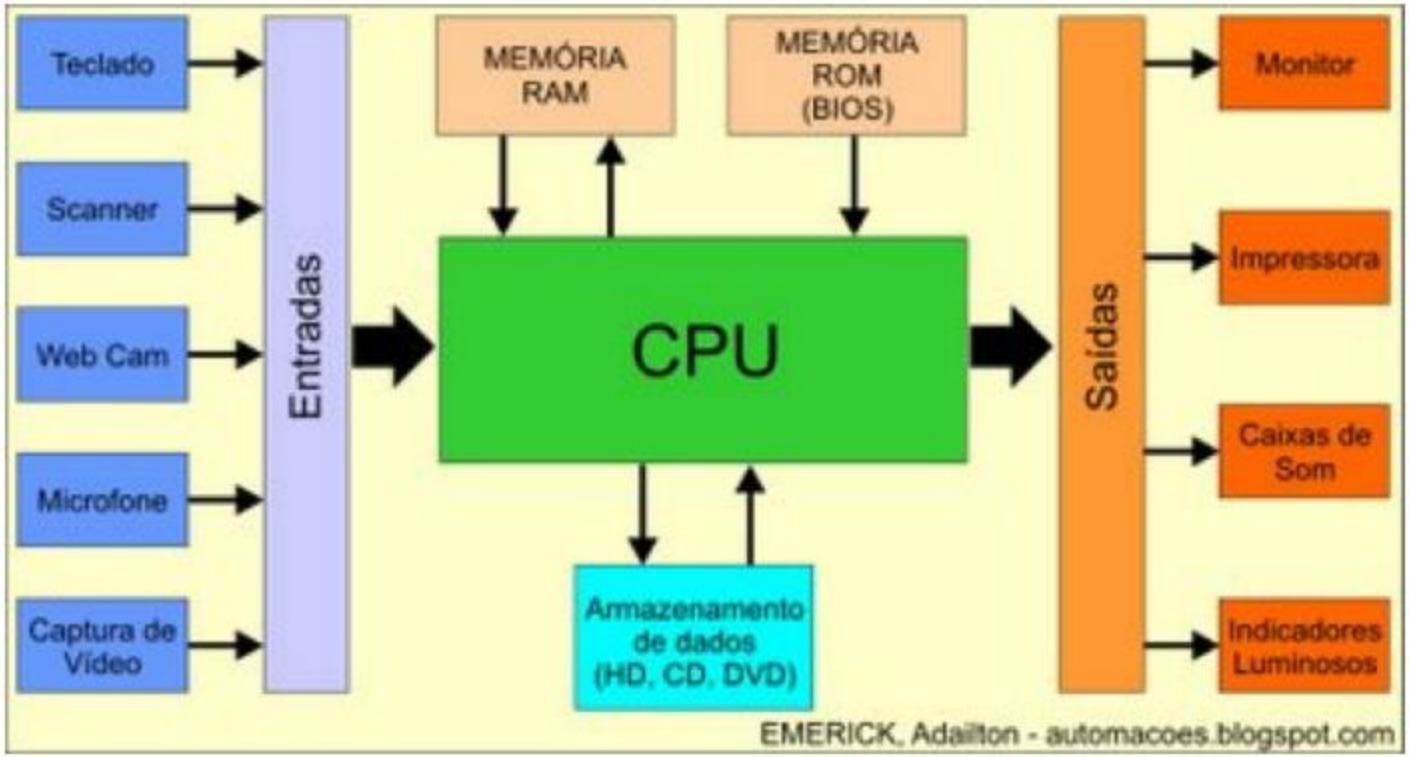
um processador multi-core implementa múltiplos processadores em um único chip de circuito integrado; um processador dual-core possui duas CPUs e um quad-core possui quatro CPUs.

Um computador desktop atual possui processadores que podem executar biliões de instruções por segundo.



A Arquitetura Simplificada de um Computador
(modelo de Von Neumann)

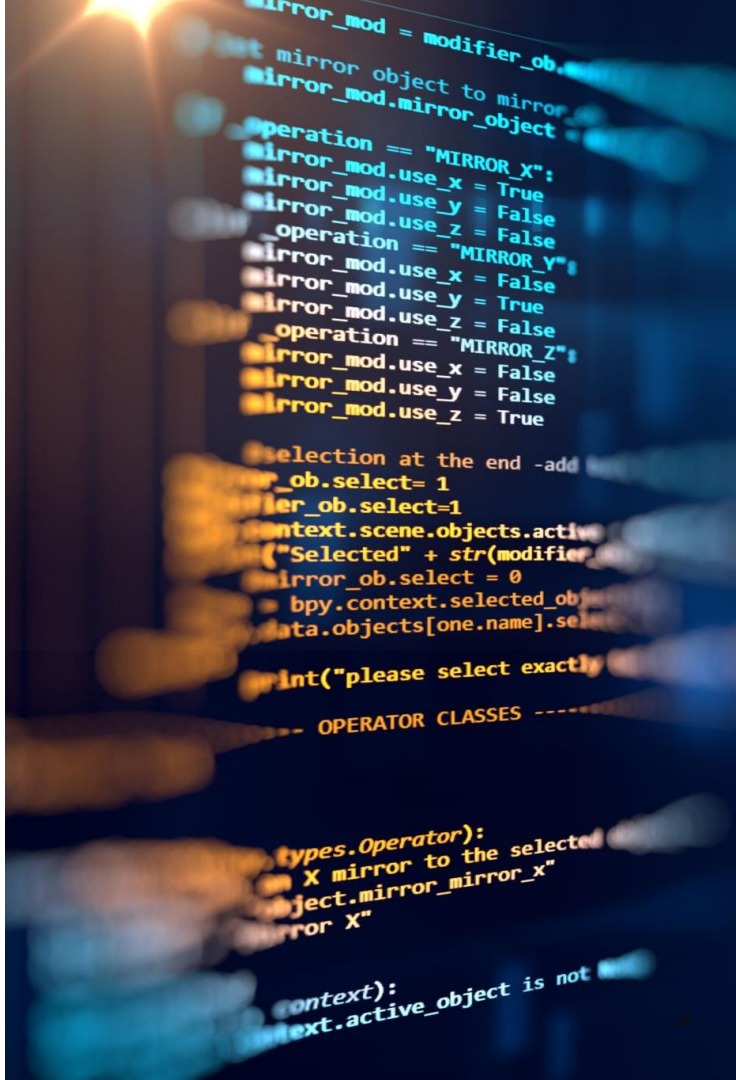



Teclado
Scanner
Web Cam
Microfone
Captura de Vídeo
Entradas
MEMÓRIA RAM
MEMÓRIA ROM (BIOS)
CPU
Armazenamento de dados (HD, CD, DVD)
Saídas
Monitor
Impressora
Caixas de Som
Indicadores Luminosos
EMERICK, Adailton - automacoes.blogspot.com

PROGRAMA E PROGRAMAÇÃO

Programa: geralmente referido como software (instruções escritas para que o computador realize ações e tome decisões).

Programação Estruturada: metodologia de programação constituída por sequências, desvios e repetições de instruções de uma linguagem de programação.



PROGRAMA E PROGRAMAÇÃO

C e C++ são duas das linguagens mais populares para o desenvolvimento de software.

Deitel: usa a versão do C/C++ padronizado nos EUA através do ANSI - American National Standards Institute e mundialmente por meio dos esforços da ISO - International Organization for Standardization



HIERARQUIA DE DADOS

Lembre-se que um computador digital utiliza bits (dados binário: 0 ou 1). Assim, os dados são processados por computadores a partir de uma hierarquia de dados que se torna maior e mais complexa em estrutura, a medida que avançamos de bits para caracteres, depois para campos, e assim por diante.



HIERARQUIA DE DADOS

Bit

É o menor item de dado em um computador, o qual pode assumir o valor 0 ou 1 Binary Digit. É notável como funções importantes de hardware realizam computações por meio da simples mudança do bit 0 para o bit 1, e vice-versa.

Byte

Conjunto de 8 bits



HIERARQUIA DE DADOS

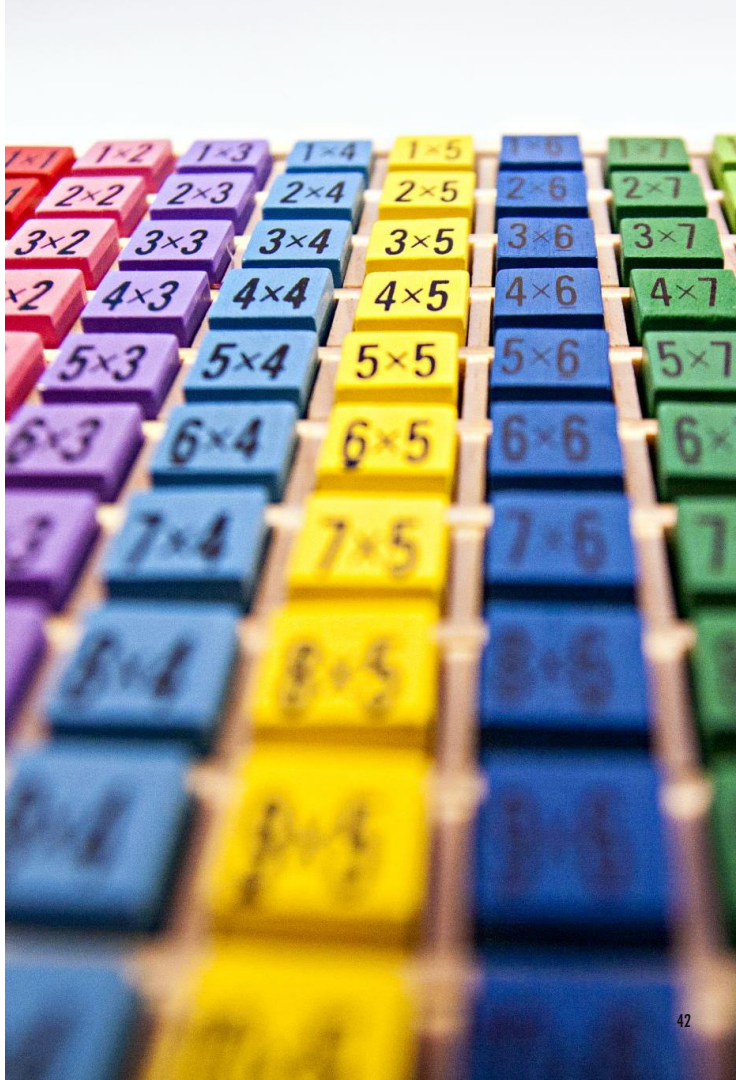
Caractere

É tedioso para o ser humano trabalhar com dados no formato de baixo nível (bit). Preferência: dígitos decimais (0-9), letras (A-Z e a-z), símbolos especiais ($, @, %, #, &, *, (,), +, etc). Caracteres: dígitos, letras e símbolos especiais.

Conjunto de Caracteres do Computador: é o conjunto de todos os caracteres usados para escrever os programas e os itens de dados.

Os computadores processam somente 1s e 0s, assim o conjunto de caracteres do computador representa todo caractere com um padrão de 1s e 0s.

C/C++ usa a codificação ASCII – American Standard Code for Information Interchange.



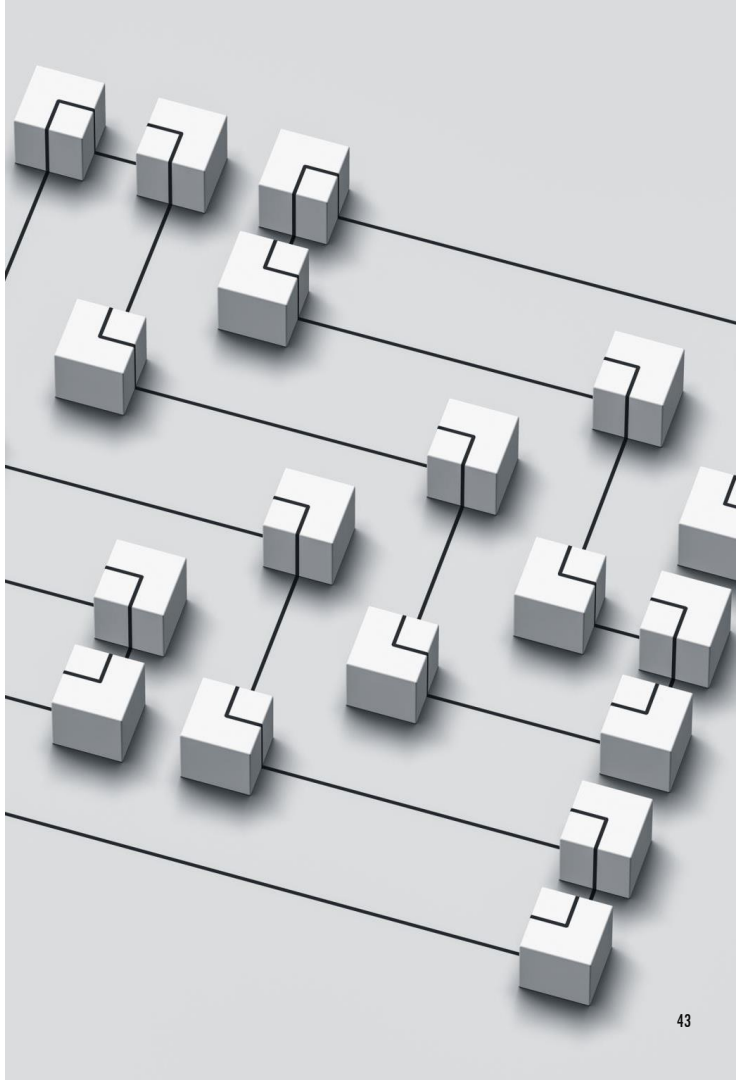
HIERARQUIA DE DADOS

Campo – Field

Da mesma forma que os caracteres são formados por bits, os campos são compostos por caracteres.

Um campo é um grupo de caracteres, ou bytes, que transmite um significado.

Por exemplo, um campo constituído por letras maiúsculas e minúsculas pode ser usado para representar o nome de uma pessoa, e um campo constituído por dígitos decimais poderia apresentar a idade de uma pessoa.



HIERARQUIA DE DADOS

Registro – Record

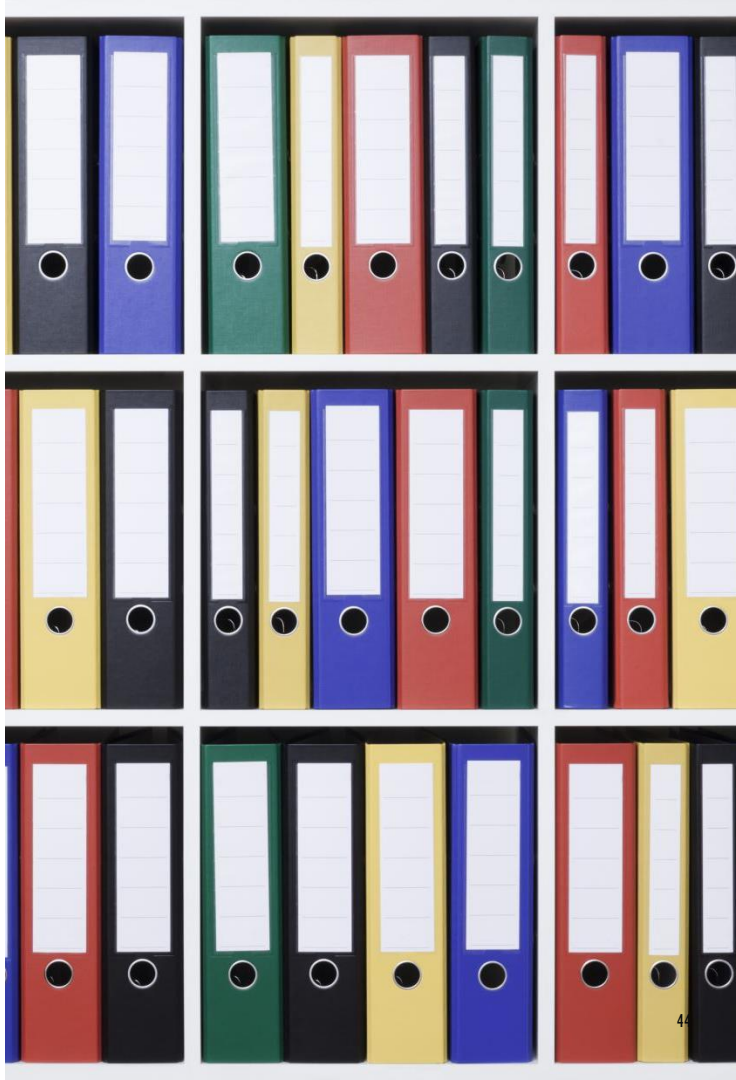
Um conjunto de campos inter-relacionados compõe um registro.

Por exemplo, um registro de um empregado poderia conter:

Identificação (um número decimal)

Nome (uma cadeia de caracteres ? campo)

Ano de admissão (cadeia de caracteres numéricos).



HIERARQUIA DE DADOS

Arquivo – File

Um arquivo é um grupo de registros.

Genericamente, um arquivo contém dados arbitrários em formatos arbitrários.

Em alguns sistemas operacionais, um arquivo é simplesmente visto com uma sequência de bytes.

Uma organização de bytes em um arquivo, tal como uma lista de registros, é uma visão criada pela aplicação do programador.



UNIDADE DE ARMAZENAMENTO

Prefixo	Abreviação	Valor Geral	Valor em Ciência da Computação	Exemplo de Uso
Kilo	k	1.000	1.024	256 kilobytes
Mega	M	1.000.000	1.048.576	512 megabytes
Giga	G	1.000.000.000	1.073.741.824	1 gigabytes
Tera	T	1.000.000.000.000	1.099.511.627.776	

- 1 Megabyte = 1.024 * 1 kilobyte
- 1 Gigabyte = 1.024 * 1 Megabyte
- 1 Terabyte = 1.024 * 1 Gigabyte

LINGUAGENS DE PROGRAMAÇÃO – LP

Os programadores escrevem seus programas em várias LPs, algumas entendidas diretamente pelos computadores, outras requerendo passos intermediários de tradução.

As LPs são divididas em três tipos gerais:

1 Linguagens de Máquina

2 Linguagens Assembly

3 Linguagens de Alto-nível

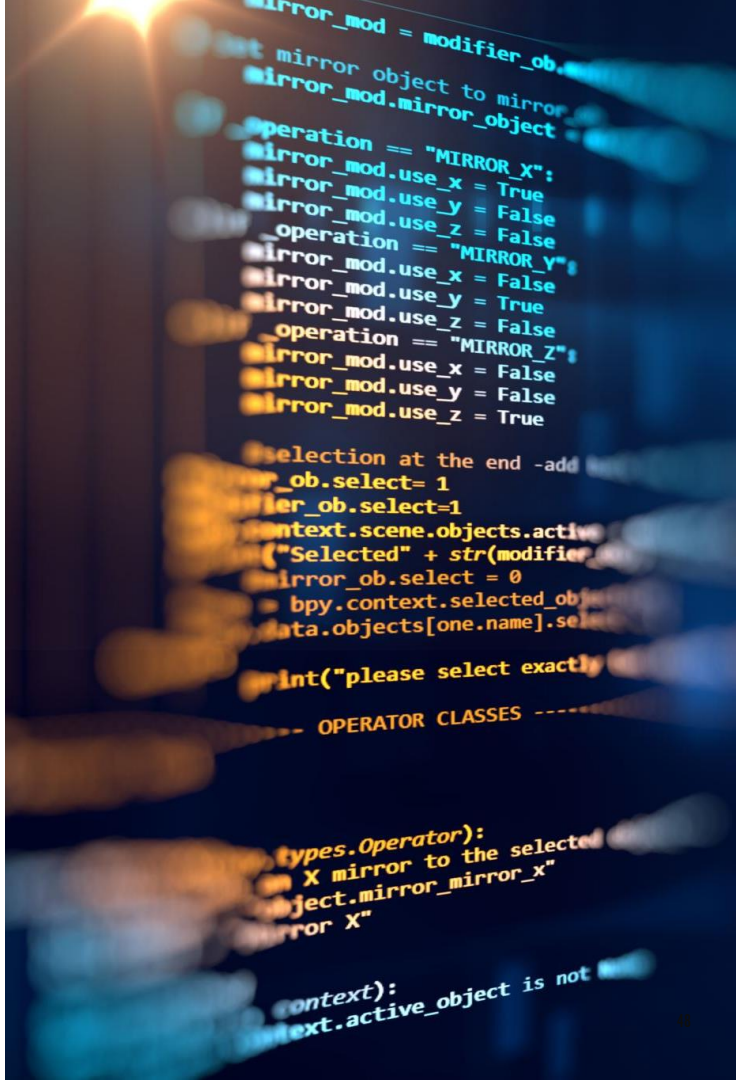


LINGUAGENS DE PROGRAMAÇÃO – LP

Qualquer computador entende diretamente sua própria linguagem de máquina, definida por seu projeto de hardware.

São geralmente são constituídas por strings de números (1s e 0s), as quais instruem ao computador como realizar as operações mais elementares, uma de cada vez.

São dependentes de máquina: uma linguagem de máquina particular somente pode ser usada em um único tipo de computador.



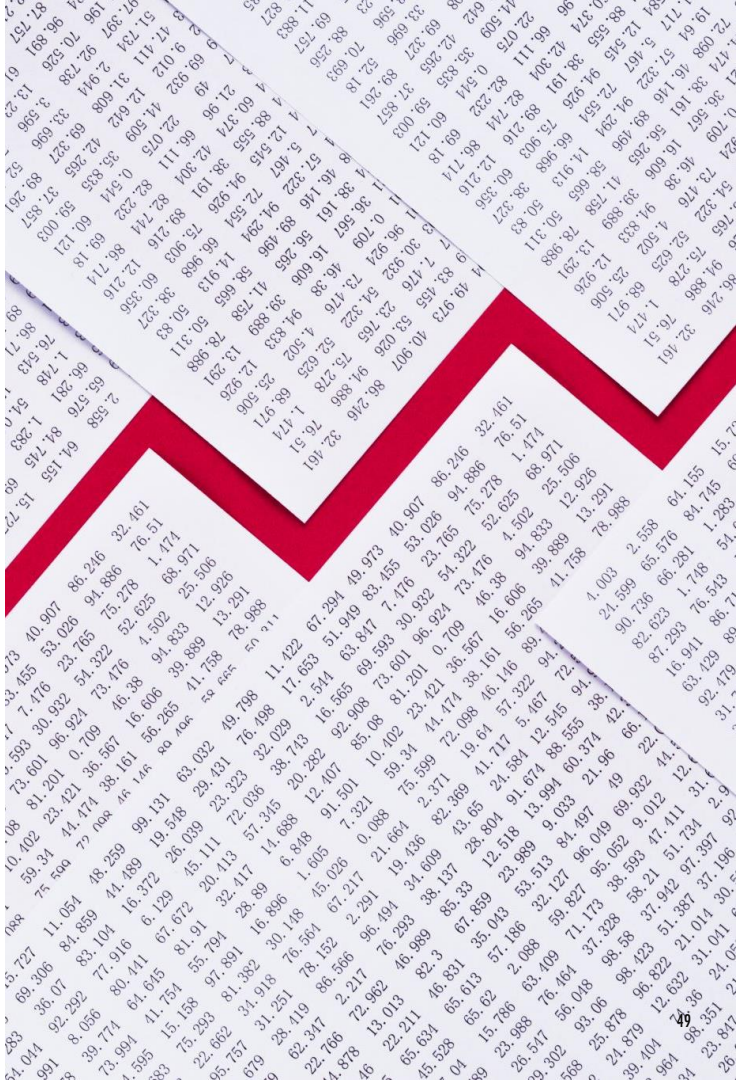
LINGUAGEM DE MÁQUINA

São incômodas para o ser humano.

Por exemplo, seja uma seção de um programa que soma o pagamento da hora extra ao salário base, e a seguir, armazena o resultado no salário bruto:

11000101 10010001 10100101 10111010
11100111 10011110

São dependentes de máquina: uma linguagem de máquina particular somente pode ser usada em um único tipo de computador.

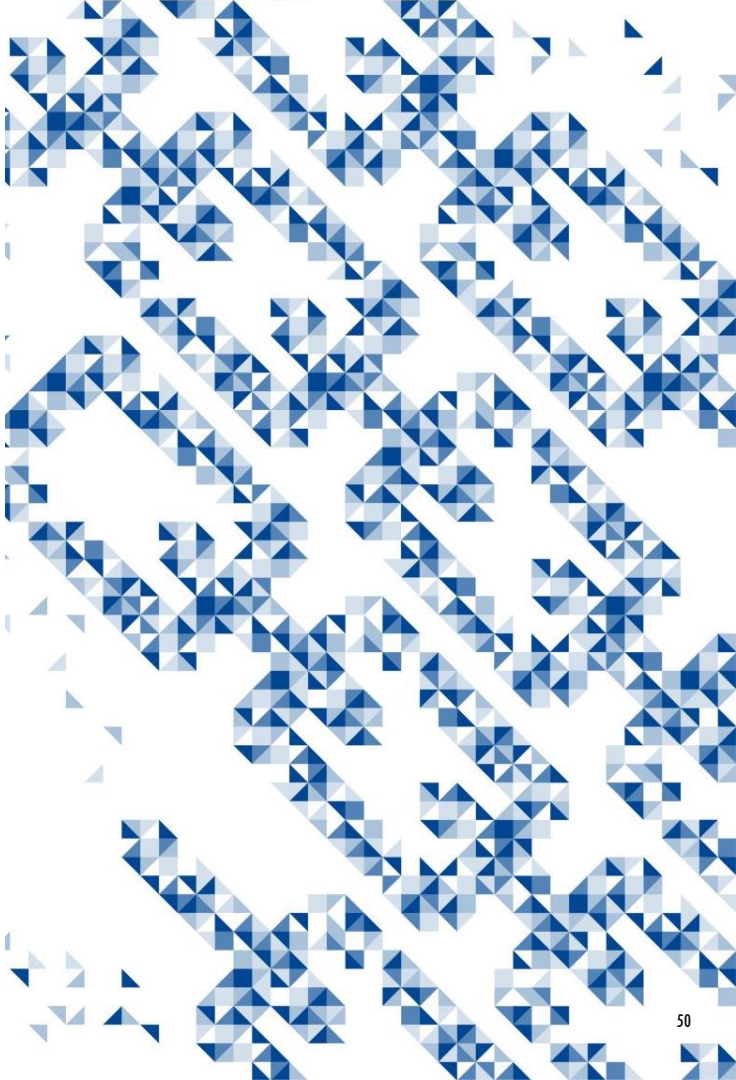


LINGUAGENS DE ASSEMBLY

Substituir as strings de números (entendidas diretamente pelo computador) por abreviações similares ao inglês que representavam as operações elementares (mnemônicos).

Esses mnemônicos formaram as bases das Linguagens Assembly.

Assembler: é um programa tradutor para converter os programas em linguagem assembly para programas em linguagem de máquina (na velocidade do processamento de um computador).



LINGUAGENS DE ASSEMBLY

Traduzindo o programa:

Linguagem de Máquina		Linguagem Assembly
11000101 10010001		`load salarioBase`
10100101 10111010	⟸	`add horaExtra`
11100111 10011110	tradução	`store salarioBruto`

Embora esse código seja mais claro para o ser humano, ele é incompreensível para os computadores (até que seja traduzido para linguagem de máquina.

LINGUAGENS DE ALTO-NÍVEL

A utilização dos softwares aumentou com o surgimento das linguagens assembly.

Porém, os programadores escreviam muitas linhas de código, mesmo para tarefas simples!

Linguagem de Alto-nível: desenvolvidas para acelerar o processo de criação de softwares, onde as instruções para o computador realizavam tarefas mais complexas.

Os programas tradutores, chamados compiladores, traduziam programas em linguagem de alto-nível para programas em linguagem de máquina.

LINGUAGENS DE ALTO-NÍVEL

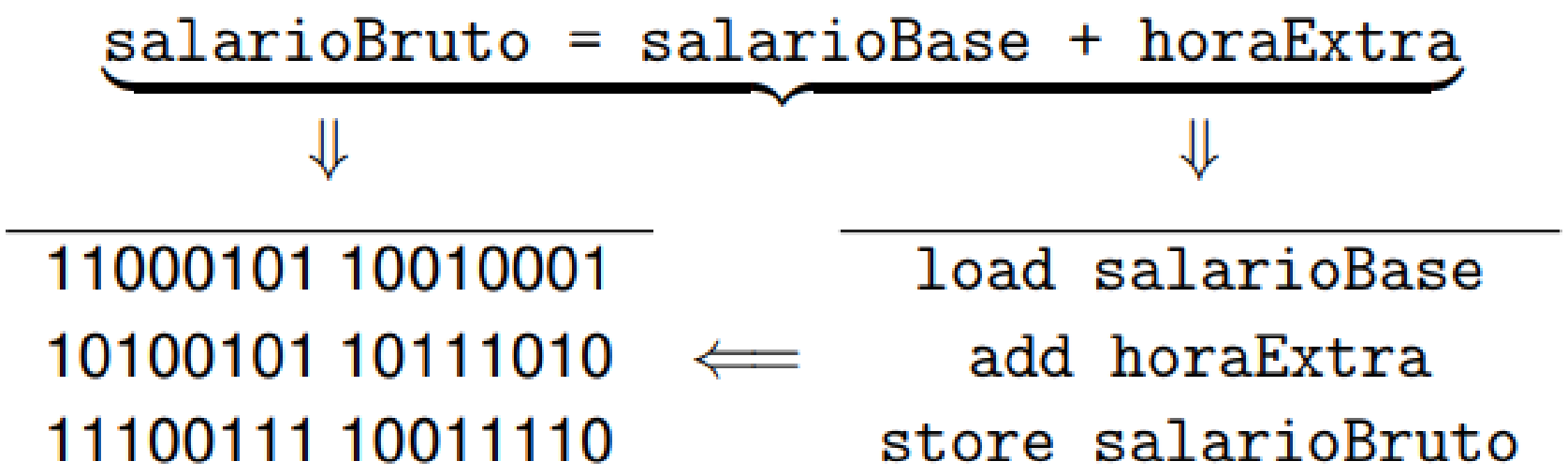
As instruções em alto-nível parecem com palavras cotidianas do inglês e contém comumente notações matemáticas.

A instrução em alto-nivel representa o programa descrito anteriormente:

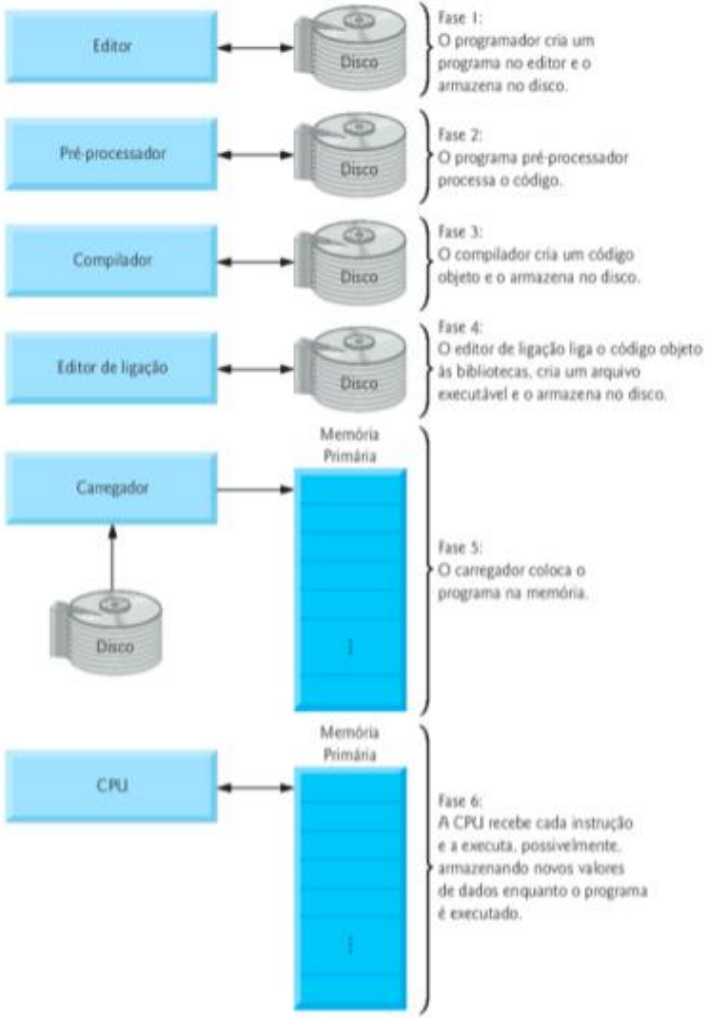
salarioBruto = salarioBase + horaExtra

LINGUAGENS DE ALTO-NÍVEL

Tradução:



ESTRUTURA DE UM PROGRAMA

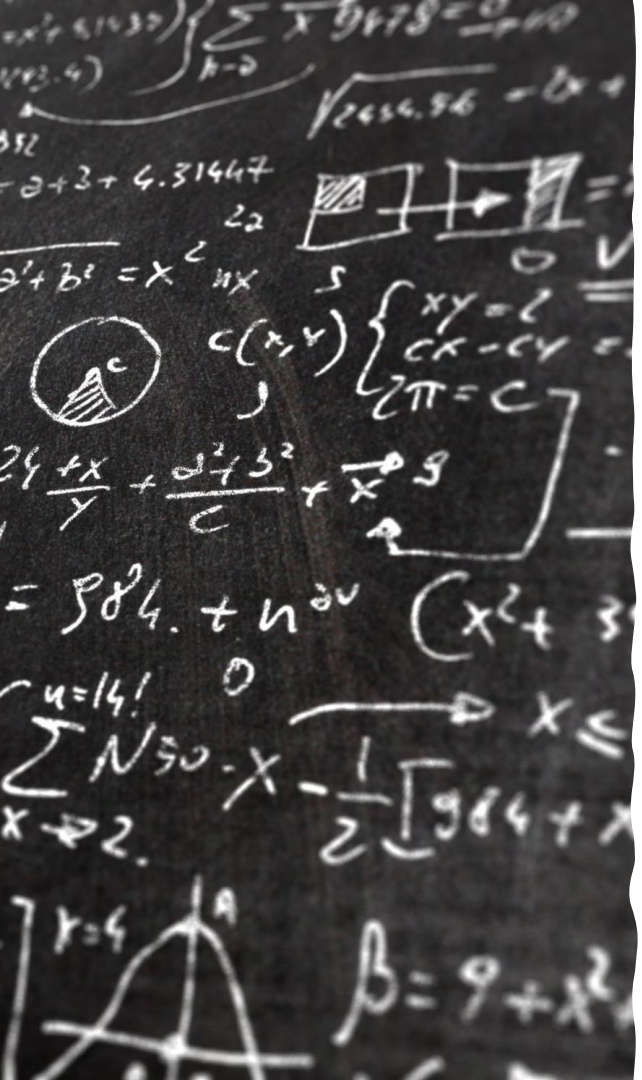


Critérios em um bom projeto de linguagem de programação

- Escrita: a qualidade de uma linguagem que permite a um programador usá-la para expressar uma computação de maneira clara, correta, concisa e rápida.
- Legibilidade: a qualidade de uma linguagem que permite ao programador entender e compreender a natureza de uma computação com facilidade e precisão.
- Ortogonalidade: a qualidade de uma linguagem fornecida pelos recursos tem o mínimo de restrições possível e pode ser combinada de qualquer maneira significativa.
- Confiabilidade: a qualidade de uma linguagem que garante que um programa não se comporte de maneira inesperada ou desastrosa durante a execução.
- Manutenibilidade: A qualidade de uma linguagem que ameniza erros pode ser encontrada e corrigida e novos recursos adicionados.

Critérios em um bom projeto de linguagem de programação

- Generalidade: a qualidade de uma linguagem que evita casos especiais na disponibilidade ou uso de construções e pela combinação de construções estreitamente relacionadas em uma única e mais geral.
- Uniformidade: a qualidade de uma linguagem em que recursos semelhantes devem ser semelhantes e se comportar da mesma forma.
- Extensibilidade: a qualidade de uma linguagem que fornece algum mecanismo geral para o usuário adicionar novas construções a uma linguagem.
- Padronização: a qualidade de uma linguagem que permite que programas escritos sejam transportados de um computador para outro sem mudança significativa na estrutura da linguagem.
- Capacidade de implementação: A qualidade de uma linguagem que fornece um tradutor ou intérprete pode ser escrita. Isso pode abordar a complexidade da definição da linguagem.



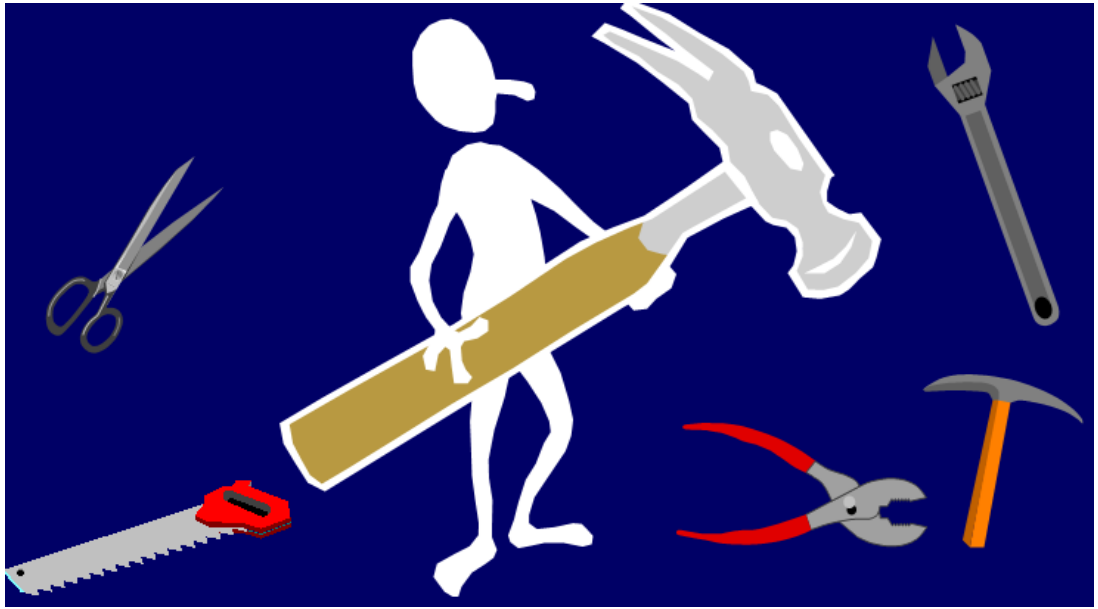
AULA 2: APRESENTAÇÃO DA PLATAFORMA DE PROGRAMAÇÃO, INTRODUÇÃO AS SENTENCIAS EM C, CRIAÇÃO DE DIGRAMAS DE FLUXO.

Conteúdo

- Um levantamento das técnicas de design algorítmico.
- Pensamento abstrato.
- Como desenvolver novos algoritmos para qualquer problema que possa surgir.

Conteúdo

- Um levantamento das técnicas de design algorítmico.
- Pensamento abstrato.
- Como desenvolver novos algoritmos para qualquer problema que possa surgir.



Estudo

- Muitos programadores experientes foram solicitados a codificar a pesquisa binária.

Exemplo Inicial:

1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16
14	21	5	45	12	3	86	98	46	53	24	2	1	15	90	47

Após ordenação: !!!???

1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16
1	2	3	5	12	14	15	21	24	45	46	47	53	86	90	98

Pergunta: Como verificar se o **elemento 90** está presente no vetor acima?

Pergunta: Quantas comparações são necessárias para achar o **elemento 90**?

Pergunta: Como verificar se o **elemento 71** está presente no vetor acima?

Pergunta: Quantas comparações são necessárias para achar o **elemento 71**?

80% entendeu errado

O bom é que não era para uma usina nuclear ou para um marca-passo!!!

Visão abstrata de alto nível Representação de um Algoritmo

- **Prós**
  - ➥Intuitivo para humanos
  - ➥Útil para
    - ↓ pensar sobre o problema
    - ↓ Projetar a solução
    - ↓ descrever algoritmos
  - ➥Visão a partir da qual a correção é evidente
- **Contra**
  - ➥Muito abstrato
  - ➥Resistência dos alunos/programadores

Meta Modelo

- Meta-modelo ou meta-modelagem é a análise, construção e desenvolvimento dos quadros, regras, restrições, modelos e teorias aplicáveis e úteis para modelar uma classe predefinida de problemas

META-MODELO MATEMÁTICO

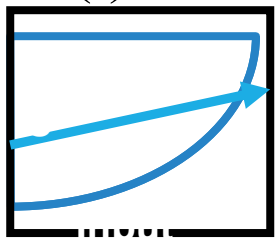
Quantificadores lógicos

$\exists g \, \forall b$ Loves(b,g)

$\forall b \, \exists g$ Loves(b,g)

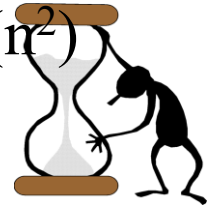
Funções de classificação

$f(i) = n^{\Theta(n)}$



Complexidade de tempo

$t(n) = \Theta(n^2)$



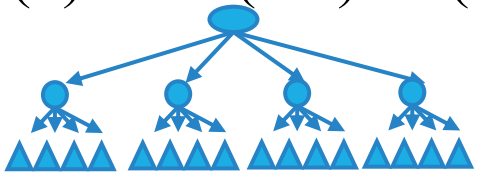
Expressões e Logaritmos

$$2^a \times 2^b = 2^{a+b}$$

$$2^{\log n} = n$$

Relações de Recorrência

$T(n) = a\,T(n/b) + f(n)$

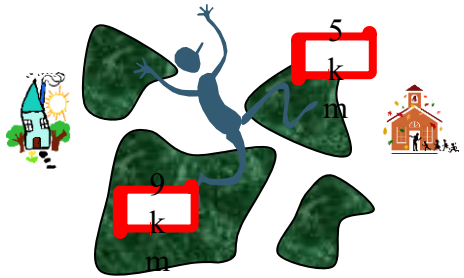


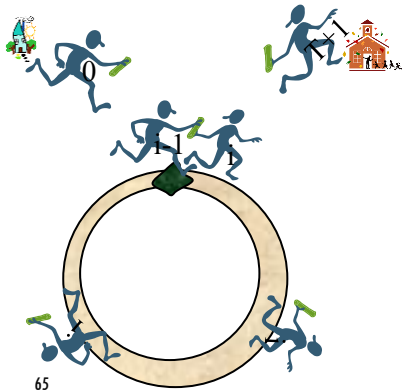
Meta-Algoritmo:

ALGORITMOS ITERATIVOS

LOOP INVARIANTE

<preCond>
codeA
loop
    <loop-invariant>
    exit when <exit Cond>
    codeB
codeC
<postCond>

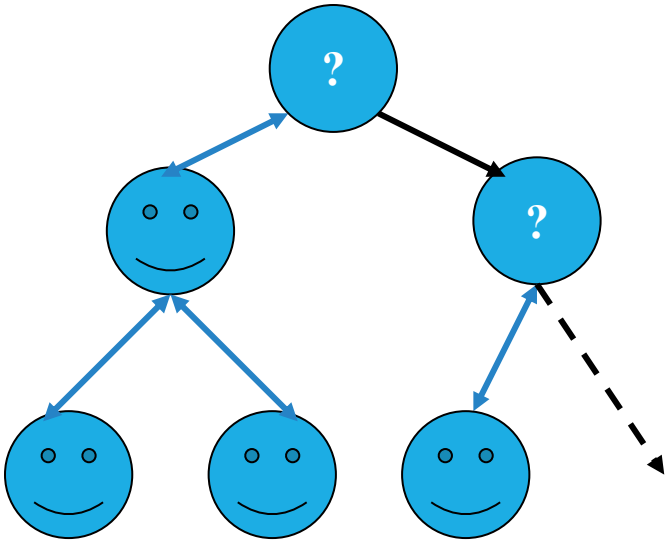


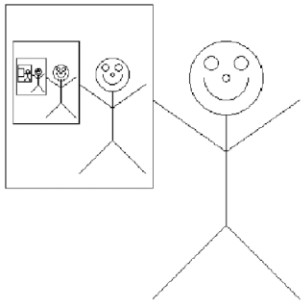


Corrida de revezamento

Meta-Algoritmo:

ALGORITMO RECURSIVO

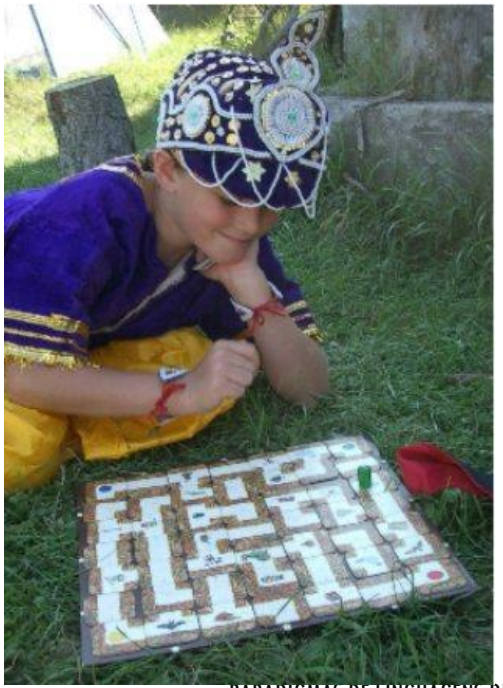






ALGORITMOS DE PESQUISA DE GRÁFICO

Meta-Algoritmo:



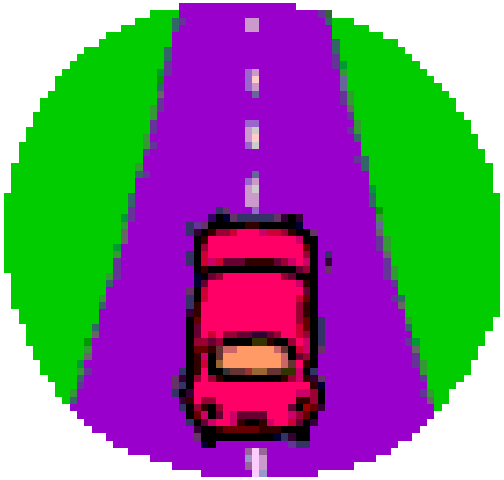
Meta-Algoritmo:

FLUXOS DE REDE



Meta-Algoritmo:

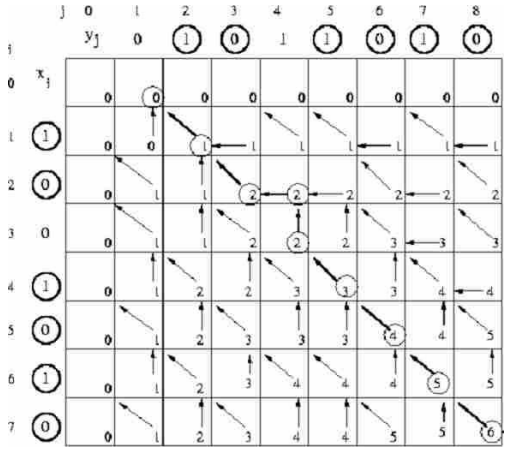
CAMINHAMENTO DE RETORNO RECURSIVO

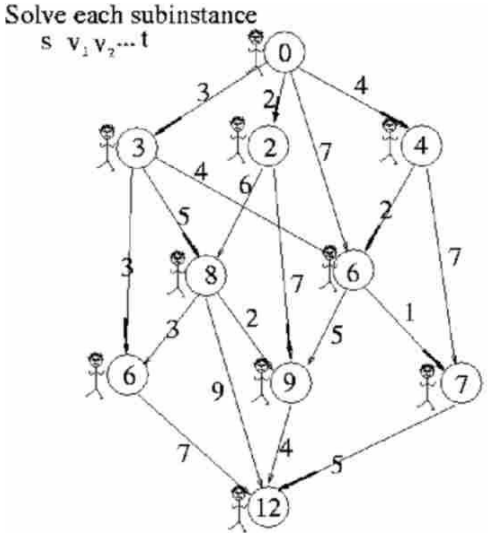




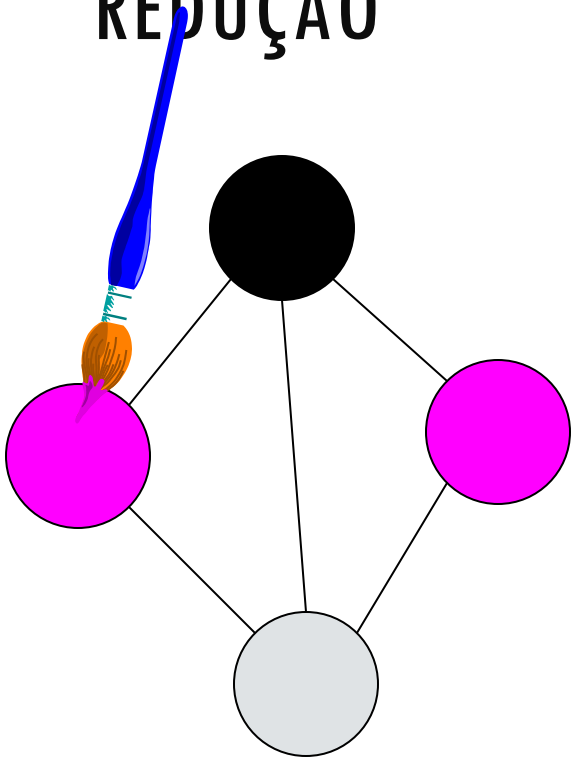
Meta-Algoritmo:

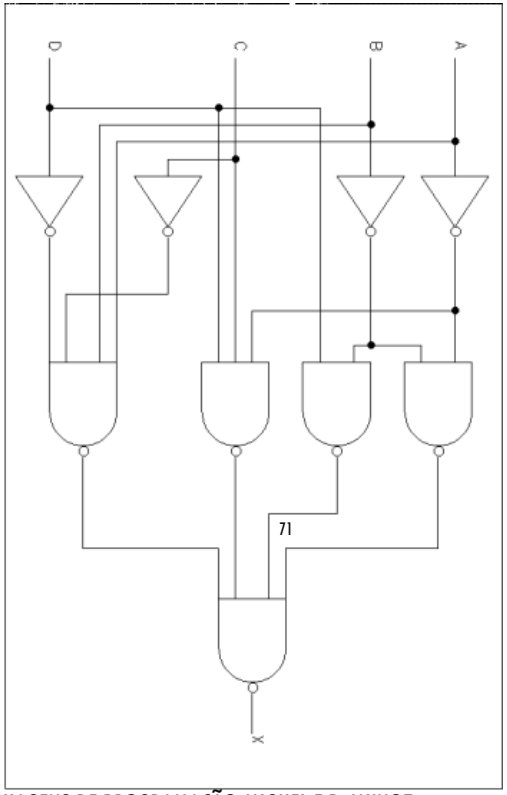
PROGRAMAÇÃO DINÂMICA





Meta-Algoritmo:





Rudich www.discretemath.com

Algoritmo e programação
Definições

- **Algoritmo** - Conjunto de regras e operações bem definidas e ordenadas, destinadas à solução de um problema, ou de uma classe de problemas, em um número finito de etapas - **representação de uma solução para um problema.**

- **Programa** - Sequência completa de instruções a serem executadas por um computador - **de acordo com um algoritmo.**

Algoritmo e programação
O que é um algoritmo?

- Algoritmo é a essência lógica de cada programa.

- Algoritmo pode ser expresso em um linguagem algorítmica e/ou graficamente (fluxograma), sendo necessário depois traduzi-lo em um programa em uma linguagem de programação.

- Ou, senão, um algoritmo pode ser diretamente expresso em uma linguagem de programação tonando-se assim um programa.

Algoritmo e programação
O que é um programa?

- Uma sequência de instruções que especifica como executar uma operação de computação.
- A operação de computação pode ser:
  - algo matemático (solução de um sistema de equações)
  - uma operação de computação simbólica (busca e a substituição de textos
  - algo gráfico (processamento de uma imagem)

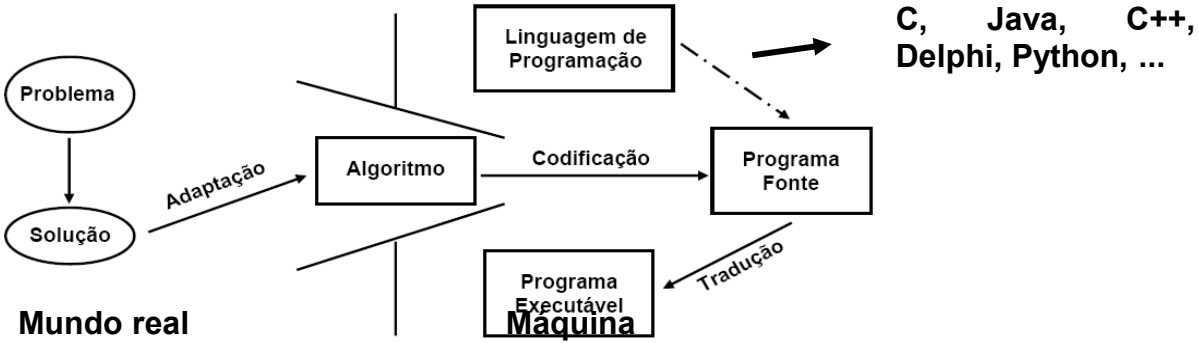
Algoritmo e programação
O que é um programa?

- Instruções básicas:
  - ***Entrada***
    - Receber dados do teclado, de um arquivo, da rede ou de algum outro dispositivo.
  - ***Saída***
    - Exibir dados na tela, salvá-los em um arquivo, enviá-los pela rede etc.
  - ***Matemática***
    - Executar operações matemáticas básicas como adição e multiplicação.
  - ***Execução condicional***
    - Verificar a existência de certas condições e executar o código adequado.
  - ***Repetição***
    - Executar várias vezes alguma ação, normalmente com algumas variações.

Algoritmo e programação
Definições

- O **algoritmo**, do ponto de vista computacional, tem um papel fundamental por ser o elo de ligação entre dois mundos (real e computacional).

- A atividade de programação começa com a construção do algoritmo.



Algoritmo e programação
Exemplos

- **Algoritmo: trocar lâmpada**

- **Passo 1:** pegar a lâmpada nova.
- **Passo 2:** pegar a escada.
- **Passo 3:** posicionar a escada embaixo da lâmpada queimada.
- **Passo 4:** subir na escada com a lâmpada nova.
- **Passo 5:** Retirar a lâmpada queimada.
- **Passo 6:** Colocar a lâmpada nova.
- **Passo 7:** Descer da escada.
- **Passo 8:** Ligar o interruptor.
- **Passo 9:** Guardar a escada.
- **Passo 10:** Jogar a lâmpada velha no lixo

Algoritmo: sacar dinheiro

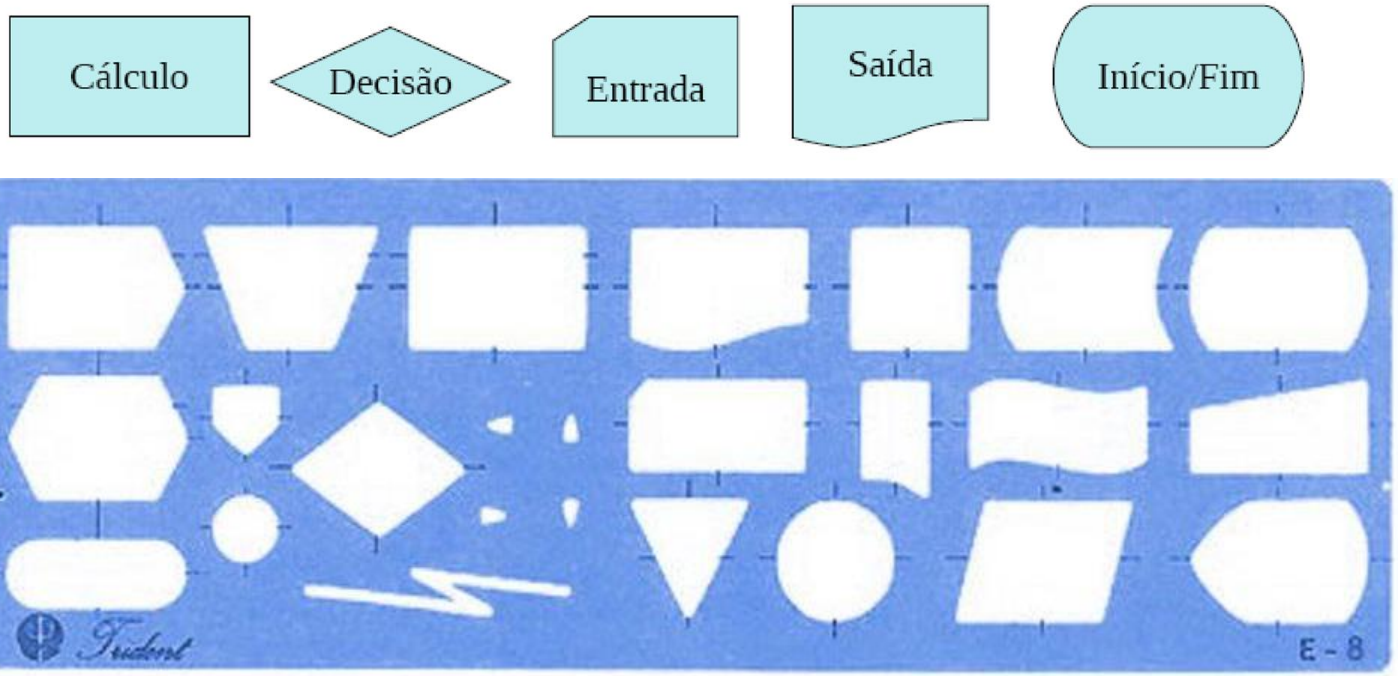
Passo 1: ir até o caixa eletrônico.
Passo 2: colocar o cartão.
Passo 3: digitar a senha.
Passo 4: solicitar o saldo.
Passo 5: se o saldo for maior ou igual à quantia desejada, sacar a quantia desejada; caso contrário sacar o valor do saldo.
Passo 6: retirar dinheiro e cartão.
Passo 7: sair do caixa eletrônico.

Métodos de Representação de Algoritmos

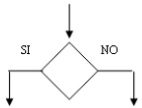
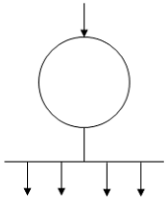
- **Fluxograma -** Representação gráfica
  - **Características**
    - A representação gráfica é mais concisa que a representação textual.
    - É necessário aprender a simbologia dos fluxogramas.
- **Pseudocódigo (Português estruturado) -** Representação textual
  - **Características**
    - A transcrição para qualquer linguagem de programação é quase direta.
    - É necessário aprender as regras do pseudocódigo.

Métodos de Representação de Algoritmos

➥**Fluxograma -** Representação gráfica por meio de símbolos geométricos, da solução algorítmica de um problema



SÍMBOLOS

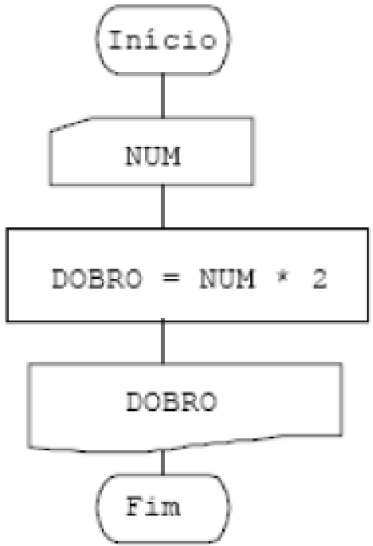
Simbolo	Descrição
	Para inicio/fim ou para uma parada indeterminada
	Simbolo de entrada/saida genérico
	Representa uma operação ou processo geral com dados de memória.
	Simbolo de sub-programa ou sub-rotina. Utiliza-se para realizar uma chamada a um módulo do programa.
	Simbolo de decisão para realizar uma pergunta com duas possiveis respostas. É o que chamamos de simbolo de seleção simples.
	Simbolo de múltipla seleção
	Simbolo de loop definido.
	Conector. Utiliza-se para agrupar várias linhas de fluxo que saem da mesma origem.
	Simbolo para colocar comentários





Métodos de Representação de Algoritmos

- **Exemplo - Fluxograma.**



EXPLICAÇÃO

Início do algoritmo

Entrada do número

Cálculo do dobro do número

Apresentação do resultado

Fim do algoritmo

Métodos de Representação de Algoritmos

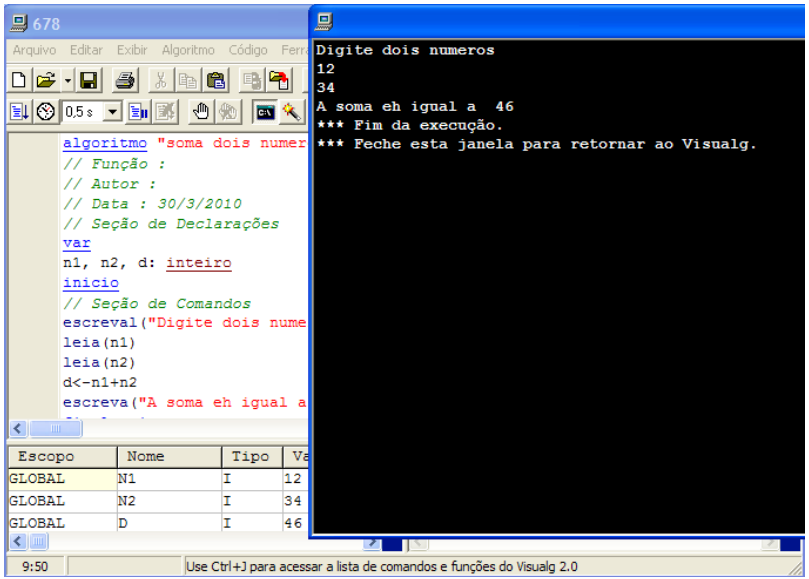
- **Pseudocódigo.**
  - Descrição narrativa utilizando nosso idioma para descrever o algoritmo.
  - **Exemplo de uma descrição narrativa.**
    - Soma de dois números.

      1. Receber os dois números.
      1. Efetuar a soma dos dois números.
      1. Mostrar o resultado.

Métodos de Representação de Algoritmos

- **Exemplo – Descrição narrativa (Visualg).**

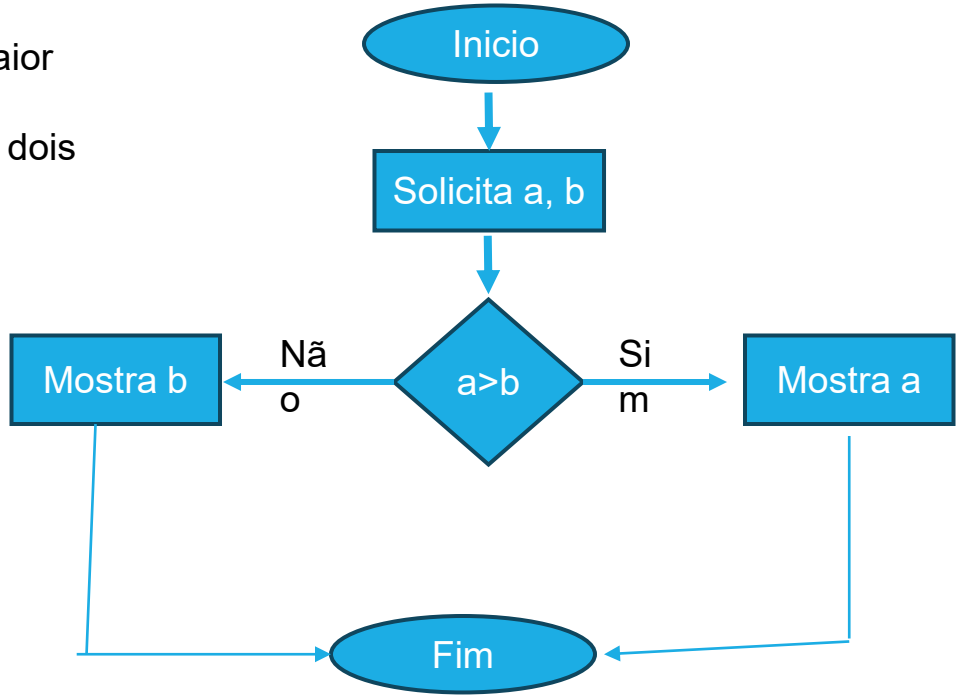
```
algoritmo "soma dois numeros"
// Função :
// Autor :
// Data : 30/3/2010
// Seção de Declarações
var
n1, n2, d: inteiro
inicio
// Seção de Comandos
escreval("Digite dois numeros")
leia(n1)
leia(n2)
d<-n1+n2
escreva("A soma eh igual a ", d)
fimalgoritmo
```



Métodos de Representação de Algoritmos

- **Resumindo**
  - Escrever algoritmos e, por fim, programar, consiste em dividir qualquer problema em vários **passos** menores, usando uma ou mais formas de representação.
  - Esses **passos** que compõem o algoritmo são denominados de **comandos**.

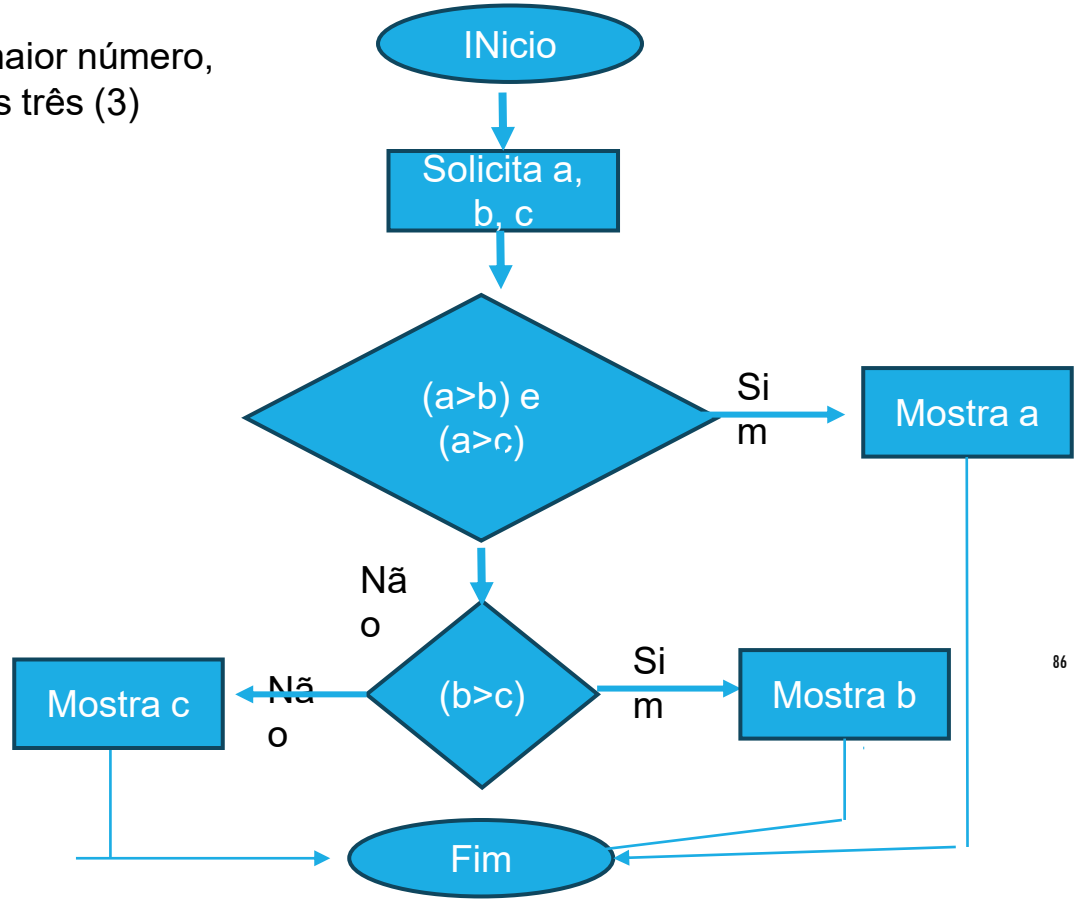
Identificar o maior número,
Sendo apenas dois números



(a, b)
(2, 5)

(20,19)
(15,15)

Identificar o maior número, Sendo apenas três (3) números

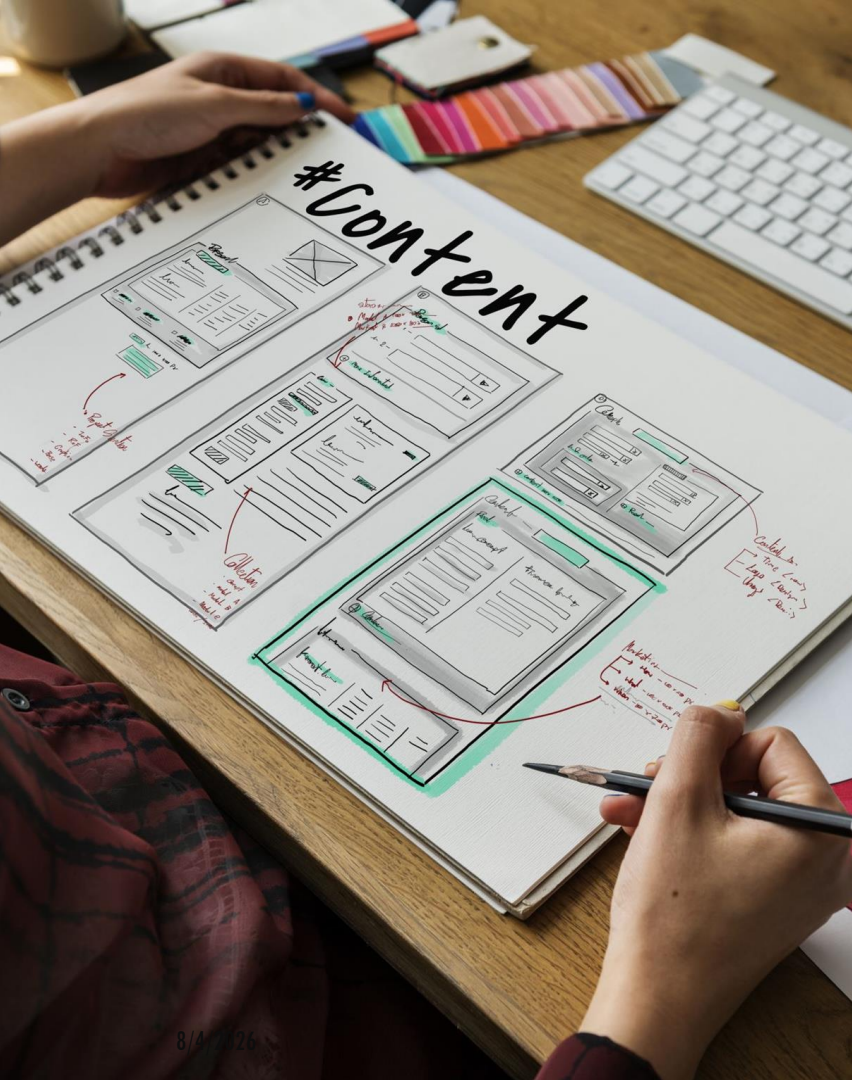


(a, b, c)
(2, 5, 7)

(2, 7, 5)

(7, 5, 2)

86



- Aula 3: Utilização de fluxogramas na programação

EXEMPLO DE AULA

8/4/2026

Uma concessionária de carros oferece três categorias de automóveis:

1. **Carro Popular**: Preços de R$50.000.
2. **Carro Intermediário**: Preços entre R$50.000 e R$100.000.
3. **Carro de Alto Padrão**: Preços entre R$100.000 e R$150.000.

Os automóveis podem ser adquiridos de três formas:

- **À vista**: Com desconto de 5% sobre o preço final, caso seja do interesse do cliente.
- **Parcelado**: Sem desconto adicional, tem um acréscimo do 10% do preço de fábrica sendo parcelado a 48X
- **Troca com veículo usado**: O cliente pode entregar um carro usado, sendo abatido até 30% do preço de mercado do veículo usado, independentemente da modalidade escolhida, porém paga o restante a vista.

O preço final de um automóvel é calculado como:

- **Preço de fábrica** + **Impostos** (30% do preço de fábrica) + **Percentual do revendedor** (10% do preço de fábrica)+lucro_loja (15% do preço de fábrica).

Tarefa:
Desenvolva um programa que leia o nome do automóvel e o preço de fábrica e calcule:

1. O **preço final do automóvel**, considerando os descontos aplicáveis.
2. A diferença entre o **custo de produção** e o **preço final de venda** na loja.
3. Impostos, preço de fábrica e percentual do revendedor.

Além disso:

- Escreva o **pseudocódigo** para o problema.
- Crie o **diagrama de fluxo** representando o processo.
- Implemente soluções em **C** e **JavaScript** para resolver o problema.

ATIVIDADE

Realize um programa que resolva as quatro operações fundamentais em Javascript, C e Python e compare e realize um comparativo entre estas linguagens de programação.



PSEUDOCÓDIGO

- **Passo 1:** Declarar as variáveis de entrada e saída.
- **Passo 2:** Mostrar uma mensagem com os preços do carro.
- **Passo 3:** Mostrar uma mensagem com as modalidades de venda.
- **Passo 4:** Avaliar a escolha do carro. Verificar se é uma opção correta.
- **Passo 5:** avaliar a escolha da modalidade de pagamento.
- **Se o a compra é a vista.**
- Pfabrica=Pvenda/1,55;
- Pfinal=Pvenda*0,95;
- Imposto= Pfabrica*0, 3;
- Comissão= Pfabrica*0,10;

  Ganho da loja=Pfinal-Imposto-Comissão;

  Imprimir o boleto
- **Se o a compra é parceçado.**
- Pfabrica=Pvenda/1,55;
- Pfinal=Pvenda + 0,1* Pfabrica;
- Prestação= Pfinal/48;
- Imposto= Pfabrica*0,3;
- Comissão= Pfabrica*0,1;

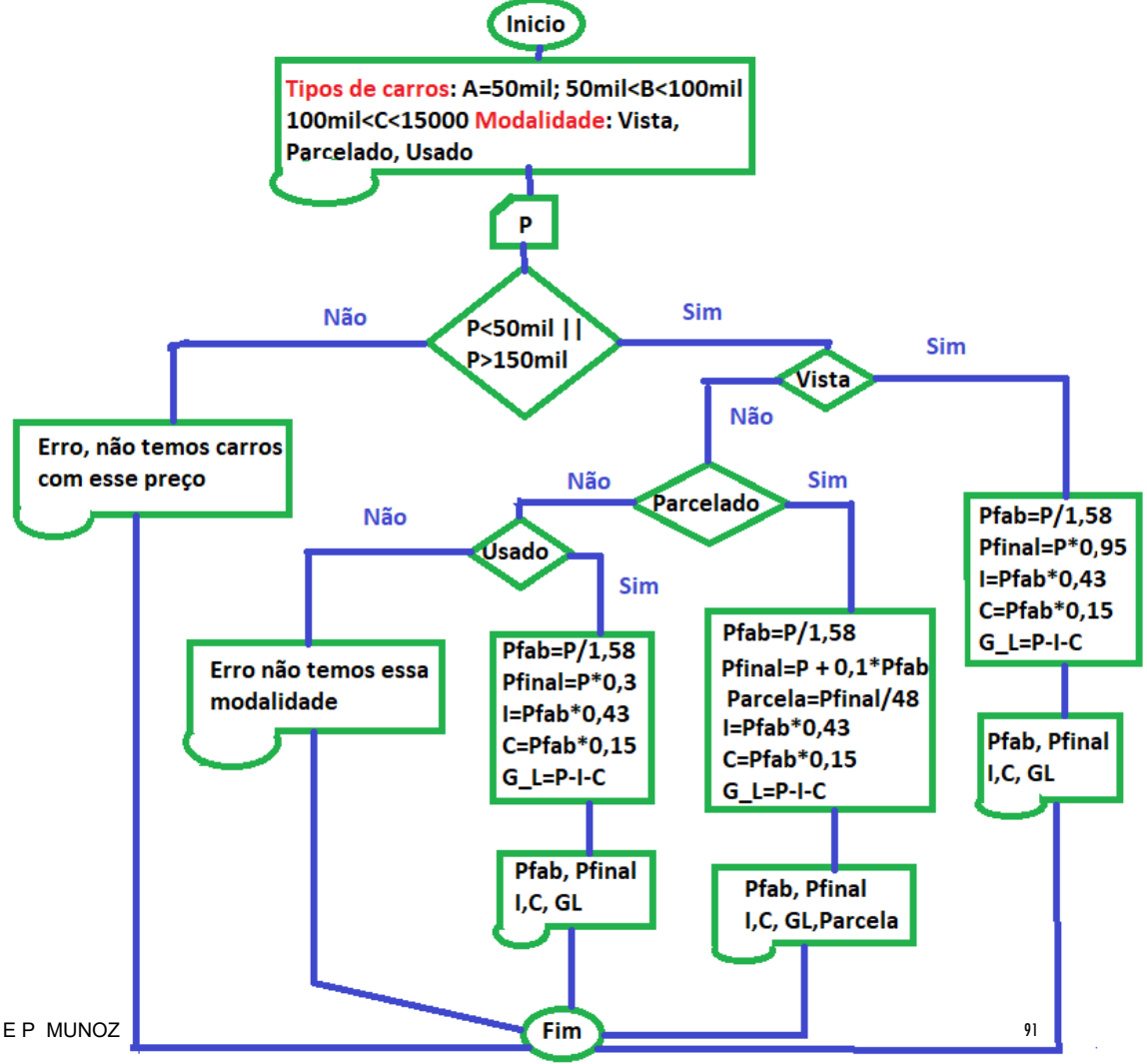
  Ganho da loja=Pfinal-Imposto-Comissão;

  Imprimir o boleto

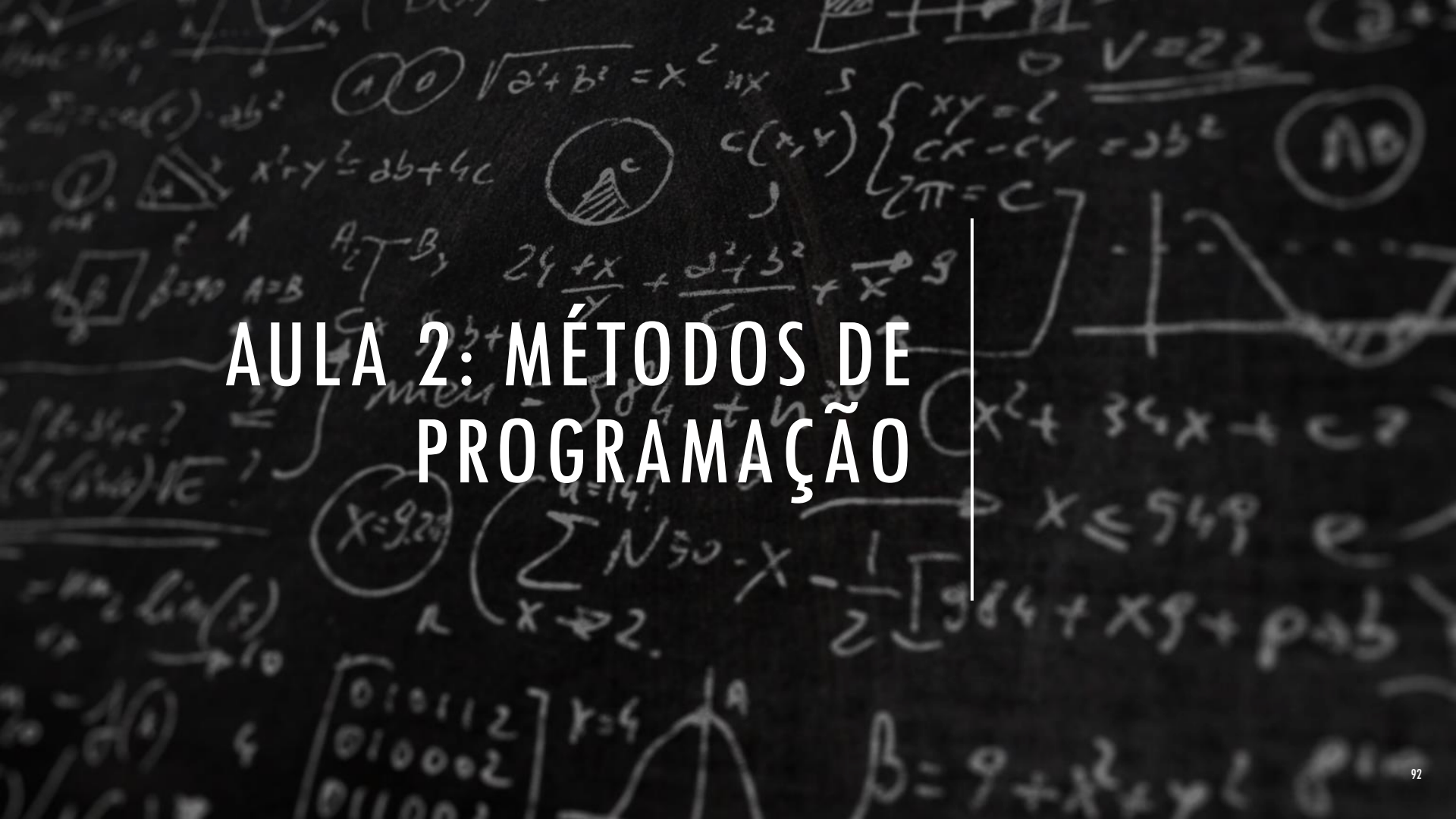
Se o a compra usa carro como parte do pagamento.

Pfabrica=Pvenda/1,55;
Pfinal=Pvenda *0,3;
Imposto= Pfabrica*0, 3;
Comissão= Pfabrica*0,1;
Ganho da loja=Pfinal-Imposto-Comissão;
Imprimir o boleto

FLUXO GRAMA



8/4/2026



AULA 2: MÉTODOS DE PROGRAMAÇÃO

OBJETIVOS DA AULA



Compreender os principais métodos de programação: top-down, bottom-up, estruturado, recursivo e iterativo.



Aplicar esses métodos na resolução de problemas computacionais.



Desenvolver a capacidade de escolha do método mais adequado para diferentes situações.

CONTEÚDO PROGRAMÁTICO



Introdução aos Métodos de Programação



Programação Top-Down e Bottom-Up

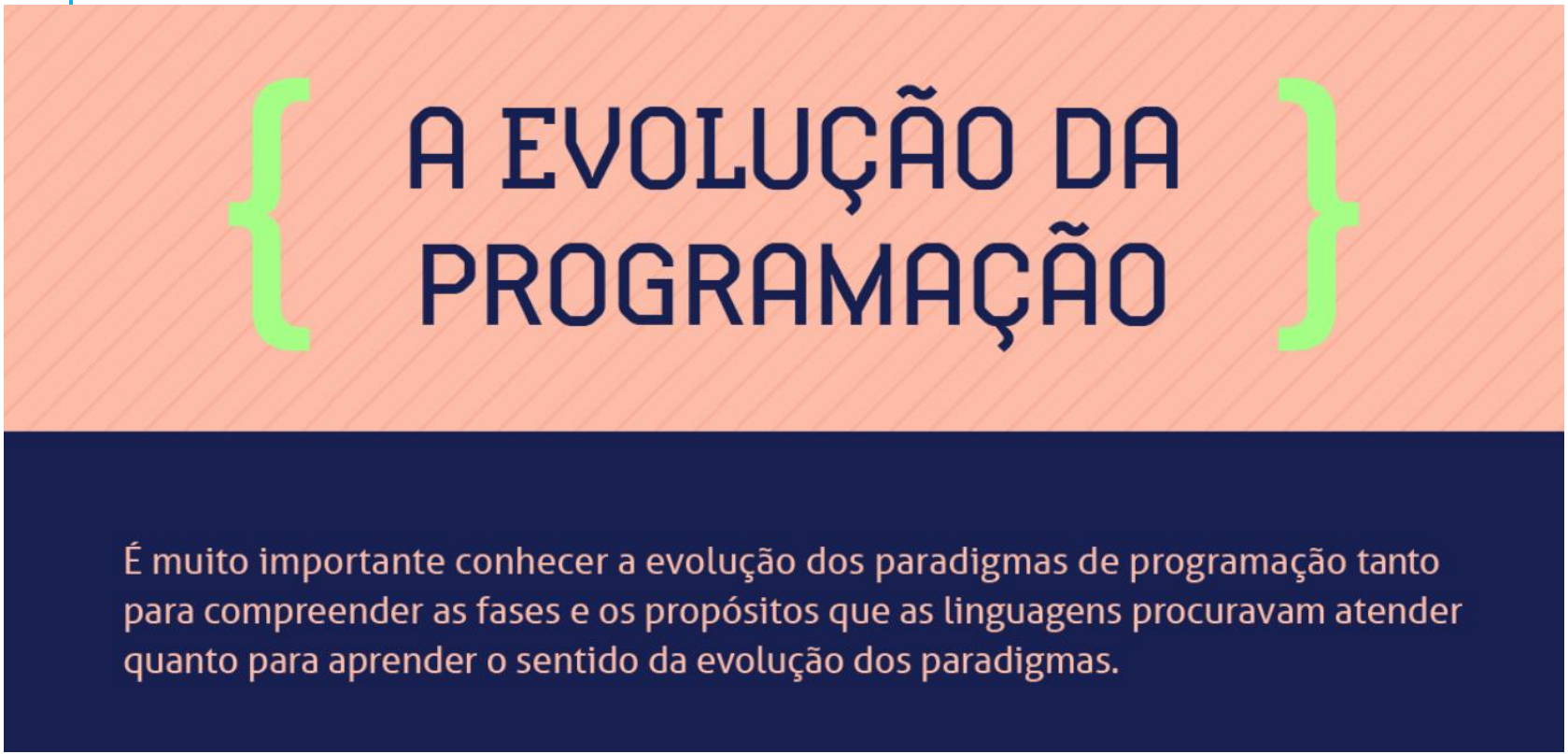


Programação Recursiva e Iterativa



Programação Estruturada

INTRODUÇÃO


A EVOLUÇÃO DA PROGRAMAÇÃO
É muito importante conhecer a evolução dos paradigmas de programação tanto para compreender as fases e os propósitos que as linguagens procuravam atender quanto para aprender o sentido da evolução dos paradigmas.

INTRODUÇÃO

01 PROGRAMAÇÃO BINÁRIA

No início da computação, a única forma de programação era inserindo os binários de um programa diretamente na memória principal do computador. Dessa forma, ocorriam muitos problemas relacionados à manutenção e alta probabilidade de erros. Além disso, o reuso era impossível.



INTRODUÇÃO

02 PROGRAMAÇÃO BINÁRIA

Para tentar solucionar os problemas decorrentes da era da programação binária, surgiram os compiladores, responsáveis pela **conversão da linguagem natural em linguagem de máquina**.

Assim, foram criadas linguagens que possibilitavam a criação de programas com instruções lineares. Apesar disso, ainda assim a programação não era tão simplificada e o entendimento do código era muito complexo.



INTRODUÇÃO

03 PROGRAMAÇÃO PROCEDURAL

Com as dificuldades ainda não transpostas pela programação linear, surgiu a ideia de **divisão do código em procedimento**.

Com isso, torna-se possível criar blocos de código independentes e reutilizáveis o que, por sua vez, fez surgir o conceito de **bibliotecas de código e funções**, as quais possibilitavam o início do reuso de código. Porém, ainda não existiam conceitos como herança e polimorfismo, os quais são encontrados no paradigma de orientação a objetos.

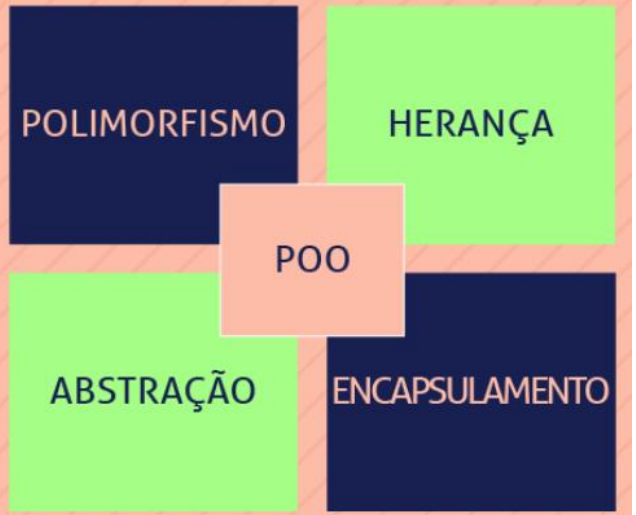


04 PROGRAMAÇÃO ORIENTADA A OBJETOS

A orientação a objetos surgiu como um novo paradigma para a programação, trazendo o conceito de **abstração das entidades do mundo real em objetos independentes** que apresentam ações e atributos específicos e se relacionam entre si.

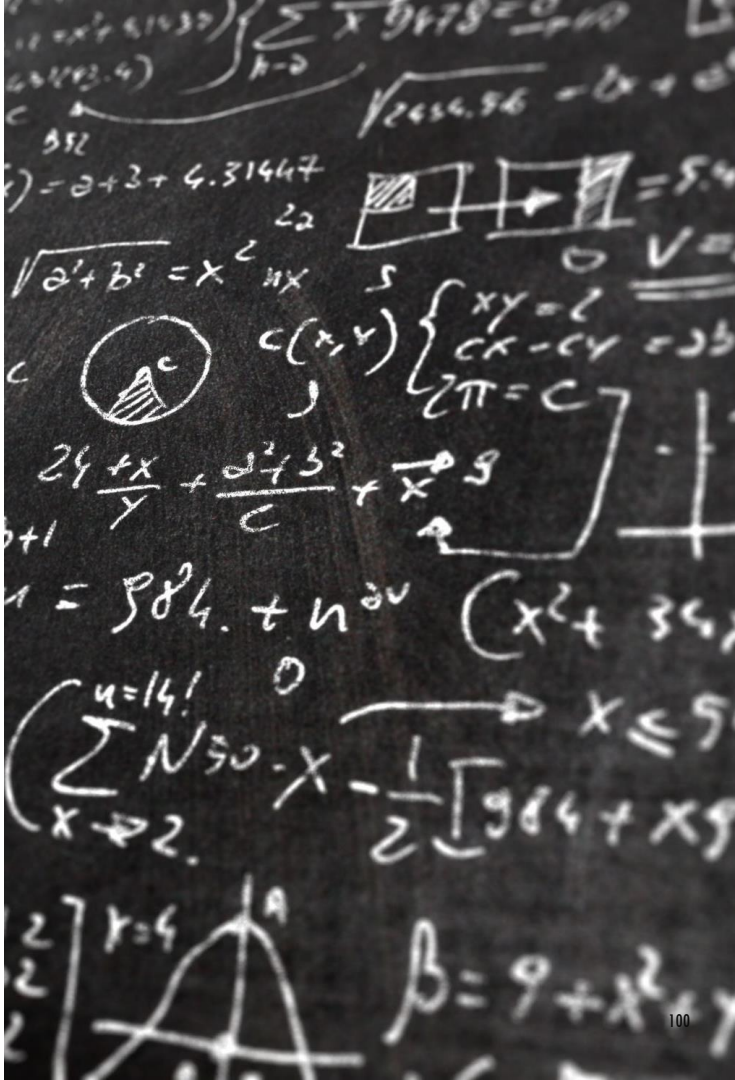
Os atributos são exatamente como no mundo real: o objeto carro, por exemplo, apresenta métodos específicos e se relaciona com o objeto pessoa.

Esse paradigma foi o advento da reusabilidade de código e serviu como base para diversos outros recursos que hoje são encontrados na tecnologia, como serviços.



PARADIGMAS DE PROGRAMAÇÃO

Como seres humanos, sentimos necessidade de expressar nossos pensamentos, seja de forma verbal, utilizando o nosso poder de comunicação, ou escrita, escrevendo textualmente o que estamos pensando. A linguagem de programação, assim como a nossa linguagem natural, permite nossa comunicação com as máquinas. Dessa maneira, podemos instruir, por meio de linhas de comandos, as máquinas a executarem determinada instrução



PARADIGMAS DE PROGRAMAÇÃO

Toda a linguagem de programação está construída sobre um paradigma. Mas, afinal, o que é um paradigma? Um paradigma representa um padrão de pensamento que guia um conjunto de atividades relacionadas, trata-se de um padrão que define um modelo para a resolução de problemas e regra, basicamente, toda e qualquer linguagem de programação existente. Os paradigmas de programação estão classificados em quatro diferentes tipos, que evoluíram ao longo das últimas décadas:

- programação imperativa;
- programação funcional;
- programação lógica;
- programação orientada a objetos

PARADIGMAS DE PROGRAMAÇÃO

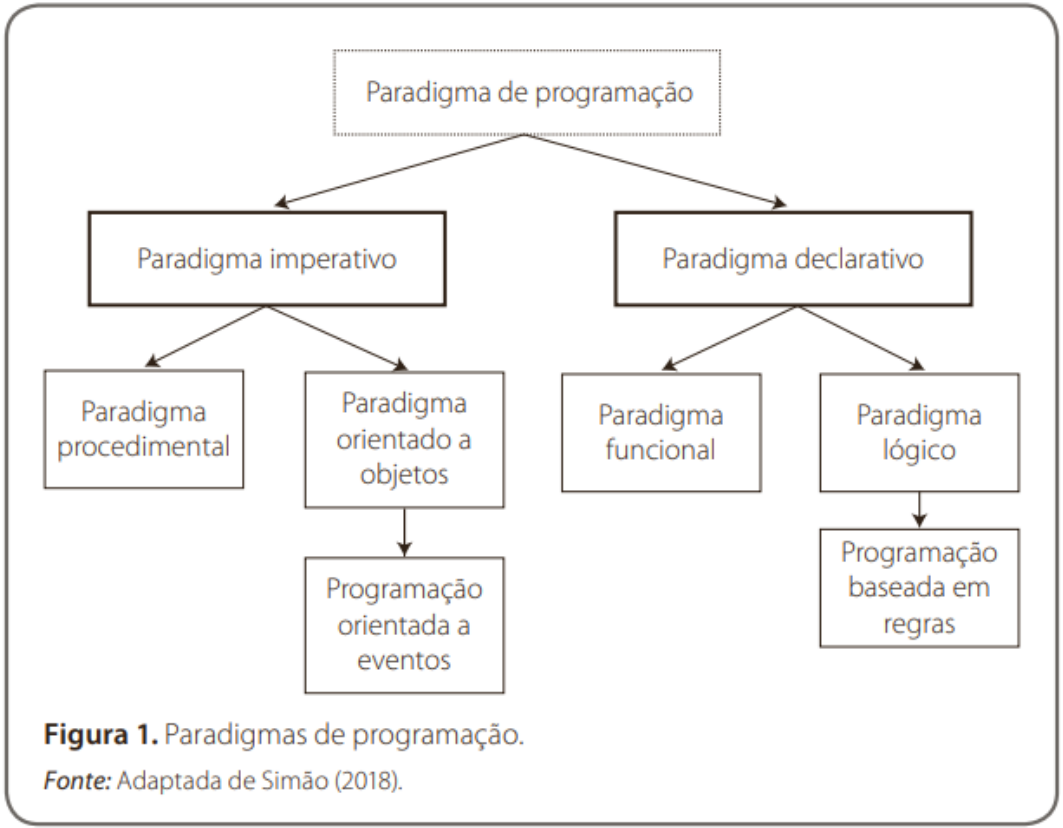


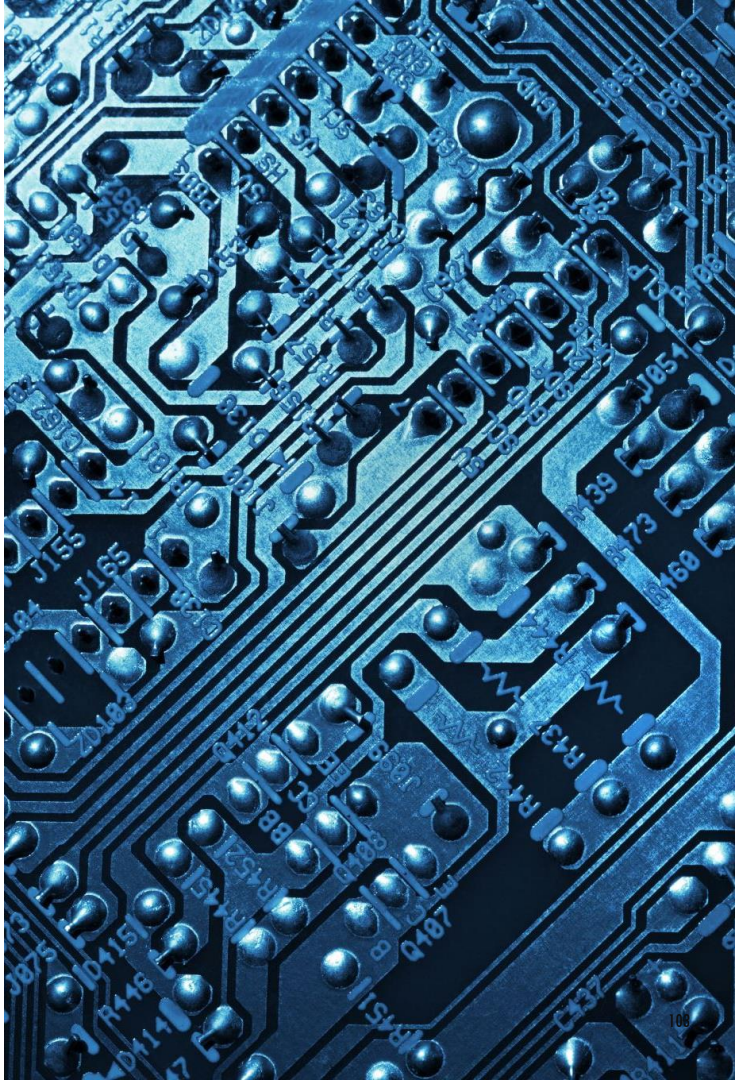
Figura 1. Paradigmas de programação.

Fonte: Adaptada de Simão (2018).

IMPORTÂNCIA

Certamente, o maior motivador para o desenvolvimento das linguagens e métodos de programação nas últimas décadas tem sido o rápido desenvolvimento dos recursos computacionais e o surgimento de novas e emergentes tecnologias, entre as quais podemos destacar as seguintes áreas:

- inteligência artificial;
- World Wide Web ;
- sistemas e redes;
- dispositivos móveis



IMPORTÂNCIA

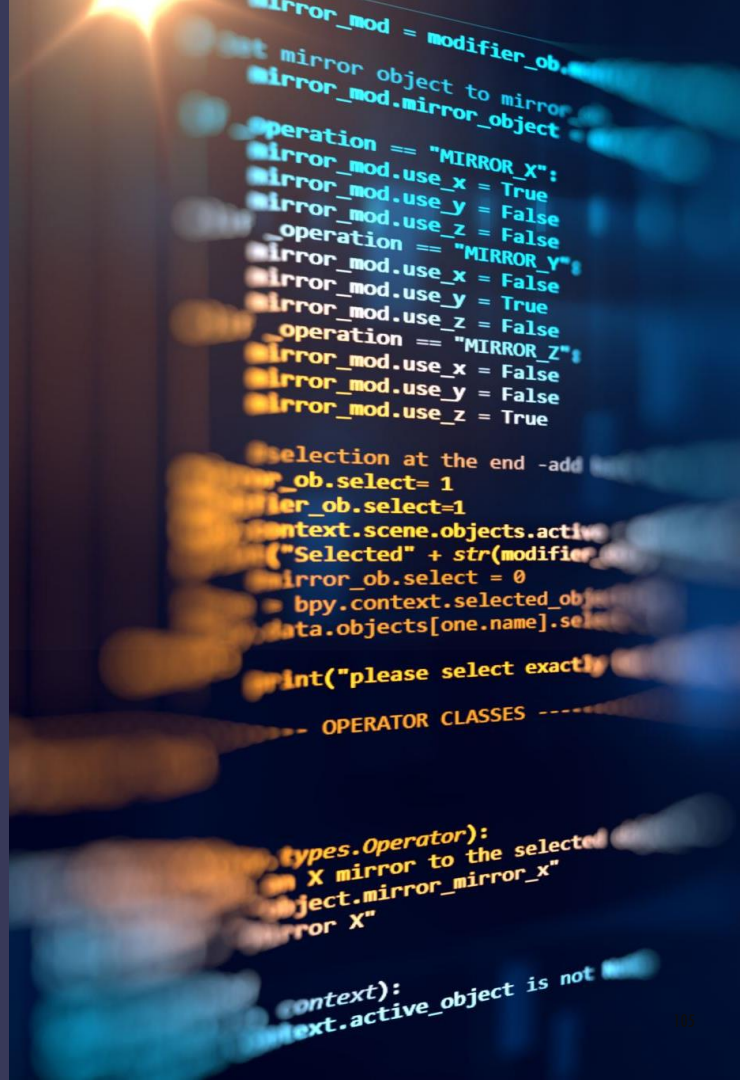
O projeto de uma nova linguagem de programação é algo bem complexo, o projetista deve se preocupar com inúmeros desafios e adotar soluções específicas, que se proponham a atender esses desafios. Os principais desafios envolvidos no projeto de uma nova linguagem de programação são:

- arquitetura;
- requisitos técnicos;
- padrões



TIPOS DE PROGRAMAÇÃO

Os tipos de programação estão diretamente relacionados ao conceito do paradigma no qual a linguagem foi concebida, por exemplo é impossível utilizar uma linguagem linear como Ada ou Assembly e tentarmos construir um programa com blocos de funções. Isso acontece porque o paradigma desse tipo de linguagem não provê recursos que o paradigma procedural possibilita, permitindo o reuso de código por meio de funções (blocos que executam uma determinada funcionalidade).



TIPOS DE PROGRAMAÇÃO

Portanto, é importante entender e conhecer bem os tipos de paradigmas de programação e seus conceitos, para que ao utilizar uma linguagem se saiba como, de acordo com o tipo de paradigma em que ela foi concebida, devem ser estruturados os códigos. A seguir você verá os tipos de paradigmas que surgiram ao longo da evolução da programação.

PARADIGMA IMPERATIVO

Após a geração de programação linear com linguagens de máquina, houve um grande avanço com o advento das linguagens procedurais. Esse tipo de paradigma foi o primeiro que apresentou as linguagens de alto nível, que permitiam a utilização de um vocabulário mais próximo ao natural para construção de programas. Esse paradigma recebe o nome de imperativo pela forma como as instruções nos códigos são repassadas para o compilador:

- Faça isso.
- Depois faça aquilo

PARADIGMA IMPERATIVO

É uma forma imperativa de dar ordens para que a máquina execute as instruções dadas, e ela executará cada uma, passo a passo, com o propósito de chegar no resultado esperado. Uma linguagem imperativa suporta algumas características comuns:

- atribuições, declarações e expressões;
- estruturas de controle;
- abstração procedural.

Basicamente, os códigos são construídos obedecendo a estrutura de declarações e as instruções. A linguagem mais popular desse paradigma é a linguagem C

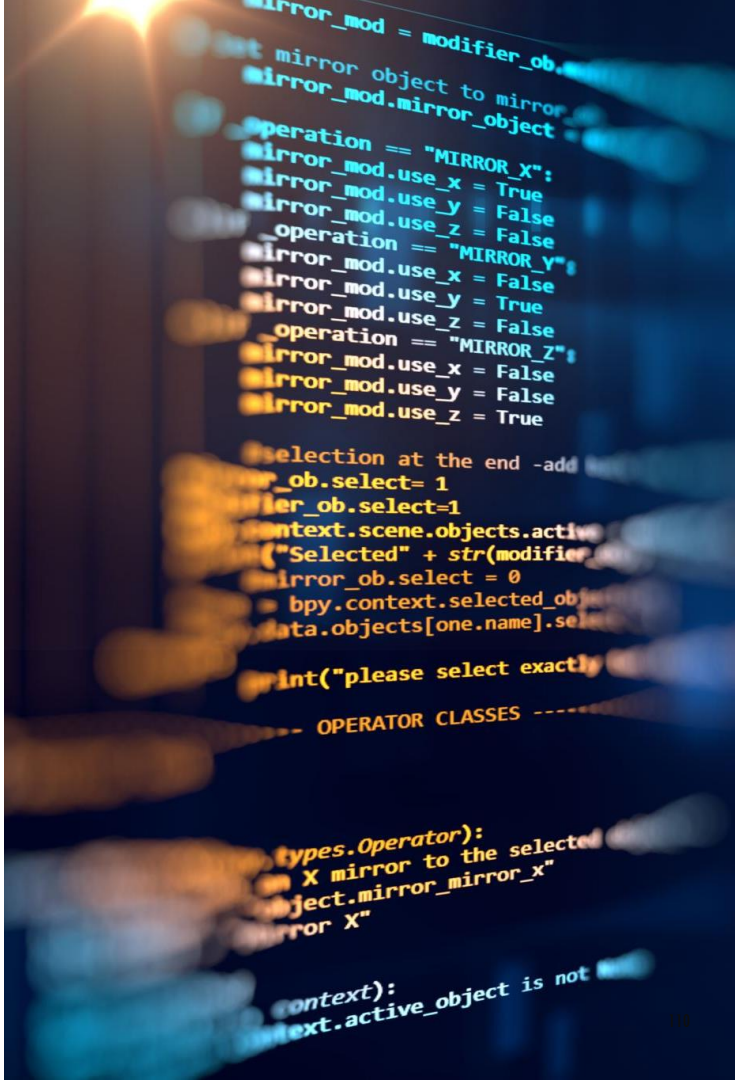
PARADIGMA DECLARATIVO

A principal característica das linguagens com programação declarativa é o foco não estar em como uma execução vai ocorrer, mas sim no resultado a ser atingido. Um dos melhores exemplos para entender esse paradigma são as instruções structured query language (SQL), pois nela são passados para o banco de dados apenas o que se pretende, sem a preocupação sobre como o banco de dados vai executar a instrução, o foco é somente o retorno ou resultado da consulta.

Atualmente, uma das principais linguagens de programação utilizada, o framework JavaScript Angular, é um exemplo de implementação desse paradigma, que, aliás, é muito utilizado em razão do advento dos sistemas Web, no qual o código submete uma execução e espera o retorno.

PARADIGMA ESTRUTURADO

No sentido mais restrito, o conceito de programação estruturada se refere à forma do programa e do processo de codificação. É um conjunto de convenções que o programador pode seguir para produzir o código estruturado, e suas regras de codificação impõem limitações sobre o uso das estruturas básicas de controle, estruturas de composição modular e documentação.



PARADIGMA ESTRUTURADO

As características do paradigma estruturado são:

- programação sem GOTO (eliminação completa ou parcial do comando GOTO, que significa “ir para”);
- programação com apenas três estruturas básicas de controle — sequência, seleção e iteração;
- forma de um programa estruturado;
- aplicação de convenções de codificação estruturada a uma linguagem de programação específica.
- Algumas linguagens com esse paradigma são Pascal e C.

PARADIGMA ORIENTADO A OBJETOS

O paradigma de orientação a objetos surge como o advento da reutilização de código e a facilidade na manutenção. No paradigma de orientação a objetos, o princípio é a construção de código, implementando as entidades do mundo real por meio do conceito de classes que possuem relação entre si. Como o desempenho das aplicações não é uma das grandes preocupações na maioria delas (devido ao poder de processamento dos computadores atuais), a programação orientada a objetos se tornou muito difundida. A programação orientada a objetos está embasada em quatro pilares.

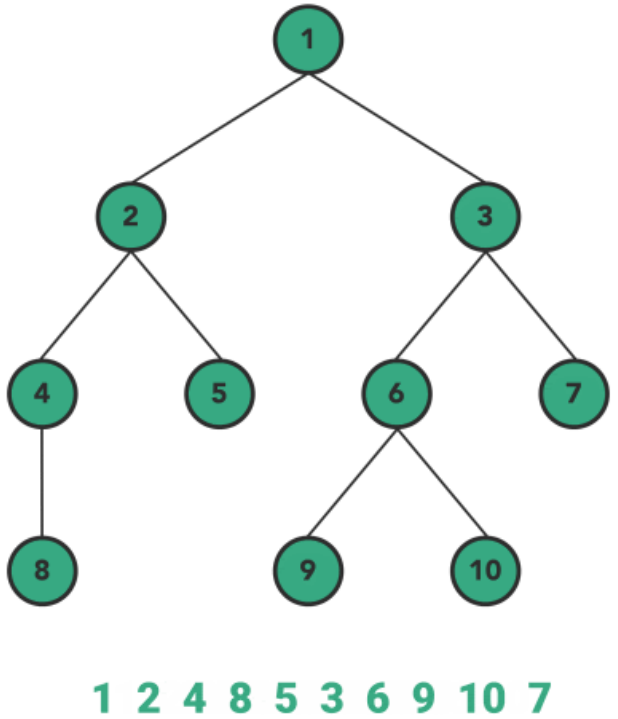


PARADIGMA ORIENTADO A OBJETOS

- **Abstração:** como estamos lidando com objetos do mundo real, por exemplo, carro, casa, pessoa etc.), precisamos imaginar como esses objetos vão se integrar dentro do nosso sistema e modelar seu comportamento abstraindo comportamento e características específicas de cada um.
- **Encapsulamento:** não importa para um código que invoca um método saber como outro vai ser executado, trata-se de uma característica que traz principalmente segurança ao código.
- **Herança:** assim como no mundo real, a herança em programação orientada a objetos seria a capacidade de uma classe herdar de outra métodos e atributos, sendo, portanto, uma característica relacionada ao reuso de código.
- **Polimorfismo:** existem animais capazes de se adaptar a algumas necessidades do mundo real e se comportar de forma diferenciada em alguns casos, essa particularidade também é possível no paradigma de orientação a objetos. Mesmo herdando o comportamento de outra classe, a classe herdeira pode modificar o seu comportamento em determinadas situações.

PROGRAMAÇÃO TOP-DOWN E BOTTOM-UP

O top down é uma técnica de resolução de problemas que envolve a decomposição de um problema complexo em subproblemas mais simples e manejáveis. Este processo continua recursivamente até que cada subproblema seja simples o suficiente para ser resolvido diretamente. Em termos de programação, essa técnica é frequentemente implementada utilizando recursão.



PROGRAMAÇÃO TOP-DOWN E BOTTOM-UP

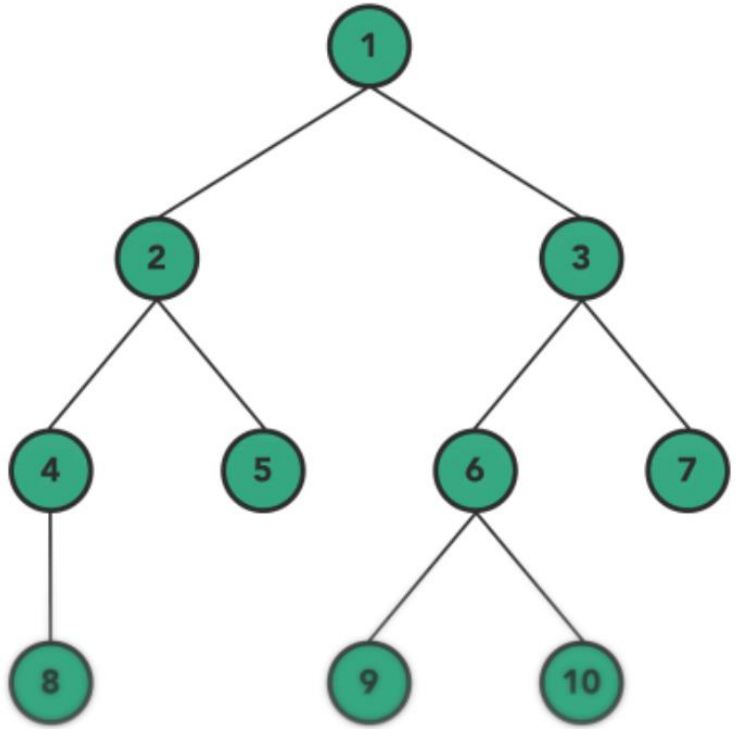
Vantagens da Técnica Top Down:

- **Clareza e organização:** A decomposição de problemas complexos em subproblemas mais simples torna o processo de resolução mais claro e organizado.
- **Estrutura modular**: Proporciona uma estrutura clara e modular para a solução de problemas através da decomposição hierárquica e recursiva.

PROGRAMAÇÃO TOP-DOWN E BOTTOM-UP

Algoritmo Bottom Up

O algoritmo bottom up, ou abordagem “de baixo para cima”, começa com a solução de subproblemas mais simples e constrói soluções para subproblemas maiores até resolver o problema principal. Essa abordagem é frequentemente usada em programação dinâmica e outros métodos iterativos que necessitam de construção de soluções a partir de componentes básicos.



PROGRAMAÇÃO TOP-DOWN E BOTTOM-UP

Vantagens da Técnica Bottom Up:

- **Eficiência de tempo:** Evita a recálculo de sub-problemas já resolvidos, reduzindo a complexidade de tempo.
- **Construção incremental:** Constrói soluções incrementais a partir de subproblemas mais simples, sendo útil em programação dinâmica e outros cenários onde a solução de subproblemas pode ser reutilizada.

COMPARAÇÃO

S. Não.	ABORDAGEM DE CIMA PARA BAIXO	ABORDAGEM DE BAIXO PARA CIMA
1.	concentre-se em dividir o problema em partes menores e mais gerenciáveis	Resolve problemas menores e os integra em um sistema completo
2.	Usado principalmente em linguagens de programação estruturadas como COBOL, Fortran, C, etc.	Usado principalmente em linguagens de programação orientadas a objetos, como C++, C# e Python.
3.	Cada parte é programada separadamente e, portanto, contém redundância.	A redundância é minimizada pelo uso de encapsulamento e ocultação de dados.
4.	a comunicação é menor entre os módulos.	Os módulos devem se comunicar para integrar o sistema.
5.	É usado para depuração e documentação de módulos	É basicamente usado em testes.
6.	Ocorre a decomposição do sistema, dividindo-o em componentes menores.	A composição do sistema acontece pela combinação de componentes de baixo nível em uma estrutura de nível superior.
7.	A principal função do sistema pode ser difícil de identificar.	Às vezes não conseguimos construir um programa a partir do que começamos.
8.	os detalhes da implementação podem variar ao longo do processo	Construir um programa pode ser difícil se os módulos não forem montados em uma ordem lógica

COMPARAÇÃO

S. Não.	ABORDAGEM DE CIMA PARA BAIXO	ABORDAGEM DE BAIXO PARA CIMA
9.	**Prós** • Isolamento mais fácil de erros de interface • É benéfico caso ocorra um erro no topo do programa. • Defeitos no projeto são detectados precocemente e podem ser corrigidos quando um módulo de trabalho inicial do programa estiver disponível.	**Prós** • Fácil de criar condições de teste • Os resultados dos testes são fáceis de observar • É adequado se ocorrerem defeitos na parte inferior do programa.
10.	**Contras-** • Dificuldade em observar a saída do caso de teste. • A escrita do stub é muito importante, pois leva à configuração dos parâmetros de saída. • Quando os stubs estão localizados longe do módulo de nível superior, a escolha de casos de teste e o projeto de stubs se tornam mais desafiadores.	**Contras-** • Não há representação do modelo de trabalho depois que vários módulos foram construídos. • Não há existência do programa como entidade sem a adição do último módulo. • Em um sistema parcialmente integrado, os engenheiros de teste não conseguem observar as funções no nível do sistema. Isso só é possível com a instalação do driver de teste de nível superior.

PROGRAMAÇÃO ITERATIVA

Definição: A programação iterativa resolve problemas repetitivos utilizando estruturas de repetição como for, while ou do-while. O algoritmo executa um bloco de código várias vezes até que uma condição de parada seja satisfeita.

Características:

Usa laços de repetição.

Mais eficiente em termos de uso de memória (não envolve empilhamento de chamadas).

Geralmente mais rápida que a recursiva.

Mais próxima do funcionamento da máquina.Exemplo: Soma de 1 a N com for.

```
int soma_iterativa(int n) {
    int soma = 0;
    for (int i = 1; i <= n; i++) {
        soma += i;
    }
    return soma;
}
```

PROGRAMAÇÃO RECURSIVA

Definição:
A **programação recursiva** resolve problemas chamando a **mesma função dentro dela mesma,** dividindo o problema em versões menores até alcançar um **caso base,** que encerra as chamadas.

Características:

Usa **autochamadas** (funções que se chamam).

Requer **caso base** para evitar chamadas infinitas.

Pode ser mais elegante e clara para problemas naturalmente recursivos (como árvores, grafos, Fibonacci).

Pode consumir mais memória (pela pilha de chamadas).

```c
int soma_recursiva(int n) {
    if (n == 0) return 0;       // caso base
    return n + soma_recursiva(n - 1); // chamada recursiva
}
```

COMPARAÇÃO

Critério	Iterativa	Recursiva
Controle	Laços (for/while)	Chamada da própria função
Eficiência	Geralmente mais eficiente	Pode consumir mais memória
Clareza	Pode ser mais verboso	Pode ser mais elegante
Aplicação comum	Algoritmos simples e diretos	Problemas recursivos (árvores, divide & conquer)

TOP-DOWN

- Exemplo: Verificar se um número é primo e exibir o resultado

```c
#include <stdio.h>

// Declaração (protótipos) das funções auxiliares
int eh_primo(int n);
void imprime_resultado(int n, int resultado);

// Função principal
int main() {
    int numero = 17;

    if (eh_primo(numero)) {
        imprime_resultado(numero, 1);
    } else {
        imprime_resultado(numero, 0);
    }

    return 0;
}

// Definição da função que verifica se é primo
int eh_primo(int n) {
    if (n < 2) return 0;
    for (int i = 2; i <= n / 2; i++) {
        if (n % i == 0) return 0;
    }
    return 1;
}

// Definição da função que imprime o resultado
void imprime_resultado(int n, int resultado) {
    if (resultado)
        printf("%d é primo.\n", n);
    else
        printf("%d não é primo.\n", n);
}
```

BOTTOM-UP

Exemplo: Cálculo de fatorial usando memorização iterativa

```c
#include <stdio.h>

int main() {
    int n = 5;
    int fatorial[10]; // tabela de resultados parciais

    fatorial[0] = 1; // base
    for (int i = 1; i <= n; i++) {
        fatorial[i] = i * fatorial[i - 1]; // construindo de baixo para cima
    }

    printf("Fatorial de %d é %d\n", n, fatorial[n]);
    return 0;
}
```

RECURSIVO VS ITERATIVO

- Comparando os dois estilos de resolver o **problema de Fibonacci**
- Recursivo:

```c
#include <stdio.h>

int fibonacci_rec(int n) {
    if (n == 0) return 0;
    if (n == 1) return 1;
    return fibonacci_rec(n - 1) + fibonacci_rec(n - 2);
}

int main() {
    int n = 6;
    printf("Fibonacci de %d é %d\n", n, fibonacci_rec(n));
    return 0;
}
```

RECURSIVO VS ITERATIVO

- Comparando os dois estilos de resolver o **problema de Fibonacci**
- Iterativo:

```c
#include <stdio.h>

int fibonacci_iter(int n) {
    int a = 0, b = 1, temp;
    for (int i = 2; i <= n; i++) {
        temp = a + b;
        a = b;
        b = temp;
    }
    return n == 0 ? 0 : b;
}

int main() {
    int n = 6;
    printf("Fibonacci de %d é %d\n", n, fibonacci_iter(n));
    return 0;
}
```

PROGRAMAÇÃO ESTRUTURADA

Foco em sequência, decisão (if), e repetição (for, while), sem usar orientação a objetos.
Exemplo: Soma dos elementos de um vetor

```c
#include <stdio.h>

int main() {
    int vetor[] = {1, 2, 3, 4, 5};
    int soma = 0;

    // Sequência
    int tamanho = sizeof(vetor) / sizeof(vetor[0]);

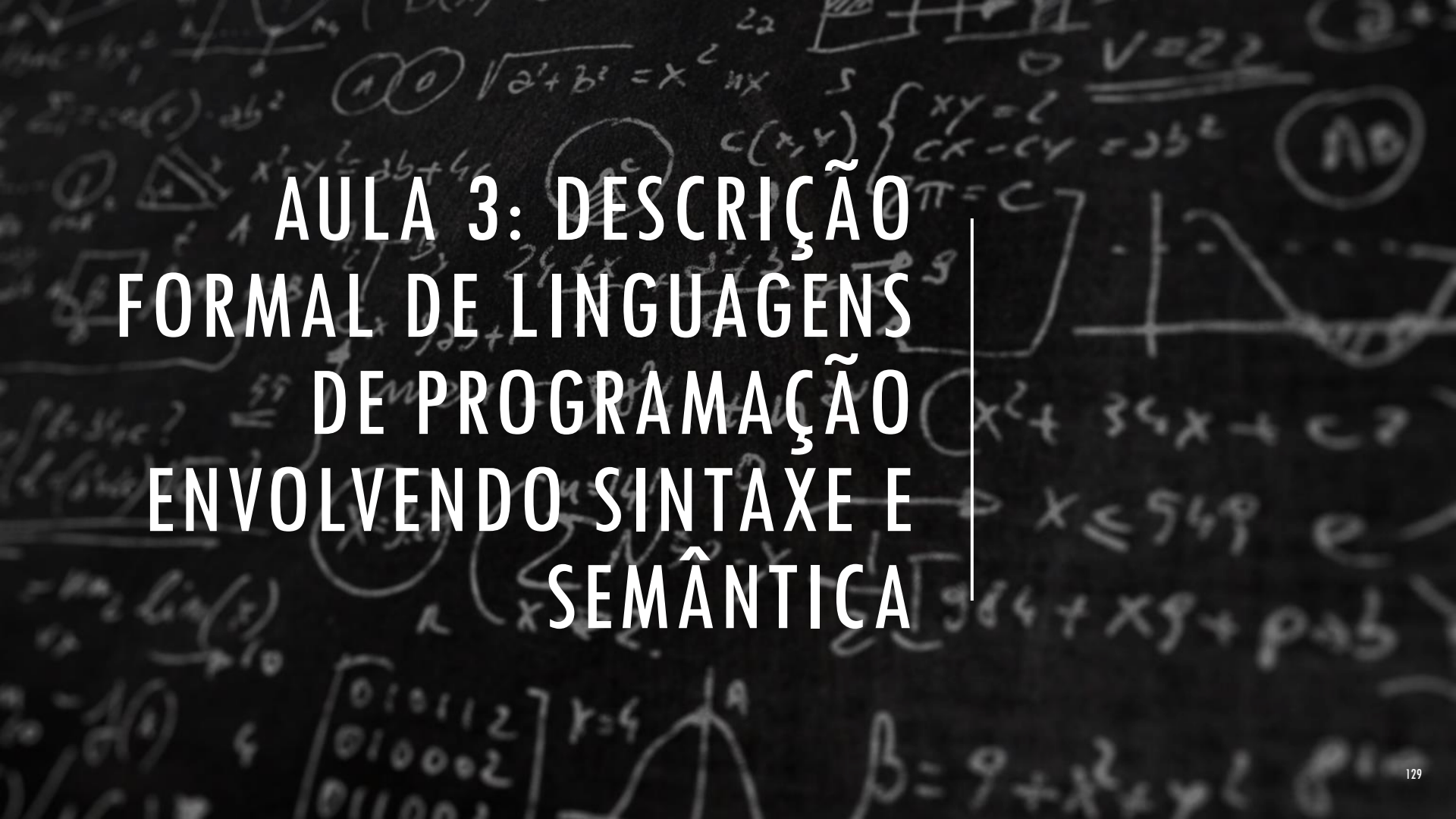
    // Repetição
    for (int i = 0; i < tamanho; i++) {
        // Decisão
        if (vetor[i] % 2 == 0) {
            soma += vetor[i];
        }
    }

    printf("Soma dos pares: %d\n", soma);
    return 0;
}
```

ATIVIDADE

Crie um código em C, que resolva o fatorial de um número inteiro n (ex: $5! = 5 \times 4 \times 3 \times 2 \times 1 = 120$) utilizando as programações:

- Programação Top-Down e Bottom-Up
- Programação Recursiva e Iterativa
- Programação Estruturada



AULA 3: DESCRIÇÃO FORMAL DE LINGUAGENS DE PROGRAMAÇÃO ENVOLVENDO SINTAXE E SEMÂNTICA

OBJETIVOS DA AULA

- Ao final da aula, os alunos devem ser capazes de:
- Compreender a importância da descrição formal em linguagens de programação.
- Diferenciar entre **sintaxe** e **semântica**.
- Interpretar regras de gramáticas formais (BNF/EBNF).
- Relacionar a semântica com comportamentos de programas.
- Aplicar noções básicas de semântica operacional e denotacional.

INTRODUÇÃO

- **Exemplo 1** – Correto na sintaxe e semântica

Frase:"O gato dorme na cadeira.“
Sintaxe: A frase está bem estruturada (sujeito + verbo + complemento).
Semântica: O sentido também é válido, já que gatos realmente podem dormir em cadeiras.

Exemplo 2 – Correto na sintaxe, mas errado na semântica

- **Frase:**"A pedra comeu o bolo.“
- **Sintaxe:** Estrutura está correta (sujeito + verbo + complemento).
- **Semântica:** Não faz sentido, pois pedras não comem bolos.

INTRODUÇÃO

Exemplo 3 – Errado na sintaxe, mas claro na semântica

- **Frase:** "Dormir o gato cadeira na.“
- **Sintaxe:** Ordem incorreta, as regras gramaticais não estão seguidas.
- **Semântica:** Apesar da desordem, conseguimos entender que é sobre um gato dormindo em uma cadeira.

- **Resumindo:**
- **Sintaxe** = forma, estrutura correta da frase.
- **Semântica** = sentido, significado do que foi dito.Quer que eu

INTRODUÇÃO

- A **sintaxe** de uma linguagem de programação determina como a estrutura gramatical de um programa deve ser – sem qualquer preocupação com o significado de cada trecho, com compatibilidade de tipos. **A semântica** de uma linguagem determina o significado formal de cada trecho de código. Como exemplo extremamente simples, tomamos a linguagem que usamos usualmente para expressões. A expressão)

$$+\ 3\{/\ +\ /5$$

- certamente não faz sentido – e o problema com ela é sintático, uma vez que há delimitadores (parênteses e chaves) dispostos de forma incorreta, e operadores em sequência, sem operandos. A expressão

$$2 + x = \text{true}$$

- também não faz sentido, mas está sintaticamente correta. O problema dela é que o lado esquerdo só pode denotar números, e o lado direito denota um valor-verdade . A expressão está semanticamente errada.
- Este texto é sobre a descrição da semântica de linguagens de programação – deixaremos de lado os aspectos sintáticos das linguagens.

ABORDAGENS PARA SEMÂNTICA

Trataremos neste texto de três abordagens diferentes para descrever a semântica de linguagens. Nesta seção damos apenas uma idéia de como é cada uma delas.

- **Semântica Operacional** – a linguagem é descrita através de sua interpretação em uma máquina virtual hipotética. Esta máquina virtual é construída de forma a facilitar a descrição formal de linguagens de programação e as demonstrações de suas propriedades. Damos um exemplo. O par <c, σ> é uma configuração de uma máquina, e significa que temos o comando (ou trecho de programa c para ser executado, no estado σ (um estado consiste dos valores atribuídos às variáveis). A seguir temos uma regra:

$$\frac{\langle c_1, \sigma\rangle \to \sigma', \langle c_2, \sigma'\rangle \to \sigma''}{\langle c_1; c_2, \sigma\rangle \to \sigma''}$$

ABORDAGENS PARA SEMÂNTICA

A regra diz que se:

- quando executamos c1 no estado σ terminamos no estado σ';
- quando executamos c2 no estado σ' terminamos no estado σ'',
- então executar a sequência c1; c2 no estado σ nos levará ao estado σ''

SEMÂNTICA DENOTACIONAL

- **cada trecho de programa é descrito como uma função.** Um programa, portanto, mapeia um estado (valores de variáveis e outros elementos relevantes à execução do programa) em um novo estado.

$$C[\![c_1; c_2]\!] = \text{se } c_1 \text{ para, } c_2(c_1),$$
$$\text{senão } \perp$$

- Aqui, C[[c]] denota o significado semântico do comando c; este significado é uma função que leva estados do programa em outros estados. A regra diz que o significado de c1; c2 é ⊥ quando c1 entra em loop; caso contrário, o estado é a função composta c1 ◦ c2. A semântica denotacional de uma linguagem imperativa, por exemplo, é a descrição da função associada a cada comando

SEMÂNTICA AXIOMÁTICA

- **o significado de cada trecho de um programa é descrito pelo seu efeito no estado do programa** – o estado é composto pelos valores das variáveis. O efeito de cada trecho é descrito por pré-condições e pós-condições. A notação é {P}c{Q}, onde P são as pré-condições e Q são as pós-condições. Damos um pequeno exemplo:

$$\{x \leq y\} y := y + 1 \{x < y\}$$

- Aqui declaramos que em qualquer estado onde valha x ≤ y, se executarmos "y := y + 1", passará a valer x < y. **A semântica axiomática de uma linguagem é um conjunto de regras de pré-condição e pós-condição.**
- Nenhuma das diferentes abordagens para semântica é melhor que as outras. Elas têm características diferentes, e são aplicáveis em situações diferentes – são, portanto, complementares

SINTAXE: CONCRETA E ABSTRATA

- **A sintaxe concreta** de uma linguagem de programação é dada por uma gramática livre de contexto, e é útil na análise sintática de programas da linguagem. Na sintaxe concreta incluímos elementos cuja função é remover ambiguidades, como parênteses, delimitadores do tipo "begin/end", e regras para forçar precedência de operadores. Usualmente, a sintaxe concreta de uma linguagem de programação é dada pela especificação de uma gramática livre de contexto, usando a notação Backus-Naur (BNF).

- **A sintaxe abstrata** de uma linguagem é normalmente mais simples, e pode não ser suficiente para realizar a análise sintática (parsing) da linguagem. Isto porque não inclui elementos extras que normalmente existem na gramática concreta para eliminar ambiguidades. A estrutura de dados que descreve a árvore abstrata construída por um compilador é um exemplo de sintaxe abstrata.

SINTAXE: CONCRETA E ABSTRATA

- **Exemplo.** Um comando de repetição com **sintaxe concreta**
- <while> :: while <exp-bool> do begin <cmd> end

- poderia ter sua **sintaxe abstrata** descrita pela gramática a seguir.
- <while> :: while <exp-bool> do <cmd>

O QUE É UMA DESCRIÇÃO FORMAL?

É o uso de **conceitos matemáticos rigorosos** (como conjuntos, gramáticas, funções, lógicas formais) para **definir com precisão** como uma linguagem de programação funciona — ou seja, quais construções são válidas e o que elas significam.

Exemplo

- Frase em linguagem natural:“Se o aluno tiver nota maior que 7, ele passa.”
- Versão em linguagem formal (em pseudocódigo):
- if (nota > 7) { resultado = "Aprovado"; }

POR QUE LINGUAGENS DE PROGRAMAÇÃO PRECISAM DE ESPECIFICAÇÕES FORMAIS?

- **Precisão**

As linguagens de programação precisam ser **sem ambiguidade.** Um mesmo código deve ter **um único significado** para qualquer compilador ou interpretador.

- **Construção de compiladores**

  Para que um compilador saiba **analisar, interpretar e traduzir** um programa corretamente, ele precisa conhecer as regras exatas de **sintaxe e semântica** da linguagem.

- **Verificação automática**

  Ferramentas como linters, validadores de código e analisadores semânticos dependem de regras formais para **verificar se o código está correto.**

- **Base para o projeto de linguagens**

  Ajuda projetistas a **definir linguagens novas,** com clareza sobre as regras e comportamentos esperados.

- **Ensino e compreensão**

  Para ensinar como linguagens funcionam (como em cursos de compiladores ou teoria da computação), é essencial usar uma descrição formal.

DIFERENÇA ENTRE LINGUAGEM NATURAL E LINGUAGEM FORMAL

Característica	Linguagem Natural	Linguagem Formal
Exemplo	"Se chover, levo o guarda-chuva."	`if (rain) { bringUmbrella(); }`
Criada por	Seres humanos	Humanos, mas com regras **matemáticas**
Interpretação	Ambígua, depende do contexto	Determinística, sem ambiguidade
Flexibilidade	Alta (gírias, regionalismos, erros tolerados)	Baixa (qualquer erro pode invalidar)
Uso em computadores	Não diretamente processável	Projetada para ser interpretada por máquinas
Regras gramaticais	Muitas vezes informais	Sempre bem definidas (BNF, autômatos etc.)

SINTAXE

A **sintaxe** em linguagens de programação define **como os símbolos podem ser combinados** para formar estruturas válidas de código (como expressões, comandos, declarações etc.).

Ela é análoga à gramática em linguagens naturais — define o que é uma “frase bem formada”.

Alfabetos e Cadeia
Alfabeto (Σ)É um conjunto finito de símbolos.
Em linguagens de programação, pode incluir:

- Letras (a a z, A a Z)
- Dígitos (0 a 9)
- Símbolos especiais (+, -, *, =, ;, {, } etc.)
- Exemplo de alfabeto:
- Σ = { a, b, +, *, (,), 0, 1 }

CADEIA (STRING)

É uma sequência finita de símbolos do alfabeto.
Pode representar um identificador, uma expressão, um comando etc.

- Exemplo de cadeia sobre
- Σ:a + b * (a + 1)

DEFINIÇÃO DE LINGUAGENS FORMAIS

- Uma linguagem formal L é um conjunto de cadeias válidas sobre um alfabeto Σ, definidas segundo regras sintáticas (gramática).

Formalmente:

- $L \subseteq \Sigma^*$ (o conjunto das cadeias possíveis formadas com símbolos de Σ)

- As linguagens de programação (como C, Java, Python) são linguagens formais — e seu conjunto de sentenças válidas é definido por gramáticas formais.

GRAMÁTICAS FORMAIS: BNF E EBNF

BNF – Backus-Naur Form

- BNF é uma forma de **especificar regras de sintaxe** usando produções do tipo:

```
<não-terminal> ::= sequência de terminais e não-terminais
```

- **Terminais:** símbolos do alfabeto (ex: +, *, x)
- **Não-terminais:** categorias sintáticas (ex: <expressao>, <numero>)

Exemplo:

```
<expressao> ::= <termo> | <expressao> + <termo>
<termo>     ::= <fator> | <termo> * <fator>
<fator>     ::= ( <expressao> ) | <numero>
<numero>    ::= 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9
```

EBNF – EXTENDED

EBNF é uma extensão mais compacta e legível da BNF. Adiciona:

- * → repetição (zero ou mais vezes)
- + → repetição (uma ou mais vezes)
- [] → opcional
- {} → repetições
- () → agrupamento

Exemplo EBNF:

```
expressao = termo, { "+", termo } ;
termo     = fator, { "*", fator } ;
fator     = "(", expressao, ")" | numero ;
numero    = digit ;
digit     = "0" | "1" | ... | "9" ;
```

EXEMPLO

- Para a expressão 3 + 4 * 2, a derivação seria:

```
<expr>
→ <expr> + <term>
→ <term> + <term>
→ <factor> + <term>
→ 3 + <term>
→ 3 + <term> * <factor>
→ 3 + 4 * <factor>
→ 3 + 4 * 2
```

RESUMO PARA FIXAÇÃO

Conceito	Explicação
Alfabeto (Σ)	Conjunto de símbolos válidos da linguagem
Cadeia	Sequência finita de símbolos de Σ
Linguagem Formal	Conjunto de cadeias válidas definidas por uma gramática
BNF	Forma clássica de definir regras com não-terminais
EBNF	Forma estendida e mais compacta da BNF

SEMÂNTICA EM LINGUAGENS DE PROGRAMAÇÃO

O que é Semântica?
A semântica define o significado das construções sintáticas de uma linguagem de programação.
Ou seja, explica o que o programa faz, não apenas como ele é escrito.

- Enquanto a sintaxe diz que x + 1 é uma expressão válida,
- a semântica diz que "essa expressão soma 1 ao valor da variável x".

Por que a semântica é importante?
Para interpretar e executar programas corretamente.
Para provar propriedades de programas (como correção ou segurança).
Para projetar linguagens novas com comportamento previsível.
Para verificação formal, como prova de ausência de erros.

SEMÂNTICA OPERACIONAL

"Como o programa é executado passo a passo“

- Define como o estado do programa muda à medida que ele é executado.
- Muito usada em interpretadores, simuladores e depuradores.
- Fornece uma interpretação operacional — como se fosse uma simulação.

```
x = 2
y = x + 1
```

- **Semântica operacional:**
- Atribui 2 a x.
- Avalia x + 1, obtendo 3.
- Atribui 3 a y.

SEMÂNTICA DENOTACIONAL

"Qual é o resultado matemático do programa“

- Associa funções matemáticas aos elementos da linguagem.
- Cada construção sintática é interpretada como uma função sobre domínios matemáticos (como números, funções, listas etc.).
- Muito usada em provas formais de propriedades, análise estática, etc.

Exemplo:

$$[\![x + 1]\!](\sigma) = \sigma(x) + 1$$

Aqui, σ é o ambiente (estado de variáveis), e ⟦exp⟧ é a interpretação semântica.

SEMÂNTICA AXIOMÁTICA

"Como provar que o programa é correto logicamente"

- Usa **lógica matemática** (como lógica de predicados) para expressar **propriedades** de programas.
- Introduz o conceito de **tripla de Hoare**:

```
{ P } C { Q }
```

Significa:

- Se a pré-condição P for verdadeira antes da execução do comando C,então a pós-condição Q será verdadeira após sua execução.

Exemplo:

```
{ x = 2 }
y = x + 1;
{ y = 3 }
```

RESUMO COMPARATIVO

Tipo de Semântica	Pergunta que responde	Forma de Descrição	Usos comuns
Operacional	Como o programa executa passo a passo?	Transições de estado	Interpretação, execução, depuração
Denotacional	Qual o significado matemático da construção?	Funções matemáticas	Análise, prova de equivalência
Axiomática	Quais propriedades podem ser provadas logicamente?	Lógica de predicados (Hoare, invariantes)	Verificação formal, segurança, testes

EXEMPLO INTEGRADO NAS TRÊS SEMÂNTICAS

```
x = 2;
y = x + 1;
```

Abordagem	Descrição
Operacional	Atribui 2 a `x`; avalia `x + 1` como `3`; atribui 3 a `y`
Denotacional	`⟦x + 1⟧(σ) = σ(x) + 1`; com σ(x)=2, temos `⟦x + 1⟧ = 3`
Axiomática	`{x = 2} y = x + 1; {y = 3}`

EXERCÍCIO

- Um trecho simples de código (ex: x = 3; y =x^2 + x * 2;)

Tarefa:

- Interpretar com semântica operacional (passo a passo).
- Definir a função denotacional que representa y =x^2 + x * 2.
- Escrever uma tripla de Hoare que representa o comportamento

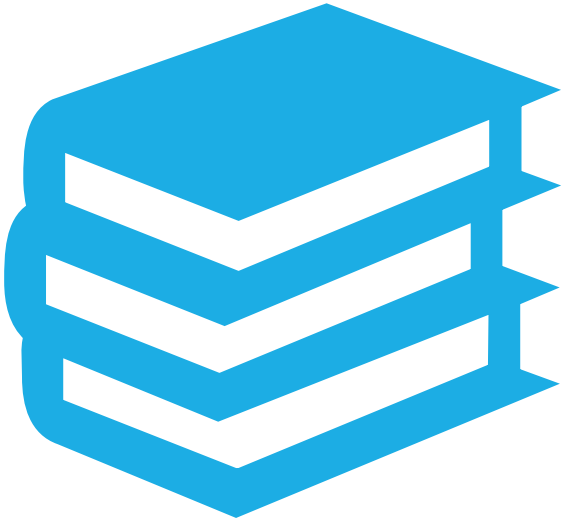
EXEMPLO

```
# Interpretação semântica simples de uma expressão
a = 2
b = 3
c = 4
expressao = "a + b * c"

# Avaliação semântica (simples)
resultado = a + b * c  # Semântica operacional: executa passo a passo
print(f"O resultado de {expressao} é {resultado}")
```

- Utilizando a expressão do código:

1. Definir o alfabeto.
2. Criar uma gramática (BNF/EBNF).
3. Definir a semântica representando a análise operacional, denotacional e axiomática



AULA 4: PRINCÍPIOS DE CONSTRUÇÃO DE COMPILADORES

FUNDAMENTOS DE COMPILAÇÃO E INTERPRETAÇÃO

Posto de forma simples, um compilador é um programa que lê um programa escrito numa linguagem — a linguagem fonte — e o traduz num programa equivalente numa outra linguagem — a linguagem alvo (ver a Fig). Como importante parte desse processo de tradução, o compilador relata a seu usuário a presença de erros no programa fonte.



SGAS Quadra 607 - Módulo 49
Via L2 Sul, Brasília - DF
CEP 70.200-670
(61) 3535-6565

COMPILAÇÃO

A compilação, de fato, é uma tradução. Nesse processo, inicialmente, há um código escrito em uma linguagem de programação de alto nível. Este código será enviado ao compilador (que é nada mais nada menos que um programa), que possui a capacidade de traduzir códigos de uma determinada linguagem para outra.

SGAS Quadra 607 - Módulo 49
Via L2 Sul, Brasília - DF
CEP 70.200-670
(61) 3535-6565

INTERPRETAÇÃO

O processo de interpretação é um pouco diferente e peculiar. Isso porque, nesse processo, não há necessidade de gerar intermediários, como um código-objeto (arquivo gerado com o código compilado) ou um executável. A interpretação também traduz o código, assim como a compilação, porém, existem algumas diferenças.

Ao utilizar o método de compilação, o compilador irá ler o código, fazer todas as análises sintáticas e demais processos, para, por fim, gerar um arquivo código-objeto ou um arquivo executável. Já na interpretação, nenhum arquivo ou código é gerado, e sim uma tradução instantânea, em tempo de execução.

SGAS Quadra 607 - Módulo 49
Via L2 Sul, Brasília - DF
CEP 70.200-670
(61) 3535-6565

INTERPRETAÇÃO

Quando é salvo um programa, este recebe uma extensão de acordo com linguagem, então ao fazer isso, quando é chamando o interpretador da linguagem, que está instalado em seu computador. O interpretador irá ler o seu código linha a linha em tempo de execução e irá traduzir cada linha para uma linguagem alvo (que geralmente será *bytecode* ou linguagem de máquina). No mesmo momento que ele traduz, ele executa o código gerado, sem processos adicionais. O que é bem diferente do processo de compilação, em que você precisa primeiro compilar para somente depois executar.

SGAS Quadra 607 - Módulo 49
Via L2 Sul, Brasília - DF
CEP 70.200-670
(61) 3535-6565

INTERPRETAÇÃO

Desvantagens da Interpretação

Alto consumo de memória, por ser um processo mais complexo, é necessário que seja alocado mais espaço.

Também há impacto direto na velocidade do código. Linguagens interpretadas, por padrão, tendem a ser mais lentas que as compiladas.

As linguagens interpretadas costumam ser de altíssimo nível e, por isso, possuem autogerenciamento de memória, tipagem dinâmica, funções prontas e menos controle sobre o sistema operacional, o que faz o processo mais lento e precisa mais memória.

SGAS Quadra 607 - Módulo 49
Via L2 Sul, Brasília - DF
CEP 70.200-670
(61) 3535-6565

COMPILAÇÃO

Vantagens da compilação em relação a interpretação.

A execução é mais rápida porque não precisa de tradução na execução.

Economia de memória na execução, pois não requer a carga de máquinas virtuais na memoria

Maior controle sobre o código gerado

Pode efetuar otimização de código

Geram um programa objeto bem mais eficiente, pois esse interage diretamente com o computador.

SGAS Quadra 607 - Módulo 49
Via L2 Sul, Brasília - DF
CEP 70.200-670
(61) 3535-6565

COMPILAÇÃO

Desvantagens da compilação em relação a interpretação.

Inadequada para usuário novatos, pois cada vez que se se comete um erro deve-se compilar o programa inteiro, já o interpretador, ao perceber erros durante a execução do programa, pausa, o erro então deve ser corrigido.

Perde-se a referência com o código fonte, em caso de erro é difícil apontar a consequência do erro.

Construção mais complexa

SGAS Quadra 607 - Módulo 49
Via L2 Sul, Brasília - DF
CEP 70.200-670

(61) 3535-6565

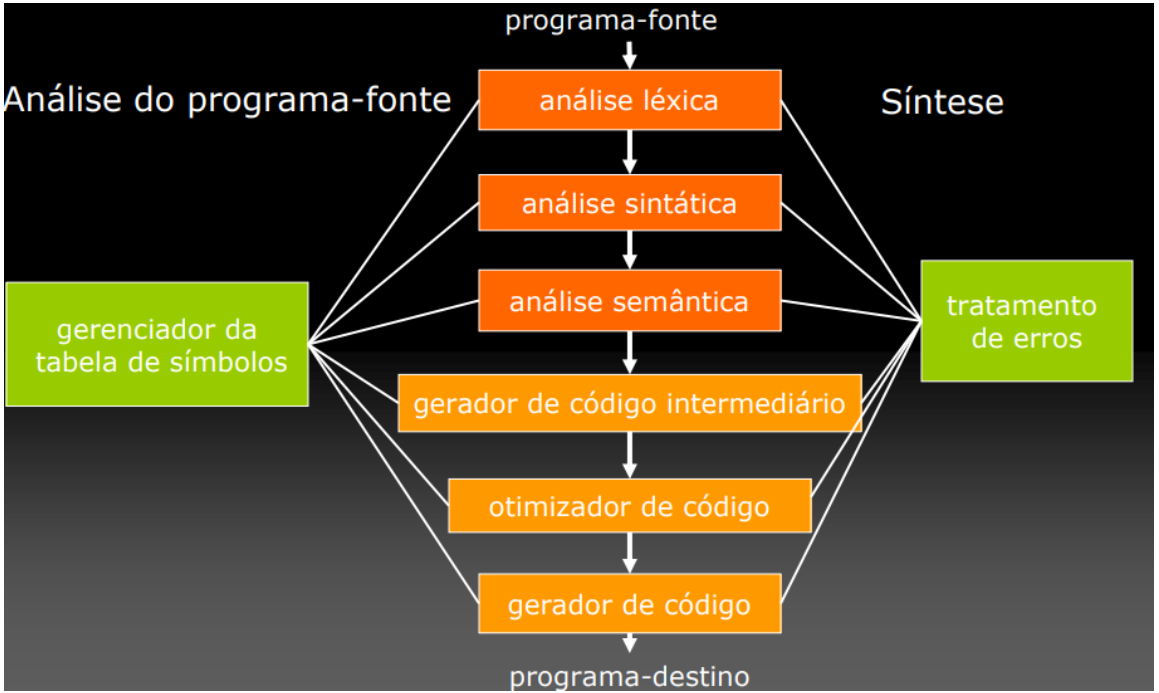
FUNDAMENTOS DE COMPILAÇÃO E INTERPRETAÇÃO

Existem milhares de linguagens fonte, que vão das linguagens de programação tradicionais, tais como Fortran e Pascal, às linguagens especializadas que emergiram virtualmente em quase todas as áreas de aplicação de computadores. As linguagens alvo são igualmente variadas; uma linguagem alvo pode ser uma outra linguagem de programação ou a linguagem de máquina de qualquer coisa entre um microprocessador e um supercomputador.

Os compiladores são algumas vezes classificados como de uma passagem, de passagens múltiplas, de carregar e executar, depuradores ou otimizantes, dependendo de como tenham sido construídos ou que função se suponha que devam realizar.

SGAS Quadra 607 - Módulo 49
Via L2 Sul, Brasília - DF
CEP 70.200-670
(61) 3535-6565

FUNDAMENTOS DE COMPILAÇÃO E INTERPRETAÇÃO



SGAS Quadra 607 - Módulo 49
Via L2 Sul, Brasília - DF
CEP 70.200-670
(61) 3535-6565

FUNDAMENTOS DE COMPILAÇÃO E INTERPRETAÇÃO

Esta etapa envolve a decomposição do programa-fonte em uma representação interna que pode ser manipulada pelo compilador.

1. **Análise Léxica**: Tem como função converter o texto do programa-fonte em uma sequência de tokens. Tokens são as menores unidades de significado, como palavras-chave, identificadores, operadores e literais, assim a sequência de tokens que será passada para a análise sintática.

2. **Análise Sintática**: Analisa a sequência de tokens para determinar sua estrutura gramatical conforme definida pela gramática da linguagem de programação

3. **Análise Semântica**: É encarregada de verificar a correção semântica do programa, garantindo que ele faça sentido no contexto da linguagem (ex: verificação de tipos, escopo de variáveis, etc.). Posteriormente sintetiza a informação realizando a transformação da representação interna do programa em código objeto ou código executável.

4. **Gerador de Código Intermediário**: convertendo a representação interna do programa em uma representação intermediária, que é uma forma mais abstrata de código que pode ser facilmente otimizada.

5. **Otimizador de Código**: Sua função é melhorar o código intermediário para torná-lo mais eficiente (ex: melhorar o desempenho, reduzir o uso de memória), gerando um código intermediário otimizado.

6. **Gerador de Código**: Converte o código intermediário (otimizado ou não) em código de máquina ou código objeto, que pode ser executado pelo hardware do computador, gerando assim um objeto ou um executável

SGAS Quadra 607 - Módulo 49
Via L2 Sul, Brasília - DF
CEP 70.200-670
(61) 3535-6565

FUNDAMENTOS DE COMPILAÇÃO E INTERPRETAÇÃO

Componentes de Suporte

- **Gerenciador da Tabela de Símbolos**: Sua função é manter informações sobre os símbolos do programa (variáveis, funções, etc.), como tipo, escopo e localização na memória. Realiza uma integração utilizado por todas as fases de análise e síntese para garantir a coerência e correta referência dos símbolos.

- **Tratamento de Erros**: sua função é detectar, reportar e, se possível, recuperar-se de erros encontrados durante todas as fases do processo de compilação. Ele Interage com todas as etapas do compilador para garantir a correta detecção e manejo de erros.

Processo Completo

- **Entrada**: Programa-fonte.

- **Processo**: Passa por análise léxica, sintática e semântica, seguido pela geração de código intermediário, otimização e finalmente geração de código.

- **Saída**: Programa-destino (código objeto ou executável).

SGAS Quadra 607 - Módulo 49
Via L2 Sul, Brasília - DF
CEP 70.200-670
(61) 3535-6565

PARTES DA COMPILAÇÃO

Na compilação existem duas partes, que são:

Análise de programa fonte

Síntese para a derivação do código objeto.

SGAS Quadra 607 - Módulo 49
Via L2 Sul, Brasília - DF
CEP 70.200-670

(61) 3535-6565

ANÁLISE:

Divide o programa fonte nas partes constituintes e cria uma representação intermediária do mesmo.

O compilador lê o texto escrito em linguagem fonte e verifica se está escrito de acordo com as construções da linguagem

São armazenadas internamente ao compilador uma representação da gramática que descreve a forma das construções válidas e uma representação das regras semânticas

Erros detectados na análise devem ser reportados ao programador

É feita a geração de uma representação intermediária

SGAS Quadra 607 - Módulo 49
Via L2 Sul, Brasília - DF
CEP 70.200-670
(61) 3535-6565

ANÁLISE:

Está constituída em três fases.

Análise léxica

Análise sintática

Análise semântica



SGAS Quadra 607 - Módulo 49
Via L2 Sul, Brasília - DF
CEP 70.200-670
(61) 3535-6565

ANÁLISE LÉXICA

Identifica-se sequências de caracteres que constituem unidades léxicas, chamadas tokens, onde são verificados se os caracteres lidos pertencem ao alfabeto da linguagem de programação aplicada, não considera espaços ou comentários.

Os tokens são:

Palavras reservadas

Delimitadores

Identificadores

Símbolos

SGAS Quadra 607 - Módulo 49
Via L2 Sul, Brasília - DF
CEP 70.200-670
(61) 3535-6565

ANÁLISE LÉXICA

Por exemplo, suponha que um programa fonte contenha o comando de atribuição

```
position = initial + rate * 60
```
(1.1)

Os caracteres nessa atribuição poderiam ser agrupados nos seguintes lexemas e mapeados para os seguintes tokens passados ao analisador sintático:

1. `position` é um lexema mapeado em um token ⟨**id**, 1⟩, onde **id** é um símbolo abstrato que significa *identificador* e 1 aponta para a entrada da tabela de símbolos onde se encontra `position`. A entrada da tabela de símbolos para um identificador mantém informações sobre o identificador, como seu nome e tipo.
2. O símbolo de atribuição = é um lexema mapeado para o token ⟨=⟩. Como esse token não precisa de um valor de atributo, omitimos o segundo componente. Poderíamos ter usado qualquer símbolo abstrato, como **atribuir** para o nome do token, mas, por conveniência de notação, escolhemos usar o próprio lexema como nome do símbolo abstrato.
3. `initial` é um lexema mapeado para o token ⟨**id**, 2⟩, onde 2 aponta para a entrada da tabela de símbolos onde se encontra `initial`.
4. + é um lexema mapeado para o token ⟨+⟩.
5. `rate` é um lexema mapeado para o token ⟨**id**, 3⟩, onde o valor 3 aponta para a entrada da tabela de símbolos onde se encontra `rate`.
6. * é um lexema mapeado para o token ⟨*⟩.
7. `60` é um lexema mapeado para o token ⟨60⟩[1].

Os espaços que separam os lexemas são descartados pelo analisador léxico.

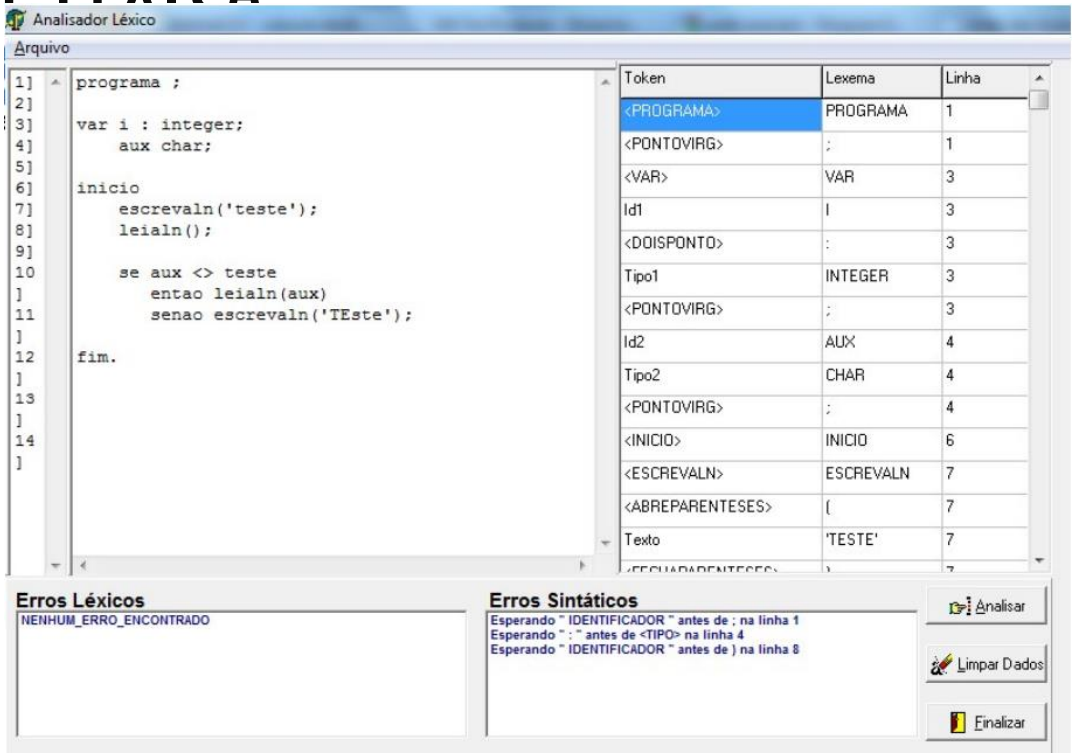
A Figura 1.7 mostra a representação do comando de atribuição (1.1) após a análise léxica como uma seqüência de tokens

⟨**id**, 1⟩ ⟨=⟩ ⟨**id**, 2⟩ ⟨+⟩ ⟨**id**, 3⟩ ⟨*⟩ ⟨60⟩ (1.2)

SGAS Quadra 607 - Módulo 49
Via L2 Sul, Brasília - DF
CEP 70.200-670

(61) 3535-6565

ANÁLISE LÉXICA



SGAS Quadra 607 - Módulo 49
Via L2 Sul, Brasília - DF
CEP 70.200-670
(61) 3535-6565

ANÁLISE SINTÁTICA

Utiliza os tokens produzidos pela análise léxica e verifica a formação do programa com o uso de GLC (gramáticas livres de contexto)

A partir dos tokens cria uma representação intermediária tipo árvore (árvore sintática), a qual está implícita nas chamadas das rotinas recursivas que se executam a análise sintática

mostra a estrutura gramatical da sequência de tokens

SGAS Quadra 607 - Módulo 49
Via L2 Sul, Brasília - DF
CEP 70.200-670
(61) 3535-6565

ANÁLISE SINTÁTICA

Deteção de erros de sintaxe, identificando a posição e o tipo de erro ocorrido

Recupera erros e prossegue com a análise do código.

Exemplo:

While <expressão> do <comando>

A estrutura <expressão> deve apresentar-se sintaticamente correta, e sua avaliação deve retornar um valor do tipo lógico

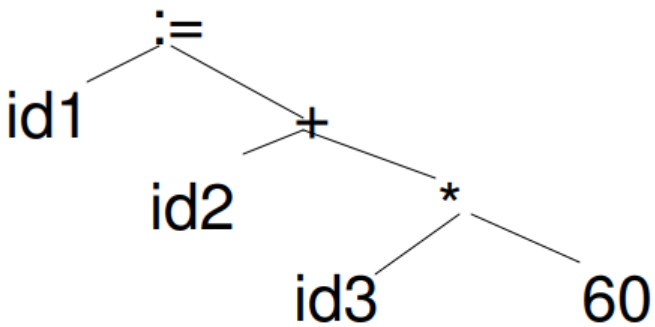
SGAS Quadra 607 - Módulo 49
Via L2 Sul, Brasília - DF
CEP 70.200-670

(61) 3535-6565

ANÁLISE SINTÁTICA

montante := depósito + taxa * 60

id1 := id2 + id3 * 60



Exemplo 2:

raiz roturala com = indica que evemos somar o resultado dessa ultiplicação com o valor id2 e mazenar numa associação entificada como montante.

SGAS Quadra 607 - Módulo 49
Via L2 Sul, Brasília - DF
CEP 70.200-670
(61) 3535-6565

ANÁLISE SEMÂNTICA.

Verifica a validade e sentido da linguagem a partir da árvore e a tabela de símbolos.

Verifica os erros semânticos no programa fonte e captura informações de tipo para a faze subsequente de geração de código

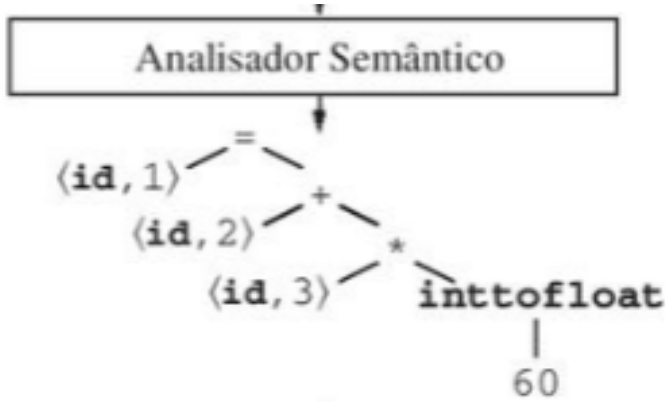
Faz verificação quanto à compatibilidade de tipos, se um identificador não foi declarado

Verifica se as estruturas semânticas fazem sentido

SGAS Quadra 607 - Módulo 49
Via L2 Sul, Brasília - DF
CEP 70.200-670
(61) 3535-6565

ANÁLISE SEMÂNTICA.

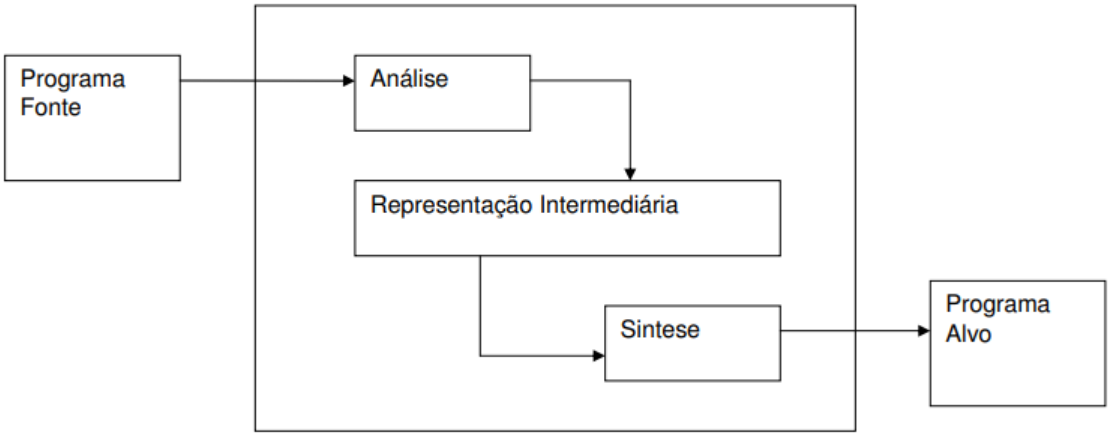
Exemplo

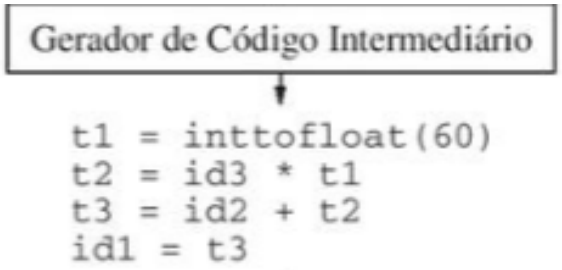


SGAS Quadra 607 - Módulo 49
Via L2 Sul, Brasília - DF
CEP 70.200-670
(61) 3535-6565

GERADOR DE CÓDIGO INTERMEDIÁRIO

Durante o processo de análise, o computador pode gerar uma representação intermediaria do programa fonte





SGAS Quadra 607 - Módulo 49
Via L2 Sul, Brasília - DF
CEP 70.200-670
(61) 3535-6565

GERADOR DE CÓDIGO INTERMEDIÁRIO

Vantagens:

Facilita o trabalho de portar programas para máquinas de arquitetura diferente

Algumas formas de representação intermedira são passíveis de otimização, facilitando e antecipando o trabalho de otimização do código objeto que é feita durante o processo de síntese.

O código gerado é de baixo nível ou linguagem de máquina, e deve ser facilmente produzida e facilmente traduzida para a máquina alvo.

SGAS Quadra 607 - Módulo 49
Via L2 Sul, Brasília - DF
CEP 70.200-670

(61) 3535-6565

GERADOR DE CÓDIGO INTERMEDIÁRIO

Propriedades:

Cada instrução de três endereços possui, no máximo um operador, além do de atributo, então ao realizar essa instrução o compilador deve decidir sobre a ordem em que as instruções devem ser realizadas.

Para isso gera uma variável temporária para receber o valor computado em cada instrução.

SGAS Quadra 607 - Módulo 49
Via L2 Sul, Brasília - DF
CEP 70.200-670
(61) 3535-6565

OTIMIZAÇÃO DE CÓDIGO.

Transforma o código intermediário com o objetivo de produzir um código objeto melhor, normalmente mais rápido.

Uma otimização para o exemplo é eliminar a operação inttofloat e usar o valor de ponto flutuante, eliminando assim t3 da expressão original

```
t1 = id3 * 60.0
id1 = id2 + t1
```


SGAS Quadra 607 - Módulo 49
Via L2 Sul, Brasília - DF
CEP 70.200-670

(61) 3535-6565

GERAÇÃO DE CÓDIGO.

A geração de código normalmente é um código de máquina ou código de montagem.

São reservadas memorias para variáveis e constantes mediante registradores

Fase difícil

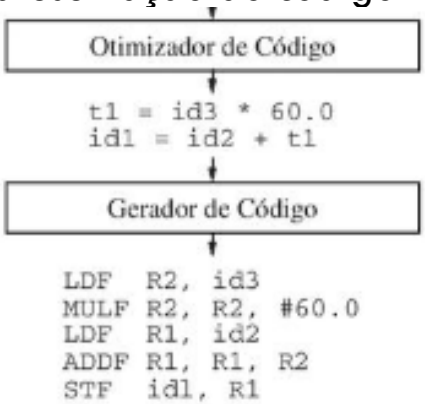
Requer uma seleção cuidadosa das instruções dos registradores da máquina alvo a fim de produzir código objeto eficiente, normalmente para ser implementado em microcontroladores

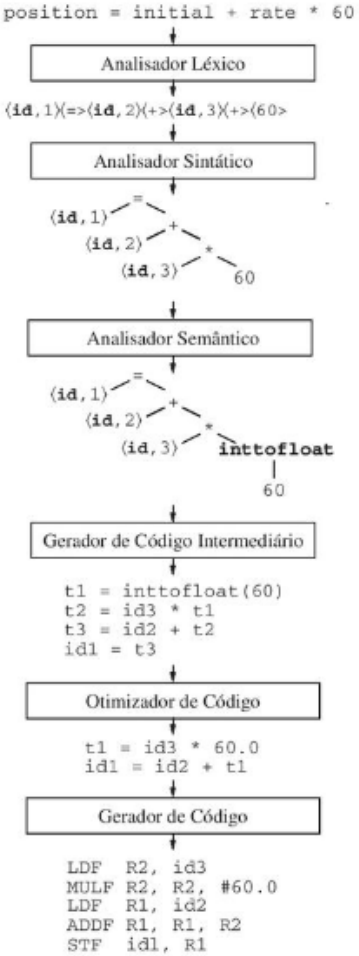
SGAS Quadra 607 - Módulo 49
Via L2 Sul, Brasília - DF
CEP 70.200-670

(61) 3535-6565

GERAÇÃO DE CÓDIGO.

Representação do código.

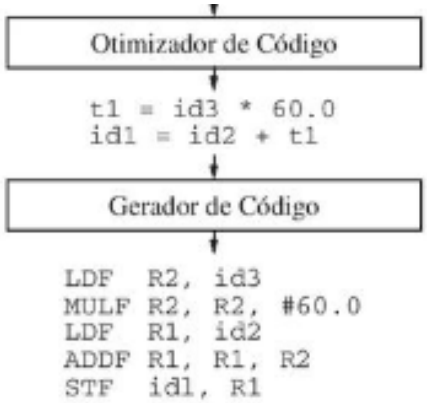




SGAS Quadra 607 - Módulo 49
Via L2 Sul, Brasília - DF
CEP 70.200-670
(61) 3535-6565

GERAÇÃO DE CÓDIGO.

Representação do código.



Tipo de Símbolo	Símbolos / Exemplos	Descrição
Operadores Aritméticos	`*`, `+`, `-`, `/`, `%`	Operações básicas: multiplicação, soma, etc.
Atribuição	`=`	Atribuição de valor
Constantes	`60.0`, `#60.0`, `0`, `1`, `-1`	Valores numéricos, literais
Identificadores	`id1`, `id2`, `id3`, `t1`, `R1`, `R2`	Variáveis e registradores
Separadores	`,`	Separador de argumentos

SGAS Quadra 607 - Módulo 49
Via L2 Sul, Brasília - DF
CEP 70.200-670
(61) 3535-6565

GERAÇÃO DE CÓDIGO.

Instrução	Significado	Categoria
`LDF Rx, id`	Carrega variável de ponto flutuante para Rx	Load (ponto flutuante)
`LD Rx, id`	Carrega inteiro para Rx	Load (inteiro)
`STF id, Rx`	Armazena Rx em variável float	Store (ponto flutuante)
`ST id, Rx`	Armazena Rx em variável inteira	Store (inteiro)
`MULF Rx, Ry, #const`	Multiplica Rx por constante float	Operação de multiplicação
`MUL Rx, Ry`	Multiplica inteiros	Operação de multiplicação
`ADDF Rx, Ry, Rz`	Soma ponto flutuante	Operação de adição (float)
`ADD Rx, Ry`	Soma inteira	Operação de adição (inteiro)
`SUB Rx, Ry`	Subtração inteira	Operação aritmética
`SUBF Rx, Ry`	Subtração ponto flutuante	Operação aritmética (float)

SGAS Quadra 607 - Módulo 49
Via L2 Sul, Brasília - DF
CEP 70.200-670
(61) 3535-6565

GERAÇÃO DE CÓDIGO.

Instrução	Significado	Categoria
`DIV Rx, Ry`	Divisão inteira	Operação aritmética
`DIVF Rx, Ry`	Divisão ponto flutuante	Operação aritmética (float)
`MOV Rx, Ry`	Copia valor de Ry para Rx	Transferência de valor
`CMP Rx, Ry`	Compara valores	Comparação
`JMP label`	Salto incondicional	Controle de fluxo
`JEQ label`	Salta se igual	Controle de fluxo condicional
`JNE label`	Salta se diferente	Controle de fluxo condicional
`JGT label`	Salta se maior	Controle de fluxo condicional
`JLT label`	Salta se menor	Controle de fluxo condicional
`CALL func`	Chamada de função	Sub-rotina
`RET`	Retorno de função	Sub-rotina
`PUSH Rx`	Empilha registrador	Pilha
`POP Rx`	Desempilha para registrador	Pilha

EXEMPLO:

Realize a análise de compilação para a seguinte expressão

Velocidade= distância/tempo +constante

SGAS Quadra 607 - Módulo 49
Via L2 Sul, Brasília - DF
CEP 70.200-670

(61) 3535-6565

EXEMPLO

Análise Léxica

<velocidade,1><= > <distancia, 2> < / > <tempo, 3> <+> <constante, 4>

<velocidade,1> => <id,1>

<distancia, 2> => <id,2>

<tempo, 2> => <id,3>

<constante, 3> > => <id,4>

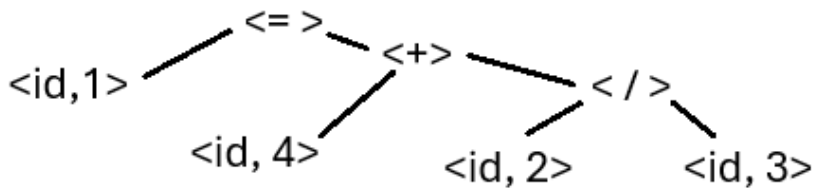
<id,1><= > <id, 2> < / > <id, 3> <+> <id, 4>

SGAS Quadra 607 - Módulo 49
Via L2 Sul, Brasília - DF
CEP 70.200-670

(61) 3535-6565

EXEMPLO

Análise Sintática



Primeiro é feita a divisão e depois realizada a soma

Para este caso a análise semântica não modifica a análise sintática

SGAS Quadra 607 - Módulo 49
Via L2 Sul, Brasília - DF
CEP 70.200-670
(61) 3535-6565

EXEMPLO

Gerador de Código Intermediário

t1 = id2 / id3

t2 = t1 + id4

id1 = t2

SGAS Quadra 607 - Módulo 49
Via L2 Sul, Brasília - DF
CEP 70.200-670

(61) 3535-6565

EXEMPLO

Otimizador de Código

t1 = id2 / id3

id1 = t1 + id4

SGAS Quadra 607 - Módulo 49
Via L2 Sul, Brasília - DF
CEP 70.200-670

(61) 3535-6565

EXEMPLO

Gerador de Código

LDF R1, id2

DIVF R1, R1, id3

ADDF R1, R1, id4

STF id1, R1

SGAS Quadra 607 - Módulo 49
Via L2 Sul, Brasília - DF
CEP 70.200-670

(61) 3535-6565

EXERCÍCIO

Realize a análise completa da expressão que representa o volume de um cilindro:

$$V = \pi r^2 * h$$

Deve mostrar a analise léxica, sintática, semântica, geração de código intermediário, otimizador de código e o gerador de código.

SGAS Quadra 607 - Módulo 49
Via L2 Sul, Brasília - DF
CEP 70.200-670
(61) 3535-6565



AULA 5: PRINCÍPIOS DE CONSTRUÇÃO DE LINGUAGENS.

OBJETIVOS DA AULA

Compreender os princípios fundamentais de construção de linguagens de programação.

Identificar os principais paradigmas de programação e suas características.

Entender como os paradigmas influenciam as construções das linguagens.

Diferenciar linguagens compiladas, interpretadas e híbridas.

Desenvolver senso crítico sobre a escolha de paradigmas e linguagens em diferentes contextos.

INTRODUÇÃO

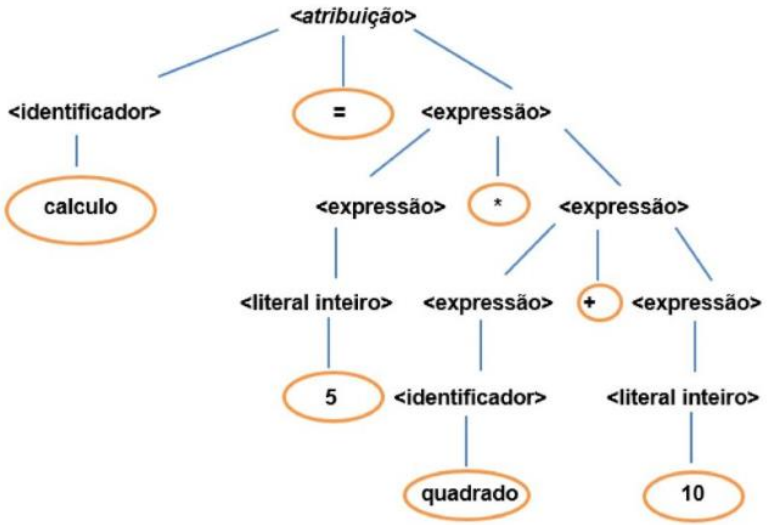


Figura 2: Exemplo de Árvore de Análise Sintática

A importância do estudo da construção das linguagens.

Breve panorama histórico das linguagens de programação.

Por que existem tantos paradigmas?

ELEMENTOS BÁSICOS DAS LINGUAGENS

Definições:

- Operadores, identificadores.
- Tipos de dados (primitivos e compostos).
- Estruturas de controle: condicionais (if, switch) e loops (for, while).
- Funções e escopo.

OPERADORES E IDENTIFICADORES

Usando C:

```
int soma = 10 + 5;  // "+" é um operador, "soma" é um identificador
```

Usando Python:

```
soma = 10 + 5  # "+" é um operador, "soma" é um identificador
```

TIPOS DE DADOS (PRIMITIVOS E COMPOSTOS)

Usando C:

```
// Primitivos
int idade = 25;
float altura = 1.75;
char letra = 'A';

// Composto (array)
int notas[3] = {7, 8, 9};
```

Usando Python:

```
# Primitivos
idade = 25
altura = 1.75
letra = 'A'

# Compostos
notas = [7, 8, 9]  # lista
```

ESTRUTURAS DE CONTROLE

Usando C:

```c
int nota = 7;
if (nota >= 6) {
    printf("Aprovado\n");
} else {
    printf("Reprovado\n");
}

switch(nota) {
    case 10:
        printf("Excelente!\n");
        break;
    case 7:
        printf("Bom!\n");
        break;
    default:
        printf("Regular\n");
}
```

Usando Python:

```python
nota = 7
if nota >= 6:
    print("Aprovado")
else:
    print("Reprovado")

# switch com match-case (Python 3.10+)
match nota:
    case 10:
        print("Excelente!")
    case 7:
        print("Bom!")
    case _:
        print("Regular")
```

LAÇOS FOR E WHILE

```c
int i = 0;
while (i < 5) {
    printf("%d\n", i);
    i++;
}

for (int j = 0; j < 5; j++) {
    printf("%d\n", j);
}
```

```python
i = 0
while i < 5:
    print(i)
    i += 1

for j in range(5):
    print(j)
```

FUNÇÕES ESCOPO

```c
int soma(int a, int b) {
    return a + b;
}

int main() {
    int resultado = soma(3, 4);  // "resultado" tem escopo local
    printf("%d\n", resultado);
    return 0;
}
```

FUNÇÕES ESCOPO

```
def soma(a, b):
    return a + b

def main():
    resultado = soma(3, 4)  # "resultado" tem escopo local à função main
    print(resultado)

main()
```

EXEMPLOS DE CÓDIGOS

C (imperativo)

```c
#include <stdio.h>

int main() {
    int idade = 25;
    float altura = 1.75;

    printf("Idade: %d\n", idade);
    printf("Altura: %.2f\n", altura);

    return 0;
}
```

python (Multiparadigma)

```python
idade = 25
altura = 1.75

print(f"Idade: {idade}")
print(f"Altura: {altura:.2f}")
```

EXEMPLOS DE CÓDIGOS

C (imperativo)

```c
#include <stdio.h>

int main() {
    int i;
    for (i = 0; i < 5; i++) {
        if (i % 2 == 0) {
            printf("%d é par\n", i);
        } else {
            printf("%d é ímpar\n", i);
        }
    }
    return 0;
}
```

python (Multiparadigma)

```python
for i in range(5):
    if i % 2 == 0:
        print(f"{i} é par")
    else:
        print(f"{i} é ímpar")
```

EXEMPLOS DE CÓDIGOS

C (imperativo)

```c
#include <stdio.h>

int soma(int a, int b) {
    return a + b;
}

int main() {
    int resultado = soma(3, 4);
    printf("Resultado: %d\n", resultado);
    return 0;
}
```

python (Multiparadigma)

```python
def soma(a, b):
    return a + b

resultado = soma(3, 4)
print(f"Resultado: {resultado}")
```

EXEMPLOS DE CÓDIGOS

C (imperativo)

```c
#include <stdio.h>

int main() {
    int numeros[] = {10, 20, 30, 40, 50};
    int i;

    for (i = 0; i < 5; i++) {
        printf("%d\n", numeros[i]);
    }

    return 0;
}
```

python (Multiparadigma)

```python
numeros = [10, 20, 30, 40, 50]

for numero in numeros:
    print(numero)
```

PARADIGMAS DE PROGRAMAÇÃO

- Imperativo – foco em instruções sequenciais (ex.: C).
- Orientado a Objetos – abstração via classes e objetos (ex.: Java, Python).
- Funcional – imutabilidade e funções como cidadãos de primeira classe (ex.: Haskell).
- Lógico – definição de relações e resolução por inferência (ex.: Prolog).

EXEMPLOS

C imperativo Puro

```c
#include <stdio.h>

int main() {
    int a = 5, b = 3, soma;

    soma = a + b;

    printf("Soma = %d\n", soma);
    return 0;
}
```

Python Estilo imperativo

```python
a = 5
b = 3
soma = a + b

print("Soma =", soma)
```

EXEMPLOS

Python POO

```python
class Pessoa:
    def __init__(self, nome):
        self.nome = nome

    def cumprimentar(self):
        print(f"Olá, meu nome é {self.nome}")

p = Pessoa("Miguel")
p.cumprimentar()
```

C POO

```c
#include <stdio.h>
#include <string.h>

typedef struct {
    char nome[50];
    void (*cumprimentar)(char*);
} Pessoa;

void dizer_ola(char* nome) {
    printf("Olá, meu nome é %s\n", nome);
}

int main() {
    Pessoa p;
    strcpy(p.nome, "Miguel");
    p.cumprimentar = dizer_ola;

    p.cumprimentar(p.nome);
    return 0;
}
```

C **não tem suporte nativo a orientação a objetos,** mas pode-se simular com structs e ponteiros a funções

EXEMPLO-PARADIGMA FUNCIONAL

Python

```python
def quadrado(x):
    return x * x

valores = [1, 2, 3, 4]
quadrados = list(map(quadrado, valores))

print(quadrados)  # [1, 4, 9, 16]
```

Em C, **programação funcional é limitada,** mas é possível usar funções como ponteiros:

```c
#include <stdio.h>

int quadrado(int x) {
    return x * x;
}

int aplicar(int (*f)(int), int valor) {
    return f(valor);
}

int main() {
    int resultado = aplicar(quadrado, 5);
    printf("Resultado: %d\n", resultado);
    return 0;
}
```

EXEMPLO-PARADIGMA LÓGICO

Baseado em regras, relações e inferência. Não é aplicável diretamente em C/Python.

Exemplo em prolog

```
pai(joao, maria).
pai(joao, jose).
mae(ana, maria).

irmao(X, Y) :- pai(P, X), pai(P, Y), X \= Y.

% Consulta:
% ?- irmao(jose, maria).
```

RESUMO

Paradigma	Linguagem Base	Exemplo
Imperativo	C, Python	Variáveis e controle
Orientado a Objetos	Python, Java	Classes e métodos
Funcional	Python, Haskell	map, lambda, recursão
Lógico	Prolog	Regras e inferência

CONSTRUÇÕES ESPECÍFICAS

- Variáveis, estruturas de dados (listas, árvores), funções.
- Classes, herança, polimorfismo.
- Funções de ordem superior, lambda, recursão.
- Tratamento de exceções.

```python
# Função
def somar(a, b):
    return a + b

# Variável
resultado = somar(3, 4)

# Lista
numeros = [1, 2, 3, 4, 5]

print("Soma:", resultado)
print("Primeiro número:", numeros[0])
```

```c
#include <stdio.h>

// Função
int somar(int a, int b) {
    return a + b;
}

int main() {
    // Variável
    int resultado = somar(3, 4);

    // Lista (array)
    int numeros[] = {1, 2, 3, 4, 5};

    printf("Soma: %d\n", resultado);
    printf("Primeiro número: %d\n", numeros[0]);

    return 0;
}
```

VARIÁVEIS, ESTRUTURAS DE DADOS E FUNÇÕES

CLASSES, HERANÇA E POLIMORFISMO

```python
class Animal:
    def falar(self):
        print("Algum som genérico")

class Cachorro(Animal):
    def falar(self):
        print("Latido")

# Polimorfismo
def fazer_animal_falar(animal):
    animal.falar()

a = Animal()
c = Cachorro()

fazer_animal_falar(a)  # Algum som genérico
fazer_animal_falar(c)  # Latido
```

C **não possui suporte nativo a classes, herança ou polimorfismo**. Pode-se simular com struct e ponteiros de função (emulação de *object-oriented programming*), mas é complexo e não natural para o paradigma.

CLASSES, HERANÇA E POLIMORFISMO

```
class ChamadaDoMetodoEstatico {
  static metodoEstatico() {
    return "O método estático foi chamado";
  }
  static outroMetodoEstatico() {
    return this.metodoEstatico() + " de outro método estático";
  }
}
ChamadaDoMetodoEstatico.metodoEstatico();
// 'O método estático foi chamado'

ChamadaDoMetodoEstatico.outroMetodoEstatico();
// 'O método estático foi chamado de outro método estático'
```

C **não possui suporte nativo a classes, herança ou polimorfismo**. Pode-se simular com struct e ponteiros de função (emulação de *object-oriented programming*), mas é complexo e não natural para o paradigma.

CLASSES, HERANÇA E POLIMORFISMO

C **não possui suporte nativo a classes, herança ou polimorfismo**. Pode-se simular com struct e ponteiros de função (emulação de *object-oriented programming*), mas é complexo e não natural para o paradigma.

```c
#include <stdio.h>
#include <string.h>

typedef struct {
    char nome[50];
    void (*cumprimentar)(char*);
} Pessoa;

void dizer_ola(char* nome) {
    printf("Olá, meu nome é %s\n", nome);
}

int main() {
    Pessoa p;
    strcpy(p.nome, "Miguel");
    p.cumprimentar = dizer_ola;

    p.cumprimentar(p.nome);
    return 0;
}
```

FUNÇÕES DE ORDEM SUPERIOR, LAMBDA, RECURSÃO

```c
#include <stdio.h>

// Ponteiro para função (ordem superior)
int somar(int a, int b) {
    return a + b;
}

int aplicar_operacao(int (*f)(int, int), int x, int y) {
    return f(x, y);
}

// Recursão
int fatorial(int n) {
    if (n == 0) return 1;
    return n * fatorial(n - 1);
}

int main() {
    printf("Soma: %d\n", aplicar_operacao(somar, 3, 4));
    printf("Fatorial: %d\n", fatorial(5));
    return 0;
}
```

```python
# Função de ordem superior
def aplicar_operacao(f, x, y):
    return f(x, y)

# Lambda
soma = lambda a, b: a + b
print(aplicar_operacao(soma, 3, 4

# Recursão
def fatorial(n):
    if n == 0:
        return 1
    return n * fatorial(n - 1)

print(fatorial(5))  # 120
```

TRATAMENTO DE EXCEÇÕES

C **não possui suporte nativo a exceções**.
Em vez disso, usa-se códigos de erro ou setjmp/longjmp para controle avançado de fluxo:

```python
try:
    x = 10 / 0
except ZeroDivisionError:
    print("Erro: Divisão por zero!")
finally:
    print("Finalizado")
```

```c
#include <stdio.h>

int dividir(int a, int b) {
    if (b == 0) {
        printf("Erro: Divisão por zero!\n");
        return -1; // código de erro
    }
    return a / b;
}

int main() {
    int resultado = dividir(10, 0);
    printf("Resultado: %d\n", resultado);
    return 0;
}
```

RESUMO

Construção	C	Python
Variáveis e arrays	Suporte direto	Suporte direto
Funções	Suporte direto	Suporte direto
Classes e herança	Não suporta nativamente	Suporte total (POO)
Polimorfismo	Simulação limitada	Suporte total
Ordem superior/lambda	Ponteiros de função	Funções como objetos
Recursão	Simples e direto	Simples e direto
Exceções	Não possui suporte nativo	try-except-finally

IMPLEMENTAÇÃO DAS LINGUAGENS

Compiladas: traduzidas para código de máquina antes da execução (ex.: C).

Interpretadas: analisadas e executadas em tempo real (ex.: Python).

Híbridas: compiladas para bytecode e interpretadas por uma VM (ex.: Java).

EXEMPLOS- COMPILADAS

O código é traduzido antes da execução para código de máquina por um compilador.

Vantagens: Alta performance em tempo de execução.

Exemplo em C:

```c
#include <stdio.h>

int main() {
    printf("Olá, mundo!\n");
    return 0;
}
```

Esse código é compilado com um compilador como `gcc`:

```
gcc programa.c -o programa
./programa
```

EXEMPLOS-INTERPRETADAS (EX.: PYTHON)

- O código é **executado linha por linha** por um interpretador.
- Vantagens: Mais fácil de testar e modificar.
- Exemplo em **Python**:

```
print("Olá, mundo!")
```

Esse código é executado diretamente:

```
python programa.py
```

EXEMPLO-HIBRIDAS

O código é **compilado para bytecode,** que é interpretado ou just-in-time compilado por uma máquina virtual (JVM).

Exemplo em **Java:**

```
public class OlaMundo {
    public static void main(String[] args) {
        System.out.println("Olá, mundo!");
    }
}
```

Compila-se com:

```
javac OlaMundo.java
java OlaMundo
```

ESCOLHA DE PARADIGMAS E IMPACTOS

Impacto na legibilidade e manutenibilidade do código.

Eficiência em tempo de execução vs facilidade de desenvolvimento.

Como o paradigma molda o raciocínio do programador.

ESCOLHA DE PARADIGMAS E IMPACTOS

A escolha do **paradigma de programação** afeta significativamente o desenvolvimento de software. Veja os principais impactos:

Legibilidade e Manutenibilidade

Paradigmas como **Orientação a Objetos** promovem organização e reutilização de código.

Paradigmas **funcionais**, como em Haskell ou com lambdas em Python, ajudam na clareza em tarefas matemáticas ou transformações de dados.

Paradigmas **imperativos** são diretos, mas o código pode ficar mais difícil de manter com o tempo.

ESCOLHA DE PARADIGMAS E IMPACTOS

Eficiência x Facilidade

Linguagens **compiladas** geralmente são mais rápidas, mas menos flexíveis para mudanças rápidas (ex.: C).

Linguagens **interpretadas** (ex.: Python) favorecem produtividade e prototipagem, mas podem ser menos eficientes.

Paradigmas **funcionais** evitam efeitos colaterais, o que melhora a testabilidade, mas pode exigir mais raciocínio para problemas imperativos.

ESCOLHA DE PARADIGMAS E IMPACTOS

Raciocínio do Programador

Um programador que trabalha com **orientação a objetos** pensa em termos de entidades e interações.

Em **funcional,** o foco está em transformação de dados e funções puras.

No **imperativo,** o pensamento é voltado a **passo-a-passo** e controle direto do fluxo de execução.

RESUMO DA AULA

Paradigma	Características-Chave	Impacto na Leitura/Manutenção	Eficiência	Raciocínio do Programador	Exemplo
Imperativo	Instruções sequenciais, controle explícito do estado	Pode ficar confuso em códigos grandes	Alta (compilado)	Passo a passo, foco em variáveis	C, Python
Orientado a Objetos	Abstração via classes, herança, encapsulamento	Alta reutilização e modularidade	Média	Pensa em termos de objetos e interações	Python, Java
Funcional	Funções puras, imutabilidade, recursão, composição de funções	Clareza em transformações, difícil de debugar efeito colateral	Varia	Pensamento declarativo e matemático	Python, Haskell
Lógico	Definição de relações, resolução por inferência	Boa legibilidade para regras lógicas	Menor eficiência	Pensamento declarativo baseado em lógica	Prolog

ATIVIDADE

Criar um programa que:

- Leia uma lista de 5 nomes e idades de pessoas.
- Mostre qual é a pessoa mais velha.
- Calcule a média das idades.

Resolver o problema usando **variáveis, estruturas de controle e laços**.

Exemplo em C e Python, com foco no controle manual dos dados.

Critérios de observação e identificação:

- Controle do fluxo
- Clareza de variáveis
- Manipulação manual de índices

ATIVIDADE

Parte 2 – Programação Orientada a Objetos (Python)

- Criar uma **classe Pessoa**, com atributos nome e idade.
- Usar uma lista de objetos para resolver o problema.

Critérios de observação:

- Uso de classes e métodos
- Encapsulamento dos dados
- Clareza da estrutura

ATIVIDADE

Programação Funcional (Python)

Usar **map, lambda, filter, reduce** (ou list comprehensions) para resolver o problema.

Manter o código mais declarativo possível.

📌 **Critérios de observação:**

Uso de funções puras

Imutabilidade (evitar variáveis globais)

Clareza na transformação de dados

ATIVIDADE

Discussão e Análise Comparativa

Em grupo ou individualmente, responda:

Qual paradigma foi mais fácil de entender?

Qual você considerou mais simples de implementar?

Qual solução seria mais fácil de manter no futuro?

Onde você percebeu maior clareza ou menor repetição de código?

Com qual paradigma você se sentiu mais confortável? Por quê?

AULA 6: PROGRAMAÇÃO IMPERATIVA

OBJETIVOS:

Ao final da aula, o aluno deve ser capaz de:

- Compreender o que é a programação imperativa.
- Identificar as principais características desse paradigma.
- Escrever pequenos programas usando comandos sequenciais, condicionais e de repetição.
- Compreender conceitos como variáveis, operadores e controle de fluxo.

INTRODUÇÃO

- Programação imperativa é um paradigma de programação onde se especifica como o computador deve realizar uma tarefa, detalhando cada passo através de instruções e comandos. Em outras palavras, o programador dita exatamente o que o computador deve fazer, passo a passo, para atingir um resultado desejado.

O QUE É PROGRAMAÇÃO IMPERATIVA?

- A programação imperativa é um paradigma de programação que enfatiza a descrição explícita das etapas que um computador deve realizar para alcançar o resultado desejado. Nesta abordagem, o programador escreve uma série de instruções ou comandos que manipulam diretamente o estado do programa através de atribuições, estruturas de controle como laços e condicionaise chamadas de função. Esses comandos são executados sequencialmente e cada etapa altera o estado do programa de maneira específica e previsível.
- O foco principal da programação imperativa está em como as tarefas são executadas, e não no que as tarefas realizam. Este paradigma é contrastado com a programação declarativa, que se concentra em descrever o que o programa deve alcançar sem especificar os passos exatos para chegar lá. Imperativo linguagens de programação, como C, Java e Python, fornecem construções que permitem aos desenvolvedores gerenciar operações complexas por meio de instruções claras e passo a passo, facilitando a compreensão e o controle do fluxo do programa.

RECURSOS DE PROGRAMAÇÃO IMPERATIVOS

Aqui estão os principais recursos da programação imperativa, juntamente com suas explicações:

- **Execução sequencial.** As instruções são executadas uma após a outra em uma ordem específica. O fluxo de controle segue a sequência em que as instruções são escritas, a menos que sejam modificadas por estruturas de controle.

- **Estado e variáveis.** O estado do programa é representado por variáveis, que armazenam dados. Estas variáveis podem ser atualizadas ou alteradas à medida que o programa é executado, refletindo o estado atual do programa.

- **Estruturas de controle**. A programação imperativa usa estruturas de controle como loops (for, while) e condicionais (if, else) para gerenciar o fluxo de execução. Estas estruturas permitem que o programa tome decisões e repita ações.

RECURSOS DE PROGRAMAÇÃO IMPERATIVOS

- **Atribuições.** Atribuições são instruções que alteram o valor de uma variável. Eles são fundamentais para a programação imperativa, permitindo que o estado do programa seja modificado explicitamente.

- **Funções e procedimentos.** Funções (ou procedimentos) são blocos de código reutilizáveis que executam tarefas específicas. Eles podem ser chamados diversas vezes dentro de um programa, promovendo a reutilização e a modularidade do código.

- **Estado mutável.** O estado do programa pode mudar ao longo do tempo através de atribuições e operações em variáveis. Essa mutabilidade é um aspecto central da programação imperativa, permitindo um comportamento dinâmico.

RECURSOS DE PROGRAMAÇÃO IMPERATIVOS

- **Fluxo de controle explícito.** O programador tem controle direto sobre a ordem de execução e o fluxo do programa. Esse controle explícito permite um gerenciamento refinado de como as tarefas são executadas.

- **Iteração.** Iteração é a execução repetida de um conjunto de instruções. Normalmente é implementado por meio de loops, permitindo que o programa execute tarefas repetitivas com eficiência.

- **Os efeitos colaterais.** As ações que afetam o estado fora do ambiente local de uma função ou procedimento são conhecidas como efeitos colaterais. A programação imperativa geralmente envolve efeitos colaterais, como modificar uma variável global ou interagir com sistemas externos.

LINGUAGENS DE PROGRAMAÇÃO IMPERATIVAS

Aqui estão algumas linguagens de programação imperativas notáveis, juntamente com breves explicações sobre suas características e usos.

C

- Desenvolvido no início da década de 1970, C é uma linguagem de programação de uso geral conhecida por sua eficiência e controle sobre os recursos do sistema. Ele fornece acesso de baixo nível à memória e Hardwares, tornando-o ideal para programação de sistemas, como desenvolvimento sistemas operacionais e sistemas embarcados.

C + +

- Uma extensão de C, C + + adiciona recursos orientados a objetos à linguagem. Ele suporta manipulação de memória de baixo nível e abstrações de alto nível, tornando-o versátil para desenvolvimento de sistema/software, programação de jogos e simulações em tempo real.

LINGUAGENS DE PROGRAMAÇÃO IMPERATIVAS

Java

- Introduzido pela Sun Microsystems em 1995, Java é uma linguagem de programação orientada a objetos de alto nível, projetada para ser portátil e independente de plataforma por meio do uso da Java Virtual Machine (JVM). É amplamente utilizado em aplicativos corporativos, desenvolvimento Android e aplicativos web.

Python

- Python é uma linguagem interpretada de alto nível, conhecida por sua simplicidade e legibilidade. Ele oferece suporte a vários paradigmas de programação, incluindo programação imperativa, processual e orientada a objetos. Python é popular em desenvolvimento web, ciência de dados, automação e scripts.

JavaScript

- Inicialmente desenvolvido para navegadores da web, JavaScript é uma linguagem dinâmica e de alto nível que agora é usada tanto para o lado do cliente quanto para serverdesenvolvimento lateral. Ele permite páginas da web interativas e é essencial para o desenvolvimento da web moderno por meio de estruturas como Node.js, React e Angular.

LINGUAGENS DE PROGRAMAÇÃO IMPERATIVAS

Ruby

- Criada em meados da década de 1990, Ruby é uma linguagem interpretada de alto nível projetada para simplicidade e produtividade. Segue o princípio da “menor surpresa”, facilitando a leitura e a escrita. Ruby é mais conhecido por seu uso em desenvolvimento web com a estrutura Ruby on Rails.

Perl

- Perl é uma linguagem interpretada de alto nível conhecida por suas capacidades de processamento de texto. Muitas vezes é usado para administração do Sistema, desenvolvimento web e programação de rede. As poderosas expressões regulares do Perl e flexA flexibilidade o torna uma escolha popular para scripts e automação.

Vá (Golang)

- Desenvolvido pelo Google, Go é uma linguagem compilada e digitada estaticamente, projetada para simplicidade e eficiência. É particularmente adequado para programação simultânea e é usado em cloud serviços, serveraplicativos secundários e ferramentas de rede.

LINGUAGENS DE PROGRAMAÇÃO IMPERATIVAS

rápido

- Introduzido pela Apple em 2014, Swift é uma linguagem compilada de alto nível projetada para desenvolvimento em iOS e macOS. Combina recursos modernos com desempenho e segurança, tornando-o a escolha preferida para o desenvolvimento de aplicativos dentro do ecossistema Apple.

Ferrugem

- Rust é uma linguagem de programação de sistemas focada em segurança, simultaneidade e desempenho. Seu objetivo é fornecer segurança de memória sem usar coleta de lixo, tornando-a adequada para programação em nível de sistema, desenvolvimento de jogos e aplicações exigindo alto desempenho.

SUBDOMÍNIOS DE PROGRAMAÇÃO IMPERATIVA

A programação imperativa abrange vários subdomínios, cada um enfocando aspectos específicos de programação e resolução de problemas. Aqui estão alguns subdomínios principais da programação imperativa, junto com suas explicações.

Programação Processual

- A programação processual é um subconjunto da programação imperativa que enfatiza o uso de procedimentos ou funções. Ele organiza o código em blocos reutilizáveis chamados procedimentos ou funções, que executam tarefas específicas. Essa abordagem promove a modularidade do código, tornando os programas mais fáceis de entender, manter e depurar. Linguagens como C, Pascal e Fortran são comumente usadas em programação processual.

Programação Orientada a Objetos (OOP)

- OOP é um paradigma que se baseia em princípios de programação imperativos, com foco no conceito de objetos, que são instâncias de classes. Ele combina dados e funções que operam nos dados em objetos, promovendo encapsulamento, herança e polimorfismo. OOP melhora a reutilização do código e escalabilidade, tornando-o adequado para sistemas de software grandes e complexos. Linguagens OOP comuns incluem Java, C++ e Python.

SUBDOMÍNIOS DE PROGRAMAÇÃO IMPERATIVA

Programação de Sistemas

- A programação de sistemas envolve o desenvolvimento de software que fornece serviços ao hardware do computador ou operações de baixo nível, como sistemas operacionais, drivers de dispositivos e sistemas embarcados. Requer um conhecimento profundo de hardware e gerenciamento eficiente de recursos. C e C++ são amplamente utilizados na programação de sistemas devido ao seu desempenho e capacidades de baixo nível.

Scripting

- A criação de scripts envolve escrever pequenos programas, ou scripts, que automatizam tarefas e processos. Scripts são frequentemente interpretados em vez de compilados, tornando-os fáceis de escrever e modificar. O script é comumente usado para administração de sistema, desenvolvimento web e automação. Linguagens de script populares incluem Python, Perl, Ruby e JavaScript.

SUBDOMÍNIOS DE PROGRAMAÇÃO IMPERATIVA

Programação simultânea

- A programação simultânea concentra-se na execução simultânea de múltiplas sequências de operações, melhorando a eficiência e a capacidade de resposta das aplicações. Esta abordagem é crucial para o desenvolvimento de aplicações que requerem multitarefa ou processamento paralelo, como web servers, sistemas em tempo real e sistemas distribuídos. Linguagens como Go, Java e Python fornecem construções para programação concorrente, como threads e goroutines.

Programação Embarcada

- A programação embarcada envolve o desenvolvimento de software para sistemas embarcados, que são sistemas de computação especializados dentro de sistemas mecânicos ou elétricos maiores. Esses sistemas geralmente apresentam restrições em tempo real e recursos limitados. A programação incorporada requer programação de baixo nível e interação de hardware, normalmente usando linguagens como C e C++.

SUBDOMÍNIOS DE PROGRAMAÇÃO IMPERATIVA

Programação de jogos

- A programação de jogos é o processo de desenvolvimento de videogames, exigindo uma combinação de gráficos, física e interações em tempo real. Muitas vezes envolve programação de baixo nível para otimização de desempenho e scripts de alto nível para lógica de jogo. C++, C# e mecanismos de desenvolvimento de jogos especializados como Unity e Unreal Engine são comumente usados neste domínio.

Programação de Rede

- A programação de rede envolve a criação de software que permite a comunicação entre dispositivos em uma rede. Inclui o desenvolvimento de protocolos, serviços de rede e cliente-server formulários. Este domínio requer conhecimento de protocolos de rede e programação de soquetes. Linguagens como C, Java e Python são frequentemente usadas para programação de rede.

SUBDOMÍNIOS DE PROGRAMAÇÃO IMPERATIVA

Programação em tempo real

- A programação em tempo real concentra-se no desenvolvimento de sistemas que respondam a entradas ou eventos dentro de restrições de tempo estritas. Esses sistemas são usados em ambientes onde o processamento oportuno é crítico, como aeroespacial, automotivo e automação industrial. A programação em tempo real geralmente envolve programação de baixo nível e controle de tempo preciso, normalmente usando linguagens como C e Ada.

VANTAGENS E DESVANTAGENS DA PROGRAMAÇÃO IMPERATIVA

Diferenciais
Aqui estão algumas vantagens principais da programação imperativa, juntamente com explicações para cada uma:

Simplicidade e intuitividade. A programação imperativa reflete de perto a maneira como os humanos dão instruções na vida cotidiana, tornando-a direta e fácil de entender. A abordagem passo a passo permite que os desenvolvedores vejam exatamente como um programa funciona e como os dados são manipulados.

Controle refinado. Este paradigma oferece controle preciso sobre o fluxo e estado de execução do programa, permitindo aos desenvolvedores otimizar o desempenho e gerenciar recursos de forma eficiente. Este controle é particularmente benéfico na programação em nível de sistema, onde o gerenciamento de recursos é crítico.

VANTAGENS E DESVANTAGENS DA PROGRAMAÇÃO IMPERATIVA

- **Amplo suporte a idiomas**. Muitas linguagens de programação populares, como C, C++, Java e Python, oferecem suporte à programação imperativa. Esta adoção generalizada significa que uma grande quantidade de recursos, bibliotecas e apoio comunitário estão disponíveis, tornando o desenvolvimento mais acessível.

- **Facilidade de depuração e teste**. Como a execução do programa é explícita e segue uma sequência clara, rastrear erros e testar partes específicas do código é relativamente fácil. O fluxo previsível de controle ajuda a isolar e corrigir bugs.

- **Versatilidade**. A programação imperativa pode ser usada para uma ampla variedade de aplicações, desde scripts simples até sistemas complexos e aplicações em tempo real. Isso é flexA flexibilidade o torna adequado para diversos domínios, como desenvolvimento web, desenvolvimento de jogos e sistemas embarcados.

VANTAGENS E DESVANTAGENS DA PROGRAMAÇÃO IMPERATIVA

- **Abstrações processuais**. O uso de funções ou procedimentos promove a reutilização e modularidade do código. Os desenvolvedores podem dividir problemas complexos em partes menores e gerenciáveis, tornando a base de código mais organizada e fácil de manter.

- **Eficiência de desempenho**. Linguagens imperativas geralmente permitem acesso e manipulação de memória de baixo nível, o que pode levar a um código altamente otimizado e eficiente. Isso é crucial para aplicativos de desempenho crítico, como desenvolvimento de jogos e software de sistema.

- **Legibilidade e facilidade de manutenção**. A sequência explícita de comandos e a estrutura clara dos programas imperativos melhoram a legibilidade, facilitando a compreensão e a manutenção do código por outros desenvolvedores.

VANTAGENS E DESVANTAGENS DA PROGRAMAÇÃO IMPERATIVA

Desvantagens

- Aqui estão as desvantagens da programação imperativa, juntamente com explicações para cada uma:

Complexidade em grandes programas. À medida que os programas crescem em tamanho, a complexidade do gerenciamento do estado e do fluxo de controle aumenta. Isso pode dificultar a compreensão, a manutenção e a depuração de grandes programas imperativos, levando a possíveis erros e à diminuição da produtividade.

Acoplamento apertado e baixa modularidade. Os programas imperativos têm frequentemente componentes fortemente acoplados, onde as alterações numa parte do programa podem impactar significativamente outras. Essa baixa modularidade dificulta a reutilização de código e torna difícil isolar e corrigir bugs ou implementar novos recursos.

VANTAGENS E DESVANTAGENS DA PROGRAMAÇÃO IMPERATIVA

- **Gestão do estado**. A programação imperativa requer gerenciamento explícito do estado do programa, o que pode levar a erros como efeitos colaterais não intencionais ou estados inconsistentes. A complexidade do gerenciamento de estado torna os programas mais propensos a erros e mais difíceis de depurar.

- **Falta de abstração**. A programação imperativa tende a se concentrar em etapas e operações específicas, muitas vezes levando a níveis mais baixos de abstração. Isto pode resultar em código detalhado que é mais difícil de ler e compreender em comparação com paradigmas que enfatizam abstrações de nível superior.

- **Problemas de simultaneidade**. Lidar com a simultaneidade na programação imperativa é um desafio devido à necessidade de gerenciamento explícito de estado compartilhado e sincronização. Isso pode levar a códigos complexos e propensos a erros ao lidar com aplicativos multithread ou paralelos.

VANTAGENS E DESVANTAGENS DA PROGRAMAÇÃO IMPERATIVA

- **Dificuldade em testar**. O estado mutável e os efeitos colaterais comuns em programas imperativos podem dificultar os testes unitários. Garantir que os componentes individuais se comportem corretamente isoladamente pode ser um desafio quando o seu comportamento depende de mudanças de estado externas.

- **Desafios de escalabilidade**. A programação imperativa pode não ser escalonável tão bem quanto outros paradigmas para certos tipos de aplicações, particularmente aquelas que exigem altos níveis de abstração, modularidade ou simultaneidade. Isto limita a sua eficácia em sistemas de grande escala ou altamente dinâmicos.

- **Legibilidade e facilidade de manutenção**. A natureza detalhada e passo a passo do código imperativo reduz a legibilidade, tornando mais difícil para novos desenvolvedores entenderem a lógica do programa. Isso impacta negativamente a capacidade de manutenção a longo prazo e a facilidade de integração de novos membros da equipe.

EXEMPLO

```c
#include <stdio.h>

int main() {
    int a = 10, b = 5;
    int soma = a + b;
    printf("Resultado: %d\n", soma);
    return 0;
}
```

```c
if (soma > 10) {
    printf("Valor alto!\n");
} else {
    printf("Valor baixo.\n");
}
```

```c
for (int i = 0; i < 5; i++) {
    printf("Valor de i: %d\n", i);
}
```

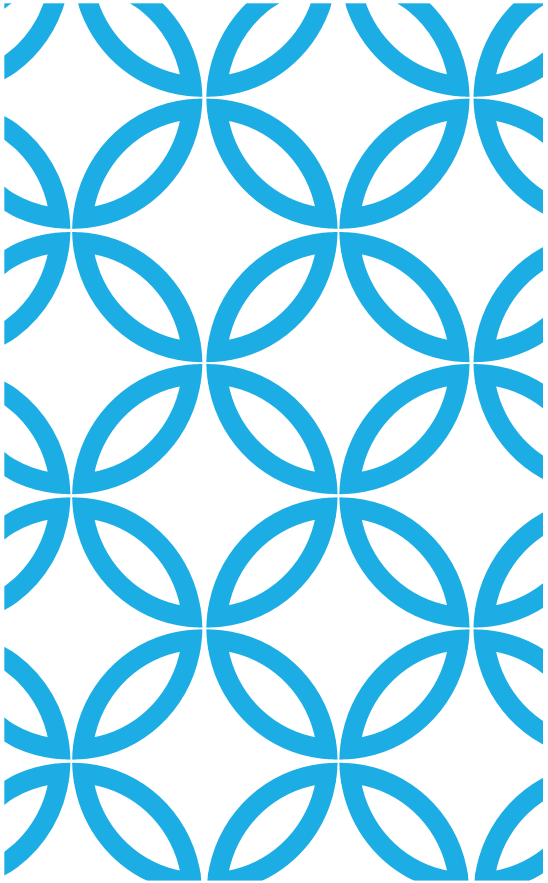
- Explicação do código

ATIVIDADE EM DUPLAS

Propondo o seguinte problema:

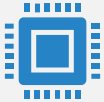
- "Crie um programa que leia dois números e mostre o maior entre eles. Depois, mostre a soma dos números de 1 até o valor do maior número.

"Critérios:
- Uso de if-else.
- Uso de for ou while.



PARADIGMA DE PROGRAMA EM LÓGICO

OBJETIVOS



Diferenciar programação procedural e declarativa.



Revisar conceitos básicos e a representação de sentenças lógicas usando lógica proposicional e cálculo de predicados.



Representar proposições atômicas em Prolog.

IMPERATIVO

A programação imperativa é um paradigma onde se dá uma sequência de comandos (instruções) ao computador para que ele execute uma tarefa, focando em como fazer algo.

Em vez de simplesmente descrever o resultado desejado (como na programação declarativa), o programador especifica explicitamente cada passo e o estado do programa que deve ser alterado, usando estruturas como variáveis, condicionais (if/else) e loops (for/while) para controlar o fluxo de execução.

DECLARATIVO

- A programação declarativa é um estilo de programação focado em descrever o resultado desejado sem especificar as etapas de controle.
- Em vez de dizer ao computador "como fazer" uma tarefa (abordagem imperativa), o programador declara "o que" precisa ser feito.
- Exemplos comuns incluem o SQL para consultas de banco de dados, linguagens de programação funcional como Haskell e frameworks declarativos como Angular para interfaces web.

FUNCIONAL VS LÓGICO

- Os dois principais paradigmas declarativos são o funcional e o lógico.

Funcional

- Baseado em declaração e aplicação de funções (cálculo lambda)
- Todos os parâmetro de uma função precisam estar instanciados
- Clara distinção entre entrada e saída

Lógico

- Baseado no cálculo de predicados
- Objetos e relações
- A computação é feita usando um mecanismo de inferência lógico
- A computação pode ser realizada com variáveis não instanciadas

PROCEDURAL VS DECLARATIVA

Abordagens de programação:

- – Procedural.
- – Declarativa.

Na abordagem declarativa deve-se especificar os

- resultados desejados (o que deve ser feito) e não os procedimentos para produzi-los (como deve ser feito).

DEFINIÇÃO

Paradigma Lógico

- Os programas lógicos são expressos na forma de lógica simbólica e os resultados são produzidos por um processo de inferência lógica.

- A lógica simbólica pode ser usada para atender às três necessidades da lógica formal:
  - – Expressar proposições.
  - – Expressar os relacionamentos entre as proposições.
  - – Descrever como novas proposições podem ser inferidas a partir de outras que se assume como verdadeiras.

DEFINIÇÃO

A lógica simbólica divide-se em:

- – Lógica proposicional.
- – Lógica ou cálculo de predicados de primeira ordem.

O cálculo de predicados de primeira ordem é a base para a programação lógica.

LÓGICA PROPOSICIONAL.

Proposições: declaração afirmativa que é sabidamente verdadeira ou falsa

- **Constantes:** símbolos utilizados para representar as proposições.
- **Conectivos lógicos**: operadores utilizados para combinar diferentes proposições e possibilitam calcular o resultado do sistema formal.

LÓGICA PROPOSICIONAL.

- **Operadores lógicos**

Negação	**¬a**	não a
Conjunção	**a ^ b**	a e b
Disjunção	**a v b**	a ou b
Equivalência	**a ↔ b**	a é equivalente b
Implicação	**a → b**	a implica em b

LÓGICA PROPOSICIONAL.

- **Tabela verdade dos operadores lóaicos**

P	Q	¬P	P∧Q	P∨Q	P⇒Q	P⇔Q
F	F	V	F	F	V	V
F	V	V	F	V	V *	F
V	F	F	F	V	F	F
V	V	F	V	V	V	V

- **P->Q. é falsa só quando P é verdadeira e Q falsa**

EXERCÍCIO

- Considere as seguintes proposições (POSCOMP, 2006):

(I) $\neg p \vee q$

(II) $\neg (p \wedge \neg q)$

(III) $p \rightarrow q$

(IV) $(V \rightarrow q) \vee (p \rightarrow F)$

Quais das proposições acima são logicamente equivalentes?

(a) Somente (I)≡(III)

(b) Somente (I)≡(II)

(c) Somente (I)≡(II)≡(III)

(d) (I)≡(III) e (II)≡III) mas (III)≠(IV)

(e) (I), (II), (III) e (IV) são todas equivalentes.

LÓGICA DE PRIMEIRA ORDEM

Lógica ou cálculo de predicados de primeira ordem:

- **Predicados**: termos que expressam a relação entre entidades, sejam elas constantes ou variáveis.
- **Variáveis**: permitem a construção de regras que enunciam princípios gerais (uma variável pode ser substituída por diferentes constantes).
- **Quantificadores**: símbolos especiais associados as variáveis que indicam a quantidade de constantes que podem ser associadas a uma variável específica.

LÓGICA DE PRIMEIRA ORDEM

Variáveis podem aparecer nas proposições, apenas precedidas por quantificadores:

- **Universal** - ∀
  - ∀ X.P – Para todo X, P é verdadeiro.
  - ∀ X.(mulher(X) → humano(X))
- **Existencial** - ∃
  - ∃ X.P – Existe uma valor de X tal que P é verdadeiro.
  - ∃ X.(mae(mary,X) ^ homem(X))

EXERCÍCIO 2

Seja o universo U = {10, 20, 30, 40} e o conjunto dos números naturais N. Com base no conhecimento sobre a lógica de predicados, avalie as afirmações a seguir.

I. $H = (\forall x \in N)(\exists y \in U)(x < y)$ é válida.

II. $H = (\forall x \in N)(\exists y \in N)(y < x)$ é válida.

III. $H = (\forall x \in U)(\exists y \in U)(x > y)$ é inválida, sendo $x = 10$ um contra-exemplo.

É correto o que se afirma em

A) I, apenas.

B) III, apenas.

C) I e II, apenas.

D) II e III, apenas.

E) I, II e III.

PLATAFORMA DE PROGRAMAÇÃO PROLOG

- O cálculo de predicados de primeira ordem constitui a base da programação lógica.
- O Prolog é a principal linguagem que implementa o paradigma lógico.
- O desenvolvimento do Prolog iniciou na década de 70 nas universidades de Edimburgo e Aix-Marseille.
- Na década de 80 o interesse pela linguagem cresceu com o anúncio dos governo japonês dos computadores de quinta geração.
- Com o abandono silencioso deste projeto o interesse por Prolog também diminuiu.

PLATAFORMA DE PROGRAMAÇÃO PROLOG

Um termo composto é formado por:

– Um termo simples constante, denominado functor ou termo de predicado, que representa uma relação matemática.

– Outros termos simples, constantes ou variáveis, que representam a lista de parâmetros do termo de predicado.

- homem(joao)
- gosta(joao,maria)
- gosta(joao, X)

Não há uma semântica intrínseca nos termos.

APLICAÇÕES

Principais aplicações se dão na área de computação simbólica:

- Lógica matemática, prova automática de teoremas e semântica;
- Solução de equações simbólicas;
- Bancos de dados relacionais;
- Linguagem Natural;
- Sistemas Especialistas;
- Planejamento Automático de Atividades;
- Aplicações de “General Problem Solving”, como jogos (Xadrez, Damas, Jogo da Velha, etc.);
- Compiladores;
- Análise Bioquímica e projetos de novas drogas.

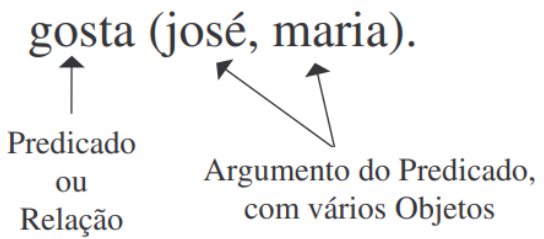
DEFINIÇÃO

Definição: Prolog é uma linguagem de programação que é utilizada para resolver problemas que envolvam objetos e relações entre objetos. Um programa em Prolog consiste basicamente de:

- **declaração de fatos (facts)** sobre objetos e suas relações;
- **definições de regras (rules)**sobre os objetos e suas relações;
- **questões (goals)** que são feitas sobre os objetos e suas relações.

DEFINIÇÃO

- Os fatos são os elementos fundamentais da programação em Prolog, pois determinam as relações que existem entre os objetos conhecidos.



CARACTERÍSTICAS DOS FATOS:

Os nomes dos predicados e dos objetos devem começar com letra minúscula. Por exemplo: joão, casa, gosta.

- Os predicados são escritos primeiro e os objetos são escritos depois, separados por vírgulas.
- Os objetos são escritos dentro de parênteses.
- Todo fato é terminado com um ponto final.
- A ordem dos objetos é importante: gosta (maria, josé). ≠ gosta (josé, maria).
- Uma coleção de fatos é chamada de "**banco de conhecimento**" ou "banco de dados".
- Os fatos podem ter um número arbitrário de objetos como argumento.

NOTA: Para padronizar a criação de predicados (tanto os fatos quanto as regras), devemos sempre pensar que o predicado é uma propriedade do primeiro objeto do argumento. Assim, "gosta (joão, maria)." deve ser lido como "João gosta de Maria".

PROPOSIÇÕES SIMPLES

Proposições simples ou atômicas podem ser usadas para definir Fatos e Consultas:

- Fatos, quando a proposição é assumida como verdadeira.

Exemplos de sentenças para definir fatos.

- homem(joao) – assume que joao é um homem.
- pai (joao, maria) – assume que joao é pai de maria.

PROPOSIÇÕES SIMPLES

Proposições simples ou atômicas podem ser usadas para definir Fatos e Consultas:

- Consultas, quando a verdade sobre a proposição deve ser verificada.

Exemplos de sentenças de objetivos ou consultas:

- – homem(joao) – o sistema responde true ou false (yes ou no).
- – pai (joao, maria) - o sistema responde true ou false (yes ou no).
- – pai(X, maria) – por unificação o sistema tentará encontrar uma instanciação para X que represente o pai de maria.

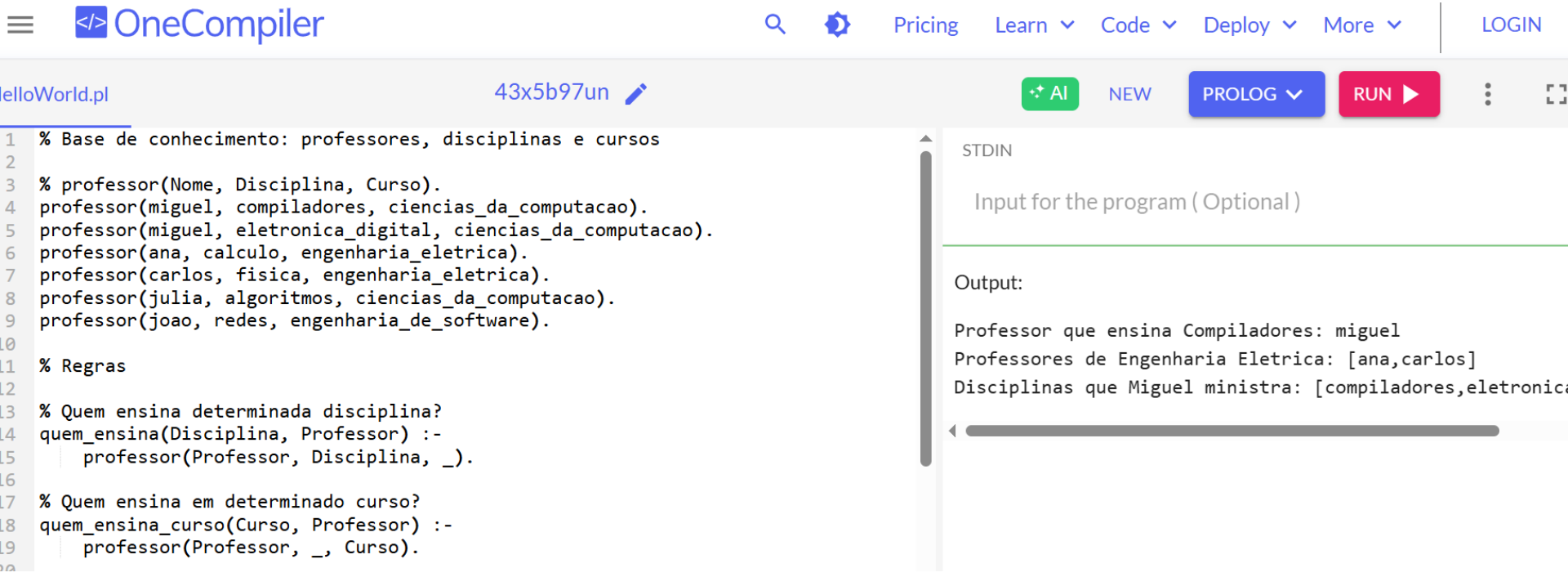
PROPOSIÇÕES SIMPLES

Após a definição dos fatos e das regras, a interação com um programa feito em Prolog pode ser realizada de duas maneiras:

- – Digitando as consultas no prompt do interpretador do Prolog.
- – Criando um programa que realiza consultas pré- determinadas.

EXEMPLO DE PROGRAMAÇÃO

- Uma pequena base de conhecimento sobre **professores, disciplinas e cursos.** A ideia é consultar quais professores lecionam determinada disciplina ou curso.



EXEMPLO DE PROGRAMAÇÃO

- Uma pequena base de conhecimento sobre **professores, disciplinas e cursos.** A ideia é consultar quais professores lecionam determinada disciplina ou curso.

```
% Quais disciplinas um professor ensina?
disciplinas_professor(Professor, Disciplina) :-
    professor(Professor, Disciplina, _).

% Ponto de entrada
:- initialization(main).

main :-
    % Consulta 1: quem ensina compiladores
    quem_ensina(compiladores, P1),
    format('Professor que ensina Compiladores: ~w~n', [P1]),

    % Consulta 2: quem ensina no curso de Engenharia Eletrica
    findall(P2, quem_ensina_curso(engenharia_eletrica, P2), ListaProfs),
    format('Professores de Engenharia Eletrica: ~w~n', [ListaProfs]),

    % Consulta 3: disciplinas que Miguel ministra
    findall(D, disciplinas_professor(miguel, D), ListaDisc),
    format('Disciplinas que Miguel ministra: ~w~n', [ListaDisc]),

    halt.
```

Input for the program (Optional)

Output:

```
Professor que ensina Compiladores: miguel
Professores de Engenharia Eletrica: [ana,carlos]
Disciplinas que Miguel ministra: [compiladores,eletronica
```

CÓDIGO

```
% Base de conhecimento: professores, disciplinas e cursos

% professor(Nome, Disciplina, Curso).
professor(miguel, compiladores, ciencias_da_computacao).
professor(miguel, eletronica_digital, ciencias_da_computacao).
professor(ana, calculo, engenharia_eletrica).
professor(carlos, fisica, engenharia_eletrica).
professor(julia, algoritmos, ciencias_da_computacao).
professor(joao, redes, engenharia_de_software).

% Regras

% Quem ensina determinada disciplina?
quem_ensina(Disciplina, Professor) :-
    professor(Professor, Disciplina, _).

% Quem ensina em determinado curso?
quem_ensina_curso(Curso, Professor) :-
    professor(Professor, _, Curso).

% Quais disciplinas um professor ensina?
disciplinas_professor(Professor, Disciplina) :-
    professor(Professor, Disciplina, _).

% Ponto de entrada
:- initialization(main).

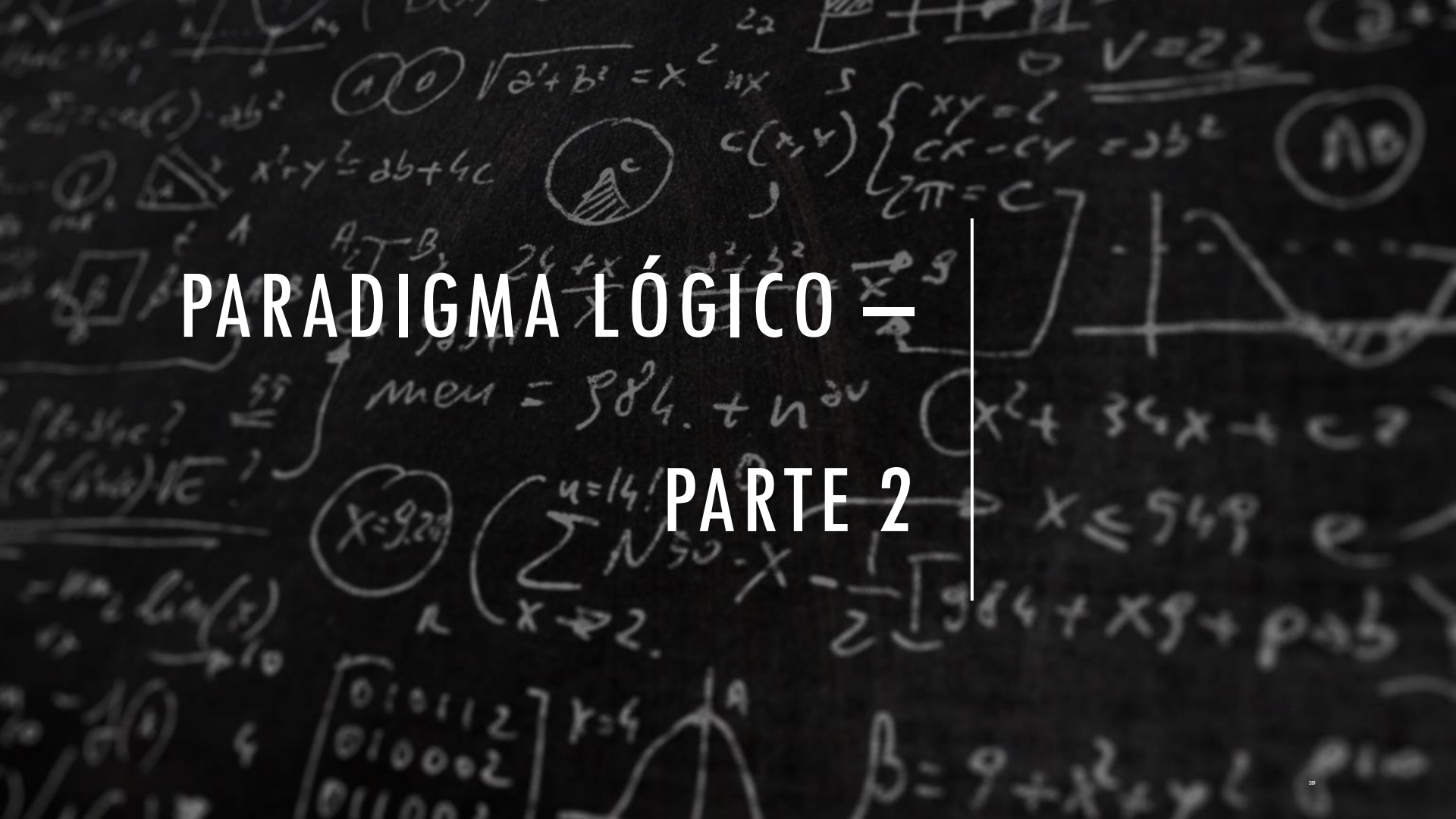
main :-
    % Consulta 1: quem ensina compiladores
    quem_ensina(compiladores, P1),
    format('Professor que ensina Compiladores: ~w~n', [P1]),

    % Consulta 2: quem ensina no curso de Engenharia Eletrica
    findall(P2, quem_ensina_curso(engenharia_eletrica, P2), ListaProfsEE),
    format('Professores de Engenharia Eletrica: ~w~n', [ListaProfsEE]),

    % Consulta 3: disciplinas que Miguel ministra
    findall(D, disciplinas_professor(miguel, D), ListaDiscMiguel),
    format('Disciplinas que Miguel ministra: ~w~n', [ListaDiscMiguel]),

    % Consulta 4: quem ensina no curso de Ciencias da Computacao
    findall(P3, quem_ensina_curso(ciencias_da_computacao, P3), ListaProfsCC),
    format('Professores de Ciencias da Computacao: ~w~n', [ListaProfsCC]),

    halt.
```



PARADIGMA LÓGICO – PARTE 2

OBJETIVOS

Conhecer a sintaxe dos operadores lógicos em Prolog.

Definir forma clausal e Cláusula de Horn.

Representar proposições compostas em Prolog.

Compreender os conceitos de Resolução, Unificação, Instanciação e Backtracking.

Conhecer as aplicações e as limitações da programação lógica.

PARADIGMA LÓGICO

Proposições compostas podem ser usadas para definir regras:

- O lado esquerdo é o consequente (então)
- O lado direito é o antecedente (se)
- Se o antecedente é verdadeiro então o consequente também é.
- Conectivos lógicos são utilizados para expressar as relações entre os diferentes predicados.

PARADIGMA LÓGICO

- O que estas sentenças afirmam?

pai(alan,bob) ^ mae(violeta,bob) ^ avo(luiz,bob) →
pai(luiz,alan) v pai(luiz,violeta)

OU

pai(luiz,alan) v pai(luiz,violeta) ←
pai(alan,bob) ^ mae(violeta,bob) ^ avo(luiz,bob)

PARADIGMA LÓGICO

- Um dos problemas do cálculo de predicados é que ele permite definir proposições com o mesmo significado de diversas maneiras (redundância).
- Este não é um problema se as proposições são analisadas por lógicos (humanos) mas sim se elas são interpretadas por sistemas automatizados.
- A forma clausal é uma forma padrão para representação de proposições, que não possui perda de generalidade.

PARADIGMA LÓGICO

- Sintaxe de uma proposição em forma clausal:
  - $A1 \wedge A_2 \wedge \ldots \wedge A_n \rightarrow B_1 \vee B_2 \vee \ldots \vee B_n$
- **ou ainda**
  - $B_1 \vee B_2 \vee \ldots \vee B_n \leftarrow A_1 \wedge A_2 \wedge \ldots \wedge A_n$
- **Se** (todos os termos A forem verdadeiros) **então** (ao menos um termo B será verdadeiro).
- Todas as proposições em cálculo de predicados podem ser convertida para a forma clausal.

PARADIGMA LÓGICO

- **Cláusulas de Horn** são tipos especiais de proposições em forma clausal, cujo consequente localiza-se a esquerda:
  - B_1 v B_2 v … v B_n ← A_1 ^ A_2 ^ … ^ A_n
- As cláusulas de Horn podem ser de duas formas:
  - Possuir apenas uma proposição atômica do lado esquerdo (com cabeça);
  - Possuir o lado esquerdo vazio (sem cabeça).
- Programas em Prolog são constituídos de sentenças representadas por **Cláusulas de Horn**, sejam elas proposições atômicas ou compostas.

PARADIGMA LÓGICO

- Cláusulas de Horn sem cabeça são usadas para definir fatos.
  - mae(joana,maria).
- Cláusulas de Horn com cabeça são usadas para definir relacionamentos:
  - pais(X, Y) ← mae(X, Y)
  - pais(X, Y) ← pai(X, Y)
  - avo(X, Z) ← pais(X, Y) ^ pais(Y, Z)
  - irmaos(X, Y) ← mae(M, X) ^ mae(M, Y) ^ pai (F, X) ^ pai (F, Y)

PARADIGMA LÓGICO

- Sintaxe dos principais operadores lógicos em Prolog.

Negação	¬a	not(a)
Conjunção	a ^ b	a,b
Disjunção	a v b	a;b
Implicação	a → b	b :- a

PARADIGMA LÓGICO

- Estas cláusulas de Horn, em Prolog são representadas da seguinte maneira

mae(joana,maria)	mae(joana,maria)
pais(X, Y) ← pai(X, Y)	pais(X, Y) :- pai(X,Y)
pais(X, Y) ← mae(X, Y)	pais(X, Y) :- mae(X,Y)
avo(X, Z) ← pais(X, Y) ^ pais(Y, Z)	avo(X, Z) :- pais(X, Y), pais(Y, Z)

irmaos(X, Y) ← (mae(M, X) ^ mae(M, Y)) v (pai (F, X) ^ pai (F, Y))

irmaos(X, Y) :- (mae(M, X), mae(M, Y));(pai (F, X), pai (F, Y))

EXEMPLO

Em um prédio inteligente, o acesso a determinadas áreas é concedido com base em permissões.

- Funcionários e administradores têm acesso às salas.
- Administradores também têm acesso à sala de servidores.
- Uma pessoa pode acessar um local se ela tiver permissão direta ou se for administrador.

```
1
2  % ----- Fatos -----
3  funcionario(joao).
4  administrador(maria).
5  funcionario(ana).
6
7  % ----- Regras -----
8  % Um funcionário tem permissão para acessar a sala.
9  tem_permissao(X, sala) :- funcionario(X).
10
11 % Um administrador tem permissão para acessar a sala de servidores.
12 tem_permissao(X, sala_servidores) :- administrador(X).
13
14 % Uma pessoa pode acessar um local se:
15 % 1. Ela tem permissão direta, OU
16 % 2. Ela é administradora.
17 pode_acessar(X, Local) :-
18     tem_permissao(X, Local);
19     administrador(X).
20
21 % Ponto de entrada
22 :- initialization(main).
23
24 main :-
25     (pode_acessar(joao, sala) ->
26         write('Joao pode acessar a sala'), nl ;
27         write('Joao NAO pode acessar a sala'), nl),
28
29     (pode_acessar(ana, sala_servidores) ->
30         write('Ana pode acessar a sala de servidores'), nl ;
31         write('Ana NAO pode acessar a sala de servidores'), nl),
32
33     (pode_acessar(maria, sala_servidores) ->
34         write('Maria pode acessar a sala de servidores'), nl ;
35         write('Maria NAO pode acessar a sala de servidores'), nl),
36
37     halt.
```

Input for the program (Optional)

Output:

```
Joao pode acessar a sala
Ana NAO pode acessar a sala de servidores
Maria pode acessar a sala de servidores
```

```
% ----- Fatos -----
funcionario(joao).
administrador(maria).
funcionario(ana).
administrador(pedro).
funcionario(luiza).

% ----- Regras -----
tem_permissao(X, sala) :- funcionario(X).
tem_permissao(X, sala_servidores) :- administrador(X).

pode_acessar(X, Local) :-
    tem_permissao(X, Local);
    administrador(X).

% Ponto de entrada
:- initialization(main).

main :-
    testar_acesso(joao, sala),
    testar_acesso(ana, sala_servidores),
    testar_acesso(maria, sala_servidores),

    % Novos testes adicionados
    testar_acesso(carlos, sala),                % Pessoa não registrada
    testar_acesso(maria, sala),                 % Admin acessando sala comum
    testar_acesso(ana, sala),                   % Funcionária acessando sala comum
    testar_acesso(joao, sala_servidores),       % Funcionário tentando acessar sala de servidores
    testar_acesso(pedro, sala_servidores),      % Outro administrador acessando servidores
    testar_acesso(luiza, sala),                 % Outro funcionário acessando sala comum
    testar_acesso(luiza, sala_servidores),      % Funcionária tentando acessar sala de servidores

    halt.

testar_acesso(Pessoa, Local) :-
    (pode_acessar(Pessoa, Local) ->
        format('~w pode acessar ~w~n', [Pessoa, Local]) ;
        format('~w NAO pode acessar ~w~n', [Pessoa, Local])).
```

Output:

```
joao pode acessar sala
ana NAO pode acessar sala_servidores
maria pode acessar sala_servidores
carlos NAO pode acessar sala
maria pode acessar sala
ana NAO pode acessar sala
joao NAO pode acessar sala_servidores
pedro NAO pode acessar sala_servidores
luiza NAO pode acessar sala
luiza NAO pode acessar sala_servidores
```

PARADIGMA LÓGICO

Inferência em Prolog:

- – Processo realizado para encontrar a resposta para uma consulta (objetivo).
- Para provar que um objetivo é verdadeiro, o processo de inferência deve encontrar um conjunto de regras e/ou fatos na base que conecte o objetivo a um ou mais fatos na base.
- Este processo pode ser feito por encadeamento para frente (forward chaining) ou para trás (backward chaining) e implementado por uma busca em largura ou profundidade.

PARADIGMA LÓGICO

- Se Q é um objetivo então:
  - Q deve ser encontrado na base

ou

  - P1
  - P2 :- P1
  - P3 :- P2
  - Q :- P3

PARADIGMA LÓGICO

Consulta homem(bob)

– Possibilidade 1:
- homem(bob) na base

– Possibilidade 2:
- pai(bob)
- homem(X) :- pai(X)

PARADIGMA LÓGICO

- **Resolução** é uma regra de inferência que permite computar as proposições a partir de proposições dadas.
  - P2 ← P1
  - P3 ← P2
  - Q ← P3
- Resolução
  - Q ← P1

PARADIGMA LÓGICO

- Definições:
  - maisvelho(joana,paulo) ← mae(joana,paulo)
  - maisexperiente(joana,paulo) ← maisvelho(joana,paulo)
- Resolução:
  - maisexperiente(joana,paulo) ← mae(joana,paulo)
- Os termos do lado esquerdo das diferentes proposições são unidos por um conector lógico E e os do lado direito também. Por fim, os termos que aparecem em ambos os lados são excluídos.

PARADIGMA LÓGICO

Unificação

- – Processo utilizado para encontrar valores para variáveis que permitam ao relacionamento ser correto.

- **Instanciação**
- – É o processo de atribuição temporária de valores às variáveis.

Backtracking ou Rastreamento para trás

- – Retorno quando o processo de resolução instancia uma variável que não possibilita um relacionamento correto.

PARADIGMA LÓGICO

Dada a base de conhecimento abaixo, como é o processo de inferência para realizar a consulta **filho(X,joão)**.

- pai(joão,ana).
- pai(joão,maria).
- pai(joão,carlos).
- pai(carlos,renata).
- mae(maria,paula).
- filho(X, Y) :- pai(Y, X);mae(Y, X).
- avo(X, Z) :- pais(X, Y), pais(Y, Z)
- irmaos(X, Y) :- (mae(M, X), mae(M, Y));(pai (F, X), pai (F, Y))

ATIVIDADE

Dada a base de conhecimento abaixo, como é o processo de inferência para realizar a consulta **avo(Y,paula).**

- pai(joão,ana).
- pai(joão,maria).
- pai(joão,carlos).
- pai(carlos,renata).
- mae(maria,paula).
- filho(X, Y) :- pai(Y, X);mae(Y, X).
- avo(X, Z) :- pais(X, Y), pais(Y, Z)
- irmaos(X, Y) :- (mae(M, X), mae(M, Y));(pai (F, X), pai (F, Y))

PARADIGMA LÓGICO

Aplicações da Programação Lógica:

- Sistemas especialistas:
  - Medicina;
  - Educação;
  - Monitoramento e Controle;
  - Planejamento, etc.
- Prova de teoremas.
- Consultas em bancos de dados.
- Processamento de linguagem natural.

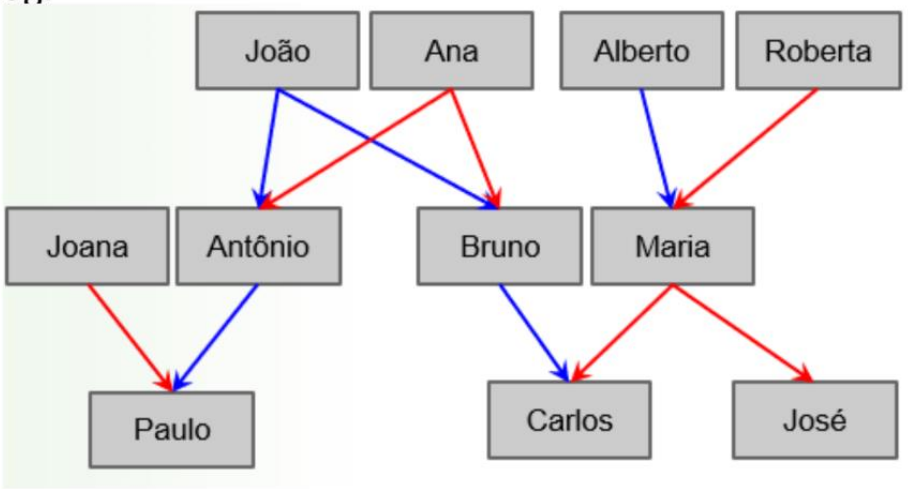
PARADIGMA LÓGICO

Deficiências do Prolog:

- Controle da ordem de resolução.
- Premissa do mundo fechado.
- Problema da negação.
- Ineficiência em certos casos, por exemplo, na ordenação de um vetor.

ATIVIDADE

Atividade 1 (1/2). Represente a árvore genealógica abaixo em Prolog.



ATIVIDADE

Atividade 1 (2/2): Realize as seguintes consultas:

- Antônio e Bruno são irmãos?
- Joana e Antônio são irmãos?
- Quem é o avô de Paulo?
- Quem é a avó de Paulo?
- Quem são os primos de Carlos?
- Quem são os primos de José?
- Quem é irmão de quem nesta árvore?
- Quem é avó de quem nesta árvore?

ATIVIDADE

Atividade 2.
Baixe a base de conhecimento de um sistema especialista que sugere o tipo de vinho a ser consumido em uma refeição, de acordo com o prato principal e o tipo de molho.

- ● Crie um programa em Prolog usando o OneCompiler que leia o prato a ser consumido, o tipo de molho e apresente o vinho sugerido.

PARADIGMA LÓGICO

PARADIGMA ORIENTADO A OBJETOS

Miguel Enrique Parra Munoz

OBJETIVOS

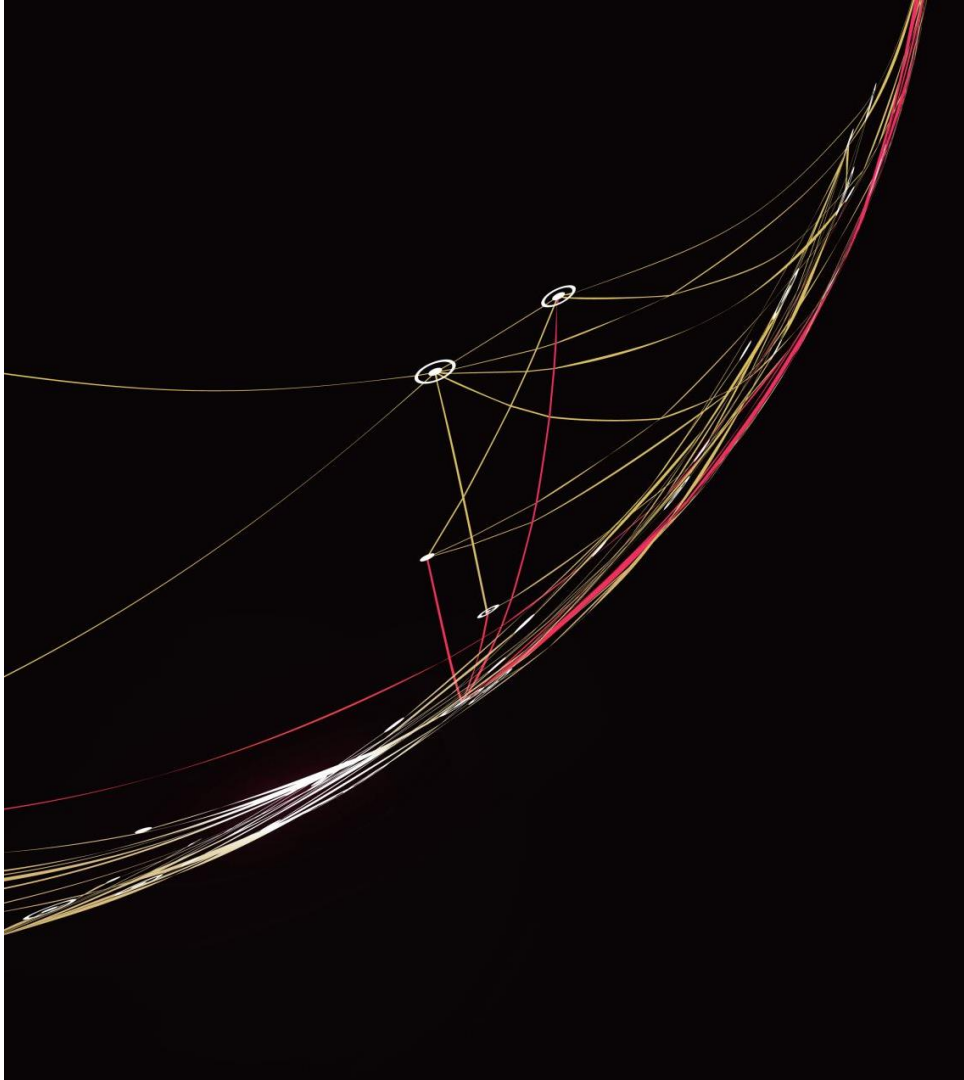
Revisar a diferença entre abstração de processos e

abstração de dados.

Conceituar tipo de dado abstrato e encapsulamento.

Conceituar classes, atributos e métodos.

Diferenciar a definição de uma classe em C++ e Javascript.



INTRODUÇÃO

Histórico:

– Crescente demanda por soluções de software e aumento na complexidade das mesmas (Crise de Software);

– Necessidade de aumentar a produtividade dos programadores:

Reusabilidade.

Manutenibilidade.

Extensibilidade.

Os recursos ofertados pelos paradigmas imperativo, funcional e lógico dificultavam estas tarefas.

INTRODUÇÃO

Reusabilidade é a utilização de conceitos ou soluções previamente elaboradas para a criação de um novo software.

Extensibilidade refere-se às condições associadas à facilidade de se estender o software:

– Adição de novas funcionalidades.

Manutenibilidade refere-se às condições associadas à facilidade de se alterar o software:

– Correções de bugs.

– Alteração de funcionalidades.

PARADIGMA ORIENTADO A OBJETOS

A Programação Orientada a Objetos está embasada em três recursos principais:

– Tipos de dados abstratos (classes).

– Herança.

– Vinculação dinâmica de chamadas à métodos (polimorfismo).

DEFINIÇÃO DO PARADIGMA ORIENTADO A OBJETOS

O **paradigma orientado a objetos (POO)** é um modelo de programação que organiza o código em **"objetos"**, que representam **entidades do mundo real.**
Cada objeto combina **dados (atributos)** e **comportamentos (métodos),** permitindo que o software seja mais modular, reutilizável e fácil de manter.

Em vez de pensar o programa como uma sequência de instruções (como na programação estruturada), a POO vê o sistema como uma **coleção de objetos que interagem entre si.**

DEFINIÇÃO DO PARADIGMA ORIENTADO A OBJETOS

Conceitos Fundamentais da POO

Classe: é o molde ou modelo que define as características e comportamentos de um objeto.
Exemplo: Classe: Carro → atributos: cor, modelo; métodos: acelerar(), frear().

Objeto: é uma instância de uma classe.
Exemplo: meuCarro = Carro("vermelho", "Fiat")

DEFINIÇÃO DO PARADIGMA ORIENTADO A OBJETOS

Encapsulamento: protege os dados, permitindo acesso apenas por métodos específicos.

Herança: permite criar novas classes baseadas em classes existentes, reutilizando código.

Polimorfismo: possibilita que métodos com o mesmo nome se comportem de forma diferente conforme o objeto.

Abstração: simplifica a complexidade, mostrando apenas o que é essencial.

DEFINIÇÃO DO PARADIGMA ORIENTADO A OBJETOS

Aplicações da POO

A POO é amplamente usada no **desenvolvimento de softwares complexos,** como:

Sistemas empresariais (ERP, CRM, bancos de dados)

Jogos digitais

Aplicativos móveis e web

Sistemas embarcados e **simulações**

DEFINIÇÃO DO PARADIGMA ORIENTADO A OBJETOS

Softwares e Linguagens que Usam POO

Linguagens orientadas a objetos:

- Java
- C++
- Python
- C#
- Ruby
- Kotlin
- Swift

DEFINIÇÃO DO PARADIGMA ORIENTADO A OBJETOS

Softwares que utilizam POO:

Microsoft Office (internamente em C++ e .NET)

Android (usando Java/Kotlin)

AutoCAD

Photoshop

Sistemas bancários e **plataformas de e-commerce** (como Amazon e eBay)

DEFINIÇÃO DO PARADIGMA ORIENTADO A OBJETOS

Vantagens do Paradigma Orientado a Objetos

Reutilização de código por meio da herança e classes genéricas.

Facilidade de manutenção alterações locais afetam menos o sistema.

Maior modularidade o código é dividido em partes independentes.

Maior segurança e controle de acesso com o encapsulamento.

Facilidade de modelagem o código reflete melhor o mundo real.

Escalabilidade permite expansão e evolução do sistema com menos impacto.

DEFINIÇÃO DO PARADIGMA ORIENTADO A OBJETOS

Desvantagens da POO

Maior complexidade inicial exige mais planejamento e abstração.

Maior consumo de recursos (memória e processamento) em comparação com abordagens mais simples.

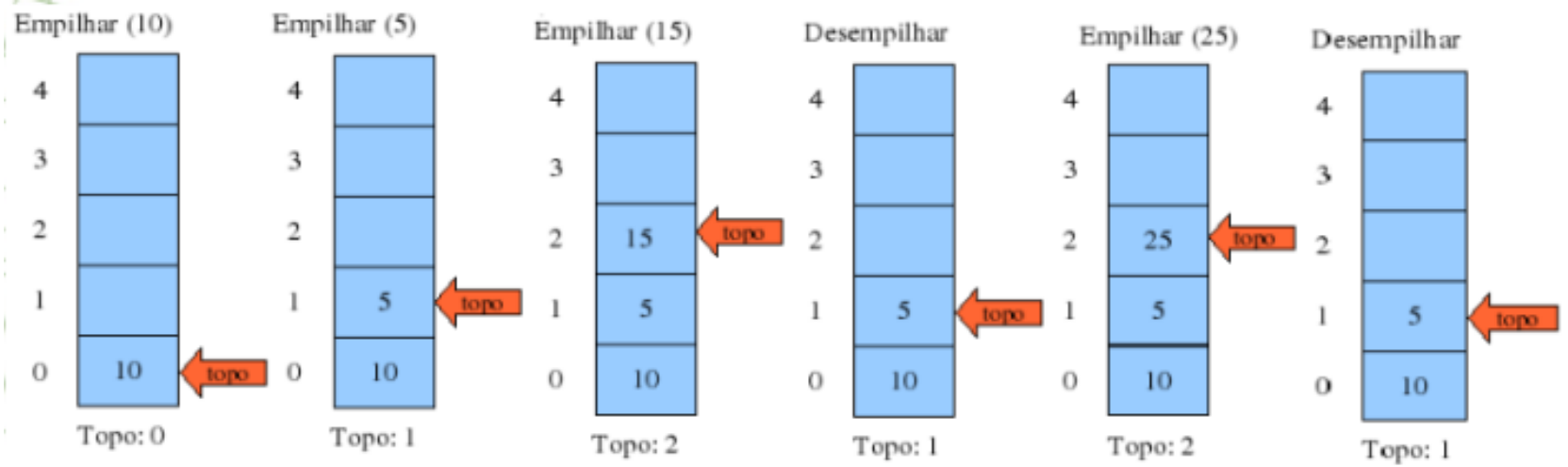
Curva de aprendizado mais longa para iniciantes.

Nem todos os problemas se encaixam bem em um modelo orientado a objetos.

EXEMPLO

Exemplo: Implementação de uma Pilha.

– Pilha é uma estrutura de dados que só permite a inserção e a retirada de elementos no seu topo.



EXEMPLO

Def

Self

constructor

main.py

Run

Output

```python
class Carro:
    def __init__(self, modelo, cor):
        self.modelo = modelo
        self.cor = cor

    def acelerar(self):
        print(f"O {self.modelo} está acelerando!")

# Criando objetos (instâncias da classe)
carro1 = Carro("Fiat Uno", "Vermelho")
carro2 = Carro("Honda Civic", "Preto")

# Usando métodos
carro1.acelerar()
carro2.acelerar()
```

```
O Fiat Uno está acelerando!
O Honda Civic está acelerando!

=== Code Execution Successful ===
```

EQUIVALENTE EM C

```cpp
#include <iostream>
using namespace std;
class Carro {
private:
    string modelo;
    string cor;
public:
    // Construtor
    Carro(string m, string c) {
        modelo = m;
        cor = c;
    }
    // Método
    void acelerar() {
        cout << "O " << modelo << " está acelerando!" << endl;
    }
};

int main() {

    // Criando objetos
    Carro carro1("Fiat Uno", "Vermelho");
    Carro carro2("Honda Civic", "Preto");

    // Chamando métodos
    carro1.acelerar();
    carro2.acelerar();

    return 0;
}
```

EQUIVALENTE JAVASCRIPT

main.js

```javascript
class Carro {

    // Construtor
    constructor(modelo, cor) {
        this.modelo = modelo;
        this.cor = cor;
    }

    // Método
    acelerar() {
        console.log(`O ${this.modelo} está acelerando!`);
    }
}

// Criando objetos
let carro1 = new Carro("Fiat Uno", "Vermelho");
let carro2 = new Carro("Honda Civic", "Preto");

// Chamando métodos
carro1.acelerar();
carro2.acelerar();
```

COMPARAÇÕES

Conceito	Python	C++	JavaScript
Classe	`class Carro:`	`class Carro {}`	`class Carro {}`
Construtor	`__init__`	Mesmo nome da classe	`constructor`
Referência ao objeto	`self`	Implícita	`this`
Método	`def acelerar()`	`void acelerar()`	`acelerar()`
Imprimir	`print()`	`cout <<`	`console.log()`

ABSTRAÇÃO EM LINGUAGENS DE PROGRAMAÇÃO:

– **Abstração de processos:**

Subprogramas

Característica básica da programação imperativa.

– **Abstração de dados**

Tipos de dados abstratos.

– Atributos (dados)

– Comportamentos (subprogramas).

Base para a programação orientada a objetos.

ABSTRAÇÃO EM LINGUAGENS DE PROGRAMAÇÃO:

"**Um tipo abstrato** de dados é um invólucro que inclui apenas a representação de dados de um tipo de dados específico e os subprogramas que fornecem as operações para este tipo" (SEBESTA, 2010).

Resumindo, um tipo de dado abstrato deve fornecer:

– Uma definição de tipo que permite às unidades de programa declararem variáveis do tipo, mas que oculte a representação de seus dados (encapsulamento).

– Um conjunto de métodos para manipular os dados contidos no tipo de dado abstrato.

ENCAPSULAMENTO

"**Encapsulamento** é o mecanismo que permite que constantes logicamente relacionadas, tipos, variáveis, métodos, entre outros, sejam agrupadas em uma nova entidade" (Tucker e Noonan, 2008).

– Os dados são chamados de atributos ou variáveis.

– Eles podem aparecer nas classes e nas instâncias.

– As funções são chamadas de funções membros ou métodos.

– Todas as instâncias compartilham o mesmo conjunto de métodos, mas cada instância tem seu próprio conjunto de atributos.

TIPOS DE DADOS ABSTRATOS

Tipos de dados abstratos são tipos que:

– Permitem a declaração de variáveis deste tipo.

– A representação interna das variáveis é ocultada das unidades do programa que as utilizam, sendo as suas operações as únicas formas para acessá-las.

– Possuem um conjunto de operações para manipular as variáveis criadas.

– A interface do tipo não depende da representação dos objetos ou da implementação das operações.

– Permitem que outras unidades do programa, denominadas clientes, utilizem seus serviços.

TIPOS DE DADOS ABSTRATOS

O uso de tipos de dados abstratos traz como vantagem a confiabilidade e legibilidade do código, pois:

– Clientes não podem alterar os valores de seus atributos diretamente, seja de forma intencional ou por acidente.

– Reduz a quantidade de código e de variáveis que o programador deve estar atento em um trecho do programa.

– Reduz os conflitos de nomes por gerar escopos menores.

TIPOS DE DADOS ABSTRATOS

Implementação de tipos de dados abstratos

– As primeiras linguagens a incorporarem o conceito de tipo de dado abstrato, implementavam-nos por meio de pacotes.

– Algumas linguagens como C++, C# e Object Pascal (Delphi) permitem a abstração por meio de classes e registros (structs e records).

– Java e Smalltalk não possuem o conceito de registros, todas as abstrações devem ser construídas como classes.

O QUE É UMA CLASSE?

Uma classe é uma declaração de tipo que encapsula constantes, variáveis e funções para a manipulação destas variáveis.

– Ela define estado e comportamento dos objetos.

– Suas variáveis, chamadas de atributos ou variáveis de instância, podem ser inicializadas pelos construtores e finalizadas pelos destrutores.

– As demais funções são chamadas de métodos.

– Define-se como Cliente de uma classe X qualquer outra classe ou método que declara e usa um objeto da classe X.

MODIFICADORES DE ACESSO:

uma classe pode ter atributos e métodos ocultos ou visíveis aos clientes da classe;

– public – qualquer classe tem acesso.

– private – apenas a própria classe tem acesso;

– protected – apenas a própria classe e suas classes derivadas (herança) tem acesso.

Em geral, os métodos são declarados públicos e os atributos privados ou protegidos, sendo o seu acesso realizado por métodos get e set.

MODIFICADORES DE ACESSO:

A criação de uma classe em C++, diferentemente do Java, pode se dar de duas maneiras:

– Definição completa na própria classe (interface e implementação em único arquivo). Ideal para a programação de sistemas de tempo real.

– Separação entre a interface e a implementação em arquivos separados (melhora legibilidade e facilita a manutenção, um dos objetivos da programação moderna).

EXEMPLO DE CÓDIGO 1

```cpp
// Define o tamanho máximo da pilha
#define TAMANHO 5

class Pilha {
private:
    // Array para armazenar os elementos
    int vetPilha[TAMANHO];

    // 'topo' indica o número de elementos
    // e também o próximo índice livre
    int topo;

public:
    // Construtor
    Pilha() {
        topo = 0; // Começa vazia
    }

    // Destrutor
    ~Pilha() {
        // Nada a fazer para um array estático
    }

    // Empilhar (adicionar) um elemento
    void push(int n) {
        if (topo == TAMANHO) {
            // A pilha está cheia
            cout << "Erro - Pilha cheia!\n";
        } else {
            // Adiciona o elemento na posição 'topo'
            vetPilha[topo] = n;

            // Incrementa o topo
            topo++;
        }
    }

    // Desempilhar (remover) um elemento
    int pop() {
        if (topo == 0) {
            // A pilha está vazia
            cout << "Erro - Pilha vazia!" << endl;
            return -1; // Retorna um valor de erro
        } else {
            // Decrementa o topo primeiro
            topo--;

            // Retorna o elemento que estava no topo
            return vetPilha[topo];
        }
    }
};
// --- Função principal para testar a Pilha ---
int main() {
    Pilha p1;
    cout << "Empilhando 10, 20, 30...\n";
    p1.push(10);
    p1.push(20);
    p1.push(30);

    cout << "Desempilhando:\n";
    cout << p1.pop() << endl; // Sai 30
    cout << p1.pop() << endl; // Sai 20
    cout << "Empilhando 40...\n";
    p1.push(40);

    cout << "Desempilhando o resto:\n";
    cout << p1.pop() << endl; // Sai 40
    cout << p1.pop() << endl; // Sai 10

    // Testando pilha vazia
    cout << "Testando pop em pilha vazia:\n";
    p1.pop(); // Vai imprimir "Erro - Pilha vazia!"

    return 0;
}
```

```
#include <iostream>

using namespace std;

// Define o tamanho máximo da pilha

#define TAMANHO 5

class Pilha {

private:

   // Array para armazenar os elementos

   int vetPilha[TAMANHO];

   // 'topo' indica o número de elementos

   // e também o próximo índice livre

   int topo;

public:

   // Construtor

   Pilha() {

      topo = 0; // Começa vazia

   }

   // Destrutor

   ~Pilha() {

      // Nada a fazer para um array estático

   }


   // Empilhar (adicionar) um elemento

   void push(int n) {

      if (topo == TAMANHO) {

         // A pilha está cheia

         cout << "Erro - Pilha cheia!\n";
```

OBJETOS DINÂMICOS

Os objetos dinâmicos de um programa devem ser criados e destruídos:

– Durante a criação de um objeto, recursos e áreas de memória podem ser alocados.

– Em algumas linguagens, é dever do programador prever a liberação destes recursos e da memória sempre que necessário durante a destruição do objeto.

– Algumas linguagens realizam coleta de lixo automática (Java).

EXERCÍCIO DE AULA

Desenvolva um sistema de calculadora básica utilizando os conceitos de Programação Orientada a Objetos nas linguagens:

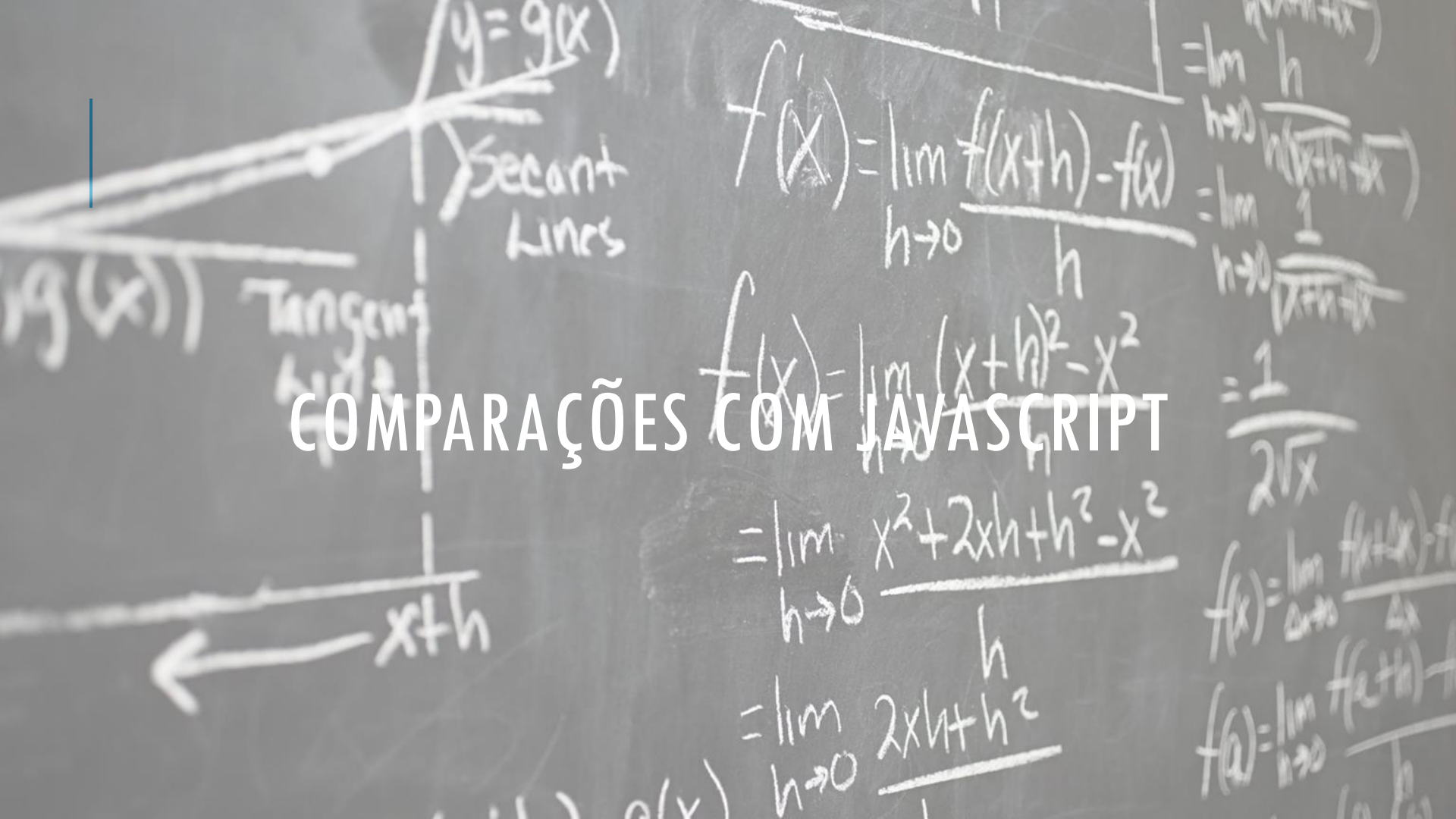
Python

C++

JavaScript

O objetivo da **atividade é comparar a sintaxe,** estrutura e implementação das três linguagens.

Chame a classe principal de calculadora e crie um objeto para cada operação.



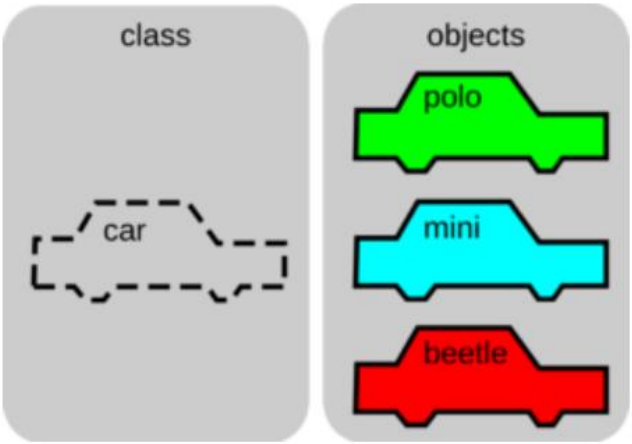
COMPARAÇÕES COM JAVASCRIPT

RESUMO

- Objetos em programação são uma forma de representar coisas ou conceitos no contexto de um programa.
- Permitem a criação de um código minimalista a partir de módulos reutilizáveis.
- Uma classe define as propriedades e os métodos que caracterizam um objeto.
- Cada objeto (instância) criado a partir dessa classe terá seus próprios valores específicos e poderá executar ações definidas pelos métodos.

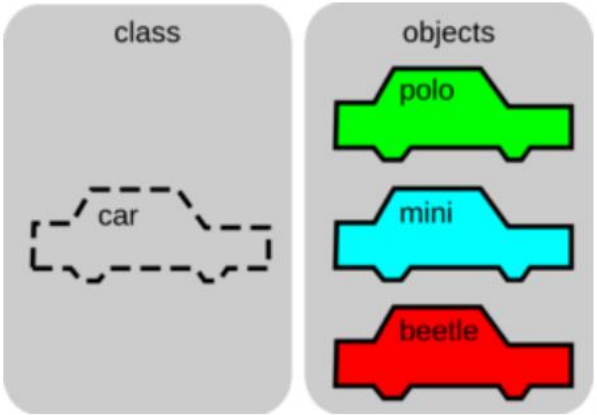
O QUE SÃO CLASSES E OBJETOS?

- Imagine que você comprou um carro recentemente e decide modelar esse carro usando programação orientada a objetos. O seu carro tem as características que você estava procurando:
- um motor 2.0 híbrido, azul escuro, quatro portas, câmbio automático etc.
- Ele também possui comportamentos que, provavelmente, foram o motivo de sua compra, como acelerar, desacelerar, acender os faróis, buzinar e tocar música.
- Podemos dizer que o carro novo é um objeto, onde suas características são seus atributos (dados atrelados ao objeto) e seus comportamentos são ações ou métodos.



O QUE SÃO CLASSES E OBJETOS?

- Seu carro é um objeto seu mas na loja onde você o comprou existiam vários outros, muito similares, com quatro rodas, volante, câmbio, retrovisores, faróis, dentre outras partes.
- Observe que, apesar do seu carro ser único (por exemplo, possui um registro único no Departamento de Trânsito), podem existir outros com exatamente os mesmos atributos, ou parecidos, ou mesmo totalmente diferentes, mas que ainda são considerados carros.
- Podemos dizer então que seu objeto pode ser classificado (isto é, seu objeto pertence à uma classe) como um carro, e que seu carro nada mais é que uma instância dessa classe chamada "carro".



COMO FAZER A PROGRAMAÇÃO EM POO

JAVA , JAVASCRIPT, PYTHON

```java
public class Carro {
    Double velocidade;
    String modelo;

    public Carro(String modelo) {
        this.modelo = modelo;
        this.velocidade = 0;
    }

    public void acelerar() {
        /* código do carro para acelerar */
    }

    public void frear() {
        /* código do carro para frear */
    }

    public void acenderFarol() {
        /* código do carro para acender o farol */
    }
}
```

```javascript
// Definição da classe
class Carro {
  constructor(marca, modelo, ano) {
    this.marca = marca;
    this.modelo = modelo;
    this.ano = ano;
  }

  // Método para exibir informações do carro
  exibirInfo() {
    console.log(`Carro: ${this.marca} ${this.modelo}, Ano: ${this.ano}`);
  }
}

// Instanciando objetos da classe Carro
const carro1 = new Carro('Toyota', 'Corolla', 2022);
const carro2 = new Carro('Honda', 'Civic', 2020);

// Chamando o método para exibir informações dos carros
carro1.exibirInfo();
carro2.exibirInfo();
```

```python
class Carro:
    def __init__(self, modelo):
        self.modelo = modelo;
        self.velocidade = 0

    def acelerar(self):
        # Codigo para acelerar o carro

    def frear(self):
        # Codigo para frear o carro

    def acenderFarol(self):
        # Codigo para acender o farol do carro
```

EXEMPLOS DE APLICAÇÃO

```
1  class Pessoa {
2    constructor(nome, idade) {
3      this.nome = nome;
4      this.idade = idade;
5    }
6
7    falar() {
8      console.log(`Olá, meu nome é ${this.nome} e tenho ${this.idade} anos.`);
9    }
10 }
11
12 const pessoa1 = new Pessoa("João", 25);
13 const pessoa2 = new Pessoa("Matheus", 20);
14 pessoa1.falar(); // Output: "Olá, meu nome é João e tenho 25 anos."
15 pessoa2.falar();
```

EXEMPLO

```
class CalculadoraIMC {
  constructor(peso, altura) {
    this.peso = peso;
    this.altura = altura;
  }
  calcularIMC() {
    return this.peso / (this.altura ** 2);
  }
  classificarIMC() {
    const imc = this.calcularIMC();
    if (imc < 18.5) {
      return 'Abaixo do peso';
    } else if (imc < 25) {
      return 'Peso normal';
    } else if (imc < 30) {
      return 'Sobrepeso';
    } else {
      return 'Obesidade';
    }
  }
}
```

Neste exemplo, a classe IMCCalculator recebe o peso e a altura da pessoa no construtor.

Ela possui dois métodos: calcularIMC() para calcular o IMC e classificarIMC() para classificar o IMC em uma categoria (abaixo do peso, peso normal, sobrepeso ou obesidade) com base no valor calculado.

Ao instanciar um objeto da classe IMCCalculator e chamar esses métodos, o programa calcula e exibe o IMC e sua classificação.

```
// Instanciando um objeto da classe CalculadoraIMC
const calculadora = new CalculadoraIMC(70, 1.75);
// Calculando o IMC
const imc = calculadora.calcularIMC();
console.log('O IMC é:', imc);
// Classificando o IMC
const classificacao = calculadora.classificarIMC();
console.log('Classificação:', classificacao);
```

EXEMPLO CALCULADORA

```
class Calculadora {
  // Método para somar dois números
  somar(num1, num2) {
    return num1 + num2;
  }
  // Método para subtrair dois números
  subtrair(num1, num2) {
    return num1 - num2;
  }
  // Método para multiplicar dois números
  multiplicar(num1, num2) {
    return num1 * num2;
  }
  // Método para dividir dois números
  dividir(num1, num2) {
    if (num2 === 0) {
      return 'Erro: divisão por zero';
    }
    return num1 / num2;
  }
  // Método para calcular a raiz quadrada de um número
  raizQuadrada(num) {
    return Math.sqrt(num);
  }
  // Método para calcular o logaritmo de um número na base 10
  logaritmo(num) {
    return Math.log10(num);
  }
  // Método para calcular a exponencial de um número
  exponencial(num, exp) {
    return num ** exp;
  }
}

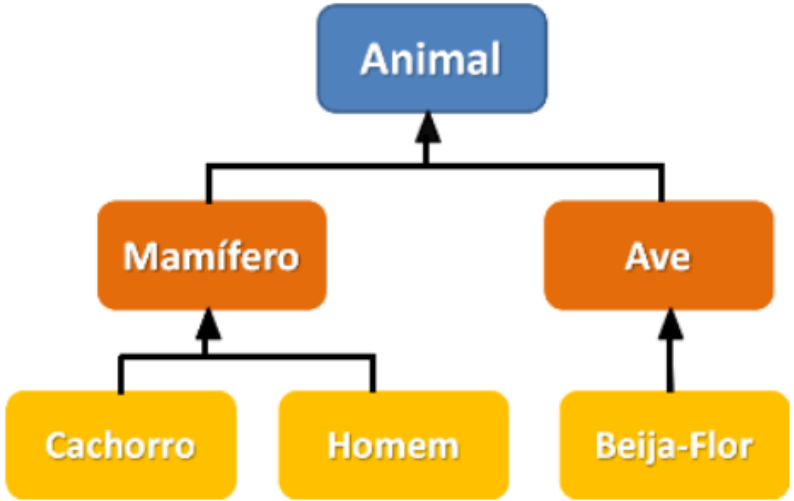
// Instanciando um objeto da classe Calculadora
const calculadora = new Calculadora();

// Realizando operações de exemplo
console.log('Soma:', calculadora.somar(5, 3));
console.log('Subtração:', calculadora.subtrair(5, 3));
console.log('Multiplicação:', calculadora.multiplicar(5, 3));
console.log('Divisão:', calculadora.dividir(6, 2));
console.log('Raiz Quadrada:', calculadora.raizQuadrada(9));
console.log('Logaritmo:', calculadora.logaritmo(100));
console.log('Exponencial:', calculadora.exponencial(2, 3));
```

HERANÇA

A herança em programação orientada a objetos é um conceito em que uma classe pode herdar características (atributos e métodos) de outra classe. A classe que herda é chamada de classe filha ou subclasse, e a classe que é herdada é chamada de classe pai ou superclasse.

Quando dizemos que uma classe A é um tipo de classe B, dizemos que a classe A herda as características da classe B e que a classe B é mãe da classe A, estabelecendo então uma relação de herança entre elas.



```
1  // Classe pai
2  class Animal {
3    constructor(nome) {
4      this.nome = nome;
5    }
6
7    comer() {
8      console.log(`${this.nome} está comer
9    }
10 }
11 // Classe filha que herda de Animal
12 class Cachorro extends Animal {
13   latir() {
14     console.log(`${this.nome} está latir
15   }
16 }
17 // Instanciando um objeto da classe filh
18 const meuCachorro = new Cachorro('Rex');
19 // Chamando métodos da classe pai
20 meuCachorro.comer();
21 // Chamando método da classe filha
22 meuCachorro.latir();
```

EXEMPLO

Neste exemplo, a classe Animal é a classe pai, que possui um construtor que define o nome do animal e um método comer().

A classe Cachorro é uma classe filha que herda de Animal usando a palavra-chave extends.

Ela adiciona um método latir().

Quando instanciamos um objeto da classe Cachorro e chamamos seus métodos, podemos ver que ele pode acessar tanto os métodos da classe pai quanto os seus próprios.

HERANÇA DE CLASSES EM JAVASCRIPT

Na herança de classes em JavaScript, uma classe pode ser considerada como a “classe pai” ou “superclasse”, enquanto a classe que herda as propriedades e métodos é denominada “classe filha” ou “subclasse”. A classe filha herda todas as características da classe pai e também pode adicionar novos comportamentos ou modificar os existentes.

HERANÇA BASEADA EM PROTÓTIPOS

A herança de classes em JavaScript é baseada no conceito de protótipos. Cada objeto em JavaScript possui um protótipo, que é um objeto do qual ele herda propriedades e métodos.

Quando uma propriedade ou método é acessado em um objeto, o JavaScript primeiro verifica se o objeto possui essa propriedade ou método. Se não, ele procura no protótipo do objeto e continua subindo na cadeia de protótipos até encontrar a propriedade ou método desejado.

VANTAGENS DA HERANÇA DE CLASSES EM JAVASCRIPT

A herança de classes em JavaScript permite criar uma hierarquia de classes, onde as classes filhas herdam as propriedades e métodos da classe pai. Isso proporciona uma estrutura organizada e modular para o código, facilitando a manutenção e o desenvolvimento de aplicações complexas.

IMPLEMENTAÇÃO DA HERANÇA DE CLASSES EM JAVASCRIPT

```
class Animal {
    constructor(nome) {
      this.nome = nome;
    }
      falar() {
      console.log(this.nome, 'faz barulho.`);
    }
  }
```

```
class Cachorro  extend Animal {
    falar() {
      console.log(this.nome, 'faz au au');
    }
  }
  let meuCachorro = new Cachorro("Rex");
 meuCachorro.falar(); // Output: Rex faz au au.
```

COMO FUNCIONA A HERANÇA DE CLASSES EM JAVASCRIPT?

A herança de classes em JavaScript funciona por meio do mecanismo de protótipos da linguagem. Cada objeto em JavaScript possui um protótipo, que é um objeto do qual ele herda propriedades e métodos.

Cadeia de protótipos

Quando uma propriedade ou método é acessado em um objeto, o JavaScript primeiro verifica se o objeto possui essa propriedade ou método. Se não, ele procura no protótipo do objeto e continua subindo na cadeia de protótipos até encontrar a propriedade ou método desejado.

IMPLEMENTANDO A HERANÇA DE CLASSES EM JAVASCRIPT

A implementação da herança de classes em JavaScript geralmente envolve o uso da palavra-chave “extends” para criar uma classe filha que herda de uma classe pai.

Além disso, é possível utilizar o método “super()” para chamar o construtor da classe pai.

EXEMPLO

```
class Animal {
  constructor(nome) {
    this.nome = nome;
  }

  falar() {
    console.log(`${this.nome} faz barulho.`);
  }
}

class Cachorro extends Animal {
  constructor(nome, raca) {
    super(nome);
    this.raca = raca;
  }

  falar() {
    console.log(`${this.nome} faz au au.`);
  }

  mostrarRaca() {
    console.log(`Sou um ${this.raca}.`);
  }
}

let meuCachorro = new Cachorro("Rex", "Labrador");
meuCachorro.falar(); // Output: Rex faz au au.
meuCachorro.mostrarRaca(); // Output: Sou um Labrador.
```

classe principal

impressão, ajuste o método

usa estends para uma subclasse.

imprime novamente a subclasse, ajustar o método

faz chanadi da subclasse e os métodos de impressão

IMPORTANTE

1. **Criação de uma hierarquia de classes:** A herança de classes permite organizar o código em uma hierarquia de classes, onde classes filhas herdam as propriedades e métodos da classe pai. Isso facilita a reutilização de código e a criação de estruturas bem definidas.

2. **Adição de comportamentos específicos**: As classes filhas podem adicionar comportamentos específicos, além de herdar os comportamentos da classe pai. Isso permite a personalização de funcionalidades e a criação de classes mais especializadas.

3. **Sobrescrita de métodos**: As classes filhas podem sobrescrever os métodos da classe pai, fornecendo uma implementação diferente ou adicionando novas funcionalidades. Isso permite a adaptação do comportamento dos objetos de acordo com suas necessidades específicas.

4. **Polimorfismo**: A herança de classes em JavaScript também permite a aplicação do conceito de polimorfismo, onde objetos de diferentes classes podem ser tratados de maneira uniforme. Isso facilita a criação de interfaces genéricas e o desenvolvimento de código mais flexível e extensível.

```
class LojaAutomotores {
  constructor(preco) {
    this.preco = preco;
  }

  calcularParcelado() {
    return (this.preco * 1.3) / 48;
  }
}

class CarroPequeno extends LojaAutomotores {
  constructor(preco) {
    super(preco);
  }
}

class CarroGrande extends LojaAutomotores {
  constructor(preco) {
    super(preco);
  }
}

class Moto extends LojaAutomotores {
  constructor(preco) {
    super(preco);
  }
}
```

```
function comprarVeiculo(preco) {
  let veiculo;
  if (preco <= 10000) {
    veiculo = new Moto(preco);
  } else if (preco <= 90000) {
    veiculo = new CarroPequeno(preco);
  } else if (preco <= 200000) {
    veiculo = new CarroGrande(preco);
  } else {
    console.log("Não vendemos veículos com preço acima de 200 mil.");
    return;
  }

  console.log('Preço total: R',veiculo.preco);
  console.log('Valor da parcela (48x):', veiculo.calcularParcelado());
}


comprarVeiculo(15000); // Moto
```

Neste código, se analisa uma loja de automotores, onde ela vende parcelado aumentando 30% do preço, vende carro pequeno, carro grande e moto. Escreva o programa e modifique ele, para que em cada subclasse dê três opções de pagamento, faça com um **switch as opções.**

CONCEITOS COMPLEMENTARES

Quem é this?

Quando utilizamos this trata-se do próprio escopo atual onde o mesmo está sendo referenciado, ou seja, dentro da classe o this nada mais é do que uma referência da própria classe.

Para que serve o this?

Para referenciarmos a própria classe (tratando-se apenas de OO) dentro de funções ou métodos.

Porque usar o this?

Para referênciar atributos ou funções da própria classe dentro de outras funções da mesma classe.

CONCEITOS COMPLEMENTARES

Criando nosso objeto

Pronto, já temos a estrutura de nossa classe definida e criada, para você que se perdeu, segue abaixo o esqueleto completo:

```
class Pessoa {

  constructor(nome, idade, sexo, dataDeNascimento) {

    this.nome = nome;

    this.idade = idade;

    this.sexo = sexo;

    this.dataDeNascimento = dataDeNascimento;

  }

  apresentar() {

    console.log('Ola, meu nome é ',this.nome, 'eu tenho' ,this.idade, 'anos`);

  }

}
```

CONCEITOS COMPLEMENTARES

Podemos criar nossos objetos através da palavra reservada new seguida pelo nome da classe que desejamos instânciar

const matheus = new Pessoa("Matheus", 23, "Masculino", "25/05/1994");

O construturor será chamado durante a criação do objeto de uma determinada classe, podemos passar parâmetros para a classe e executar determinadas validações ou funções dentro desse contexto.

```
constructor(nome, idade, sexo, dataDeNascimento) {
if (nome === undefined) {
   throw new Error("O nome é obrigatório");
}
else{
   this.nome = nome; }
   this.idade = idade;
   this.sexo = sexo;
   this.dataDeNascimento = dataDeNascimento;
}
```

CRIANDO FUNÇÕES ESTÁTICAS

Além de poder chamar funções criadas para nossos objetos, também podemos definir outro tipo de função: as estáticas. De forma resumida, uma função estática é diferente pois pode ser chamada e invocada sem instânciar e criar um objeto.

CRIANDO FUNÇÕES ESTÁTICAS

A palavra chave **static** define um método estático para a classe. Métodos estáticos não são chamados na instâncias da classe. Em vez disso, eles são chamados na própria classe. Geralmente, são funções utilitárias, como funções para criar ou clonar objetos

CRIANDO FUNÇÕES ESTÁTICAS

Chamadas a métodos estáticos são feitas diretamente na classe e não podem ser feitas em uma instância da classe. Métodos estáticos são comumente utilizados como funções utilitárias.

Para chamar um método estático dentro de outro método estático da mesma classe, podemos utilizar a palavra reservada this.

```
class ChamadaDoMetodoEstatico {
  static metodoEstatico() {
    return "O método estático foi chamado";
  }
  static outroMetodoEstatico() {
    return this.metodoEstatico() + " de outro método estático";
  }
}
ChamadaDoMetodoEstatico.metodoEstatico();
// 'O método estático foi chamado'

ChamadaDoMetodoEstatico.outroMetodoEstatico();
// 'O método estático foi chamado de outro método estático'
```

CRIANDO FUNÇÕES ESTÁTICAS

Métodos estáticos não são diretamente acessíveis utilizando-se this a partir de métodos não estáticos. É necessário chamá-los usando o nome da classe:

```
nomeDaClasse.nomeDoMetodoEstatico();
```

ou chamando o método como uma propriedade do construtor

```
this.constructor.nomeDoMetodoEstatico().
```

CRIANDO FUNÇÕES ESTÁTICAS

```
class ChamadaDoMetodoEstatico {
  constructor() {
    console.log(ChamadaDoMetodoEstatico.MetodoEstatico());
    // 'O método estático foi chamado.'

    console.log(this.constructor.MetodoEstatico());
    // 'O método estático foi chamado.'
  }

  static MetodoEstatico() {
    return "O método estático foi chamado.";
  }
}
```

Programiz
JavaScript Online Compiler

main.js

Share Run

```
1  class Pessoa {
2    constructor(nome, idade, sexo, dataDeNascimento) {
3      this.nome = nome;
4      this.idade = idade;
5      this.sexo = sexo;
6      this.dataDeNascimento = dataDeNascimento;
7    }
8    apresentar() {
9      console.log('Ola, meu nome é ',this.nome, 'eu tenho' ,this
           .idade, 'anos');
10   }
11   static tipo(){
12       console.log("método estático")
13   }
14 }
15 Pessoa.tipo();
16 let p1 = new Pessoa("Miguel", 30, "M", "01/01/1995");
17 p1.constructor.tipo();
18
19
```

Output

```
método estático
método estático

=== Code Execution Successful ===
```

CRIANDO FUNÇÕES ESTÁTIC

main.js

Share Run

```
class Pessoa {
    constructor(nome, idade, sexo, dataDeNascimento) {
        if (nome === undefined) {
            throw new Error("O nome é obrigatório");
        } else {
            this.nome = nome;
        }
        this.idade = idade;
        this.sexo = sexo;
        this.dataDeNascimento = dataDeNascimento;
    }

    apresentar() {
        console.log(`Olá, meu nome é ${this.nome}, eu tenho ${this.idade} anos`);
    }

    static tipo() {
        console.log("Função estática");
    }
}

// Criando uma instância da classe Pessoa
const pessoa1 = new Pessoa("Ana", 30, "Feminino", "1994-05-07");

// Chamando o método de instância
pessoa1.apresentar();

// Chamando o método estático
Pessoa.tipo();
```

Output

```
Olá, meu nome é Ana, eu tenho 30 anos
Função estática

=== Code Execution Successful ===
```

```javascript
class FiguraGeometrica {
  static calcularAreaCirculo(raio) {
    return Math.PI * raio * raio;
  }
  static calcularAreaQuadrado(lado) {
    return lado * lado;
  }
   static calcularAreaRetangulo(largura, altura) {
    return largura * altura;
  }
}
// Declaração das variáveis
const raio = 3;
const lado = 2;
const largura = 4;
const altura = 3;
console.log('Área do círculo com raio raio:', FiguraGeometrica.calcularAreaCirculo
    (raio));
console.log('Área do quadrado com lado lado:', FiguraGeometrica.calcularAreaQuadrado
    (lado));
console.log('Área do retângulo com largura', largura ,'e altura', altura
    ,FiguraGeometrica.calcularAreaRetangulo(largura, altura));

```

```
node /tmp/7ecx69p9UY.js
Área do círculo com raio raio: 28.274333882308138
Área do quadrado com lado lado: 4
Área do retângulo com largura 4 e altura 3 12
```

```javascript
// Definindo a classe Principal
class Calculadora {
  // Método estático para adicionar dois números
  static adicionar(a, b) {
    return a + b;
  }
  // Método estático para subtrair dois números
  static subtrair(a, b) {
    return a - b;
  }

  // Método estático para multiplicar dois números
  static multiplicar(a, b) {
    return a * b;
  }

  // Método estático para dividir dois números
  static dividir(a, b) {
    if (b === 0) {
      throw new Error("Divisão por zero não é permitida.");
    }
    return a / b;
  }
}
```

```
node /tmp/1xkqd2YA0K.js
Soma a e b: 7
Potência de 4 elevado a 6 é = 4096
```

```
// Definindo a subclasse CalculadoraCientifica que herda de Calculadora
class CalculadoraCientifica extends Calculadora {
  // Método estático para calcular a potência
  static potencia(base, expoente) {
    return Math.pow(base, expoente);
  }


  static raizQuadrada(numero) {
    if (numero < 0) {
      throw new Error("Não é possível calcular a raiz quadrada de um número negativo.");
    }
    return Math.sqrt(numero);
  }

  // Método estático para calcular o seno de um ângulo (em radianos)
  static seno(angulo) {
    return Math.sin(angulo);
  }

```

```
46    // Método estático para calcular o cosseno de um ângulo (em radianos)
47    static cosseno(angulo) {
48      return Math.cos(angulo);
49    }
50  }
51
52  // Utilizando os métodos estáticos das classes Calculadora e CalculadoraCientifica
53  const a = 5;
54  const b = 2;
55  const base = 4;
56  const expoente = 6;
57  const angulo = Math.PI / 4;
58
59  console.log('Soma a e b:', Calculadora.adicionar(a, b));
60  console.log('Potência de', base, 'elevado a',expoente,'é =', CalculadoraCientifica
        .potencia(base, expoente));
61
```

TRABALHO DE BIBLIOTECAS



Miguel Enrique Parra Muñoz

CONTEÚDO DA AULA

Bibliotecas em Javascript

Bibliotecas em C

Bibliotecas em python





BIBLIOTECA EM JAVASCRIPT

ESTRUTURA DO PROJETO

Uma biblioteca em JavaScript é utilizada para organizar funções em arquivos separados, permitindo reutilização de código em diferentes programas. Assim como em C utilizamos arquivos .h e .c, em JavaScript utilizamos módulos.

Exemplo de estrutura:

```
projeto/
│
├── calculos.js
├── main.js
└── package.json
```

PROCEDIMENTO:

Criar três arquivos:

cálculos.js : Este arquivo funciona como a biblioteca do projeto. Nele ficam armazenadas as funções matemáticas que poderão ser reutilizadas em outros programas.

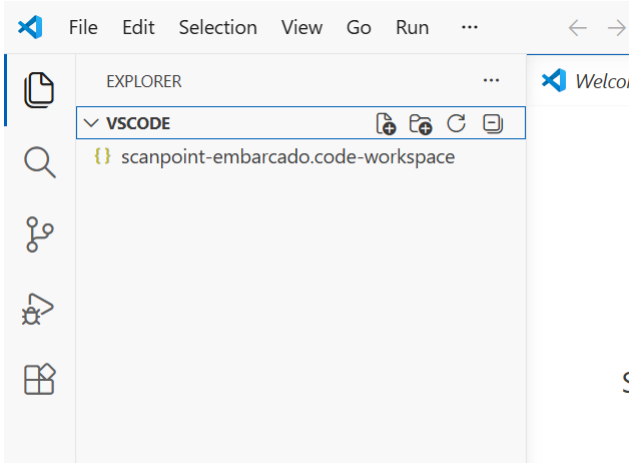
Principal.js : Este é o programa principal. Ele importa a biblioteca calculos.js, solicita dados ao usuário e utiliza as funções criadas

Package.json: O arquivo package.json é o arquivo responsável por armazenar as informações e configurações de um projeto Node.js.

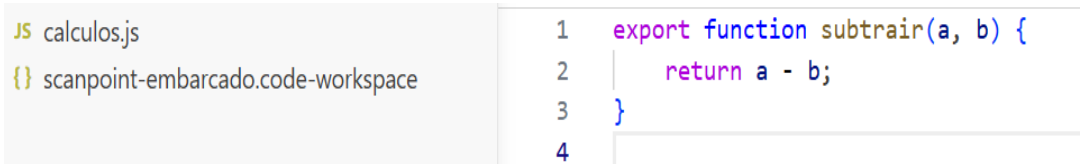


JAVASCRIPT

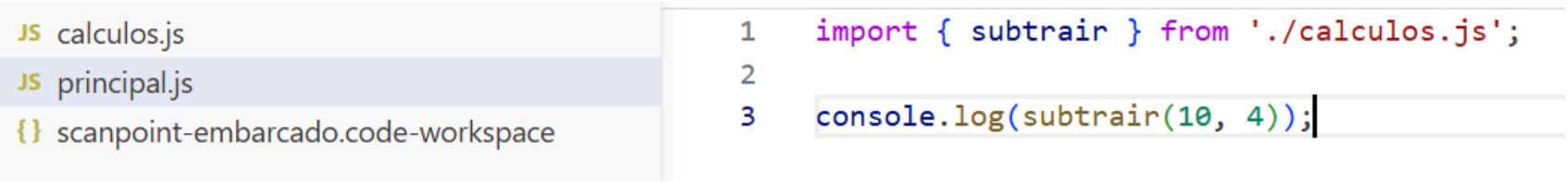
Criar a pasta

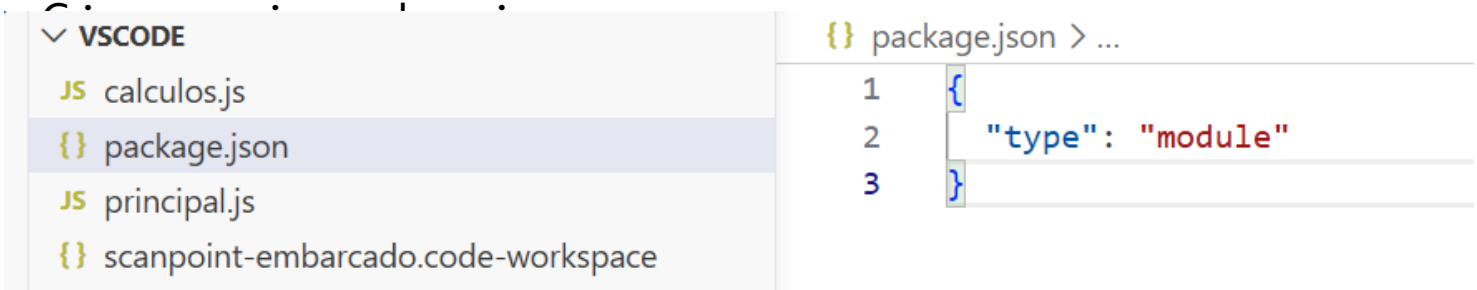


- Colocar o nome da biblioteca e escrever o arquivo



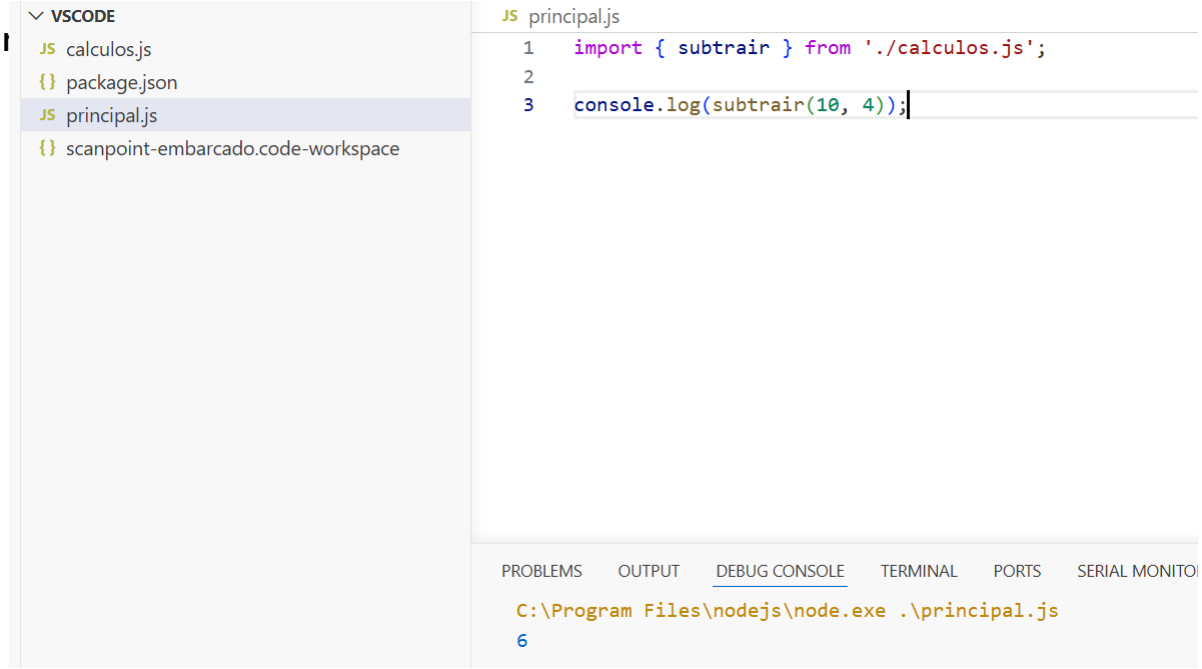
JAVASCRIPT


calculos.js
principal.js
scanpoint-embarcado.code-workspace
1 import { subtrair } from './calculos.js';
2
3 console.log(subtrair(10, 4));

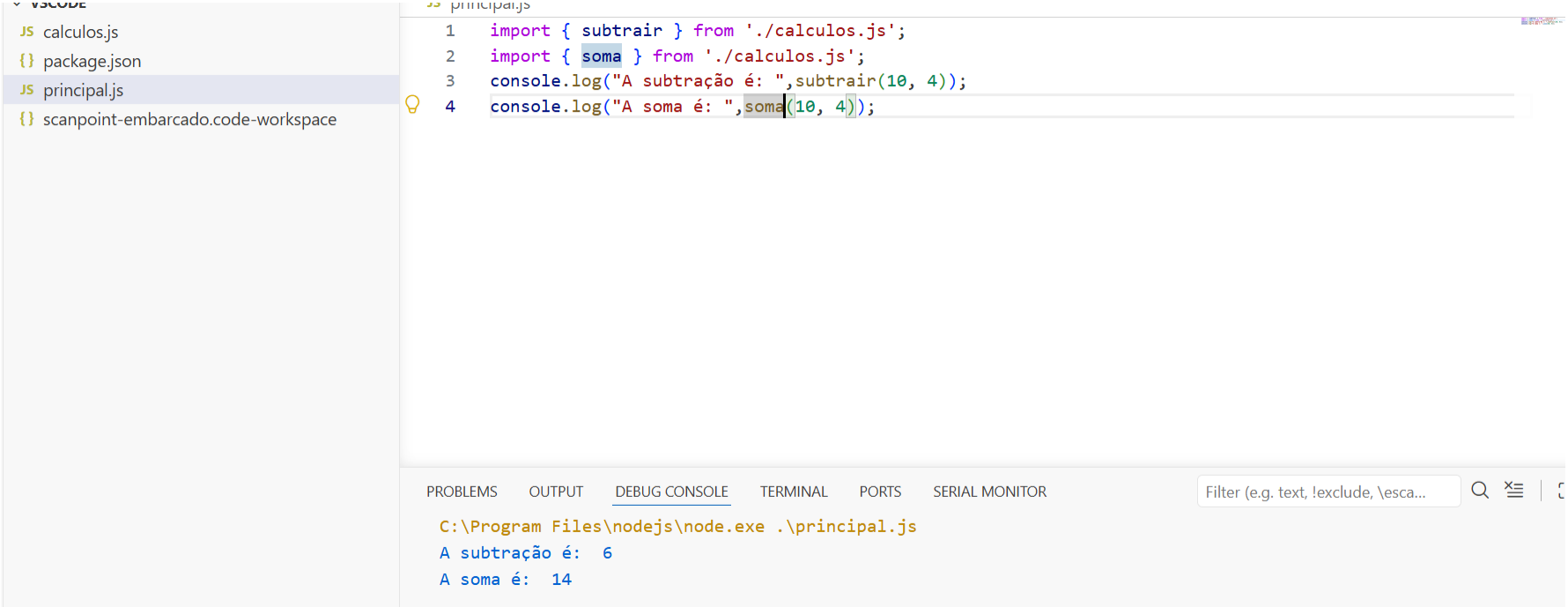

VSCODE
calculos.js
package.json
principal.js
scanpoint-embarcado.code-workspace
package.json > ...
1 {
2 "type": "module"
3 }

JAVASCRIPT

Compilar



ADICIONE À BIBLIOTECA 5 NOVAS FUNÇÕES



BIBLIOTECAS EM C

EM C

Quando criamos uma biblioteca em C, normalmente dividimos o projeto em três arquivos:

main.c
calculos.h
calculos.c



EM C

Arquivo main.c

O main.c é o programa principal.
Ele é responsável por:

executar o programa;

solicitar dados ao usuário;

chamar as funções da biblioteca;

mostrar os resultados na tela.

EM C

Arquivo calculos.h

O arquivo .h é chamado de:

Header File

ou:

Arquivo de Cabeçalho

Ele serve para declarar as funções da biblioteca.

EM C

Arquivo calculos.c

O arquivo .c contém a implementação das funções.

Ou seja:

o código real das operações;

a lógica da biblioteca.

EM C:

C: > Users > migue > OneDrive > Escritorio > VScode > codigoC > C calculos.h > .

```c
#ifndef CALCULOS_H
#define CALCULOS_H

float somar(float a, float b);
float subtrair(float a, float b);
#endif
```

C: > Users > migue > OneDrive > Escritorio > VScode > codigoC > C calculos.c > subtrair(float, float)

```c
#include "calculos.h"

float somar(float a, float b)
{
    return a + b;
}
float subtrair(float a, float b)
{
    return a - b;
}
```

main.c ● | Extension: C/C++ Compile Run | calculos.h | calculos.c

C: > Users > migue > OneDrive > Escritorio > VScode > codigoC > main.c > main()

```c
#include <stdio.h>
#include "calculos.h"

int main()
{
    float resultado,a,b,resultado1;
    printf("digite o primeiro número: ");
    scanf("%f",&a);
    printf("digite o segundo número: ");
    scanf("%f",&b);

    resultado = somar(a, b);
    resultado1 = subtrair(a, b);
    printf("Resultado soma: %f\n", resultado);
    printf("Resultado subtração: %f\n", resultado1);
    return 0;
}
```

COMPILAÇÃO

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS SERIAL MONITOR

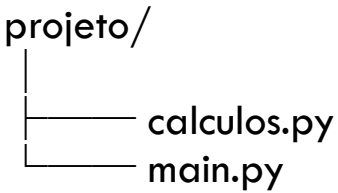
```
digite o segundo n├╢mero: 4
Resultado: 7.000000
PS C:\Users\migue\OneDrive\Escritorio\VScode\codigoC>
PS C:\Users\migue\OneDrive\Escritorio\VScode\codigoC>
PS C:\Users\migue\OneDrive\Escritorio\VScode\codigoC> gcc main.c calculos.c -o main
PS C:\Users\migue\OneDrive\Escritorio\VScode\codigoC> .\main.exe
digite o primeiro numero: 2
digite o segundo numero: 4
Resultado soma: 6.000000
Resultado subtra├║├úo: -2.000000
PS C:\Users\migue\OneDrive\Escritorio\VScode\codigoC>
```

BIBLIOTECA EM PYTHON



BIBLIOTECA EM PYTHON

Estrutura do Projeto em Python

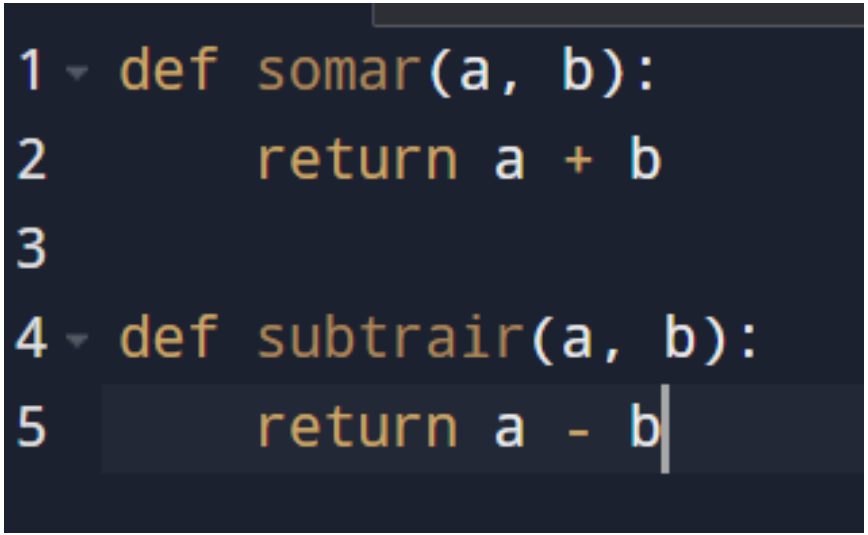


BIBLIOTECA EM PYTHON

Arquivo calculos.py

Esse arquivo funciona como a biblioteca.

Código



BIBLIOTECA EM PYTHON

Arquivo main.py

Esse é o programa principal.

```python
import calculos

a = float(input("Digite o primeiro número: "))
b = float(input("Digite o segundo número: "))

resultado_soma = calculos.somar(a, b)
resultado_subtracao = calculos.subtrair(a, b)

print("Resultado soma:", resultado_soma)
print("Resultado subtração:", resultado_subtracao)
```

BIBLIOTECA EM PYTHON

```python
import calculos

a = float(input("Digite o primeiro número: "))
b = float(input("Digite o segundo número: "))

resultado_soma = calculos.somar(a, b)
resultado_subtracao = calculos.subtrair(a, b)

print("Resultado soma:", resultado_soma)
print("Resultado subtração:", resultado_subtracao)
```

PROBLEMS OUTPUT TERMINAL ··· Python

```
PS C:\Users\migue\OneDrive\Escritorio\VScode\projetopython> & C:/Py
.exe c:/Users/migue/OneDrive/Escritorio/VScode/projetopython/main.p
Digite o primeiro número: 3
Digite o segundo número: 4
Resultado soma: 7.0
Resultado subtração: -1.0
PS C:\Users\migue\OneDrive\Escritorio\VScode\projetopython>
```

RESUMO

Linguagem	Biblioteca	Importação	Compilação
C	`.h + .c`	`#include`	Necessária
JavaScript	`.js`	`require()`	Não
Python	`.py`	`import`	Não